

GUÍA INTERACTIVA

React moderno

Repaso de hooks y React 19 a fondo, hasta React 19.3 y el React Compiler

Versión PDF · 28 de octubre de 2026
31 capítulos · Las demos interactivas están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía
2. Qué cambió desde el React "legacy"

Parte 1 · Fundamentos

3. Componentes, JSX y props
4. Estado con useState
5. Estado derivado y levantar el estado

Parte 2 · Hooks a fondo

6. useEffect: sincronizar con el exterior
7. useRef: el DOM y valores que no renderizan
8. useContext: datos sin prop drilling
9. useReducer: lógica de estado compleja
10. Custom hooks
11. memo, useMemo y useCallback
12. useTransition y useDeferredValue
13. Suspense, lazy y Error Boundaries
14. useId, useLayoutEffect, portales y más

Parte 3 · React 19 a fondo

15. Actions: la gran idea de React 19 · 19.0
16. useActionState · 19.0
17. Formularios con action y useFormStatus · 19.0
18. useOptimistic · 19.0
19. use(): promesas y contexto · 19.0

20. ref como prop y <Context> como provider · 19.0
21. Metadatos, estilos y recursos · 19.0
22. Errores y migración desde código legacy · 19.0

Parte 4 · React 19.1 → 19.3 y el Compiler

23. <Activity> · 19.2
24. useEffectEvent · 19.2
25. <ViewTransition> y addTransitionType · 19.3
26. Fragment refs · 19.3
27. React Compiler · Compiler 1.0
28. Otras novedades de 19.1 a 19.3 · 19.1+

Parte 5 · Puente a Next.js

29. Server Components · 19.0
30. Server Functions y formularios · 19.0
31. Chuleta y preguntas de entrevista

Empezá acá

Cómo está armada la guía y qué cambió en React mientras trabajabas con código legacy.

Capítulo 1

Cómo usar esta guía

Cómo correr el proyecto, dónde está cada demo y cómo aprovecharla.

Esta guía es una app de React hecha con Vite. Cada capítulo explica un tema, te muestra una **demo funcionando** y, debajo, el **código fuente real** de esa demo.

El código no es una copia pegada a mano: la guía lee el mismo archivo que se está ejecutando (con el sufijo `?raw` de Vite). Si editás el archivo de una demo, cambian a la vez la demo y el snippet. Así podés leer el código en la guía o directamente en tu editor.

Cómo correrla

```
npm install
npm run dev
```

Abrí la URL que muestra la terminal (normalmente `http://localhost:5173`).

Para verla en el celular, conectado a la misma red Wi-Fi, corré `npm run dev -- --host` y abrí desde el teléfono la dirección `Network` que aparece en la terminal.

Cómo está organizado el proyecto

Carpeta o archivo	Qué hay
`src/content/`	Los capítulos, en MDX (Markdown con JSX). Se leen bien directo en el editor.
`src/demos/`	Una carpeta por parte y un archivo por demo. **Este es el código para practicar.**
`src/components/`	La maquinaria de la guía: layout, bloques de código y el componente `<Demo>`.
`src/chapters.ts`	El índice: orden, títulos y versión de React de cada capítulo.

Carpeta o archivo	Qué hay
`vite.config.ts`	Vite con MDX y el React Compiler (en modo `annotation`, lo vemos en su capítulo).
`scripts/export-pdf.mjs`	El script que genera el PDF (`npm run pdf`).
`docs/`	El PDF de la guía, listo para leer en el celular.

Cómo aprovecharla

1. Leé la explicación, tocá la demo y recién después mirá el código.
2. En cada recuadro 🧪 **Probá esto** hay un mini ejercicio: modificá el archivo de la demo y mirá el cambio al instante (Vite recarga solo).
3. Si rompés algo, usá el botón ↻ **Reiniciar** de la demo o volvé el archivo atrás con git.
4. Los recuadros 🎤 **En una entrevista** resumen cómo explicar el tema en voz alta, en dos o tres frases.

Los recuadros que vas a ver:

> ℹ️ **Nota**
> Contexto o detalle extra.

> ⚠️ **Ojo**
> Un error común o una trampa.

> 💡 **Tip**
> Una forma mejor o más moderna de hacer algo.

> 🧪 **Probá esto**
> Un ejercicio corto sobre la demo.

> 🎤 **En una entrevista**
> Cómo responder si te lo preguntan.

Ruta sugerida

Si tenés...	Hacé esto
1 hora	Leé el capítulo 2 (qué cambió) y recorré la Parte 3 en diagonal.
2 o 3 días	Partes 1 y 2 completas: es el repaso de hooks, con todas las demos.
1 semana	Partes 3 y 4 con todos los ejercicios: React 19 a fondo.

Si tenés...	Hacé esto
Antes de Next.js	Parte 5 y después la Misión 1 del Job Tracker.

Muchas demos usan el mismo tema: **postulaciones a trabajos** (empresa, puesto, estado). Es a propósito: es el mismo dominio del Job Tracker que vas a construir en Next.js, así que los patrones te van a quedar a mano.

La app usa lo mismo que enseña

- El cambio de capítulo usa `<ViewTransition>` y `startTransition` (mirá `src/App.tsx`).
- El título de la pestaña lo pone un `<title>` renderizado dentro de un componente, una novedad de React 19.
- El tema claro u oscuro de los bloques de código usa `useSyncExternalStore` con una media query (`src/lib/useColorScheme.ts`).
- El PDF sale de esta misma app, abierta con `?print` en la URL.

La versión PDF

Arriba a la derecha está el link **Versión imprimible**: abre la guía completa en una sola página, lista para imprimir o guardar como PDF desde el navegador.

El PDF que acompaña al proyecto (`docs/guia-react-moderno.pdf`) sale de esa misma página. Si cambiás algo y lo querés regenerar, corré `npm run pdf` : arma el build, lo abre en un Chromium automático (con Playwright) y lo imprime con un tamaño de página pensado para el celular. La primera vez hace falta descargar ese navegador con `npx playwright install chromium` .

> ℹ️ **Versiones**
>
> Escrita en octubre de 2026 con React 19.3.1, Vite 8, TypeScript 7 y React Compiler 1.0.

Capítulo 2

Qué cambió desde el React "legacy"

De las clases al React Compiler: la línea de tiempo y los cambios de mentalidad.

Si tu último contacto fuerte con React fue código de hace unos años (clases, `componentDidMount`, Redux con mucho boilerplate, Create React App), esto es lo que pasó mientras tanto. La buena noticia: **lo fundamental no cambió**. Componentes, props, estado y hooks siguen siendo la base. Lo que cambió es cómo se piden los datos, cómo se manejan los formularios y cuánto trabajo hace React por vos.

Línea de tiempo

Fecha	Versión	Lo más importante
2019	16.8	Llegan los **hooks**: `useState`, `useEffect`, `useContext`... Las clases dejan de ser necesarias.

Fecha	Versión	Lo más importante
2020	17	Versión puente, sin features nuevas. Nuevo JSX transform: ya no hace falta `import React`.
2022	18	**Renderizado concurrente**: `createRoot`, batching automático, `useTransition`, `useDeferredValue`, `useId`, `useSyncExternalStore`.
Dic 2024	19	**Actions**: `useActionState`, `useOptimistic`, `<form action>`, `use()`, `ref` como prop, `<title>` en componentes, Server Components estables.
Feb 2025	—	Create React App queda deprecado. La recomendación es Vite o un framework (Next.js, React Router).
Mar 2025	19.1	Owner Stack para depurar y mejoras en Suspense.
Oct 2025	19.2	`<Activity>`, `useEffectEvent`, Performance Tracks, prerender parcial.
Oct 2025	Compiler 1.0	El **React Compiler** pasa a estable: memoización automática.
Sep 2026	19.3	`<ViewTransition>`, `addTransitionType`, Fragment refs y `browser()`.

Los seis cambios de mentalidad

1. **De clases a funciones.** Todo componente nuevo es una función. Las clases sobreviven solo para

los Error Boundaries.

2. **De "ciclo de vida" a "sincronización".** Ya no pensás "esto corre cuando se monta", sino "este efecto mantiene X sincronizado con Y". Lo vemos en el capítulo de useEffect.
3. **De fetch en `useEffect` a datos con Suspense.** En apps nuevas los datos se piden en el servidor (Server Components), en el router o con librerías como TanStack Query. El fetch en un efecto queda como último recurso.
4. **De `isLoading` a mano a Actions.** React 19 maneja el estado pendiente, los errores y las actualizaciones optimistas por vos.
5. **De `useMemo` en todos lados al React Compiler.** El compilador memoiza automáticamente; escribís código simple y él lo optimiza.
6. **De SPA pura a "server-first".** En Next.js con App Router los componentes son del servidor por defecto, y el cliente se usa solo donde hace falta interactividad.

Arrancar una app hoy

× React 17 y anteriores

```
import React from 'react'
import ReactDOM from 'react-dom'
import App from './App'
```

```
ReactDOM.render(<App />,
  document.getElementById('root'))
```

✔ React 18 y 19

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`ReactDOM.render` ya no existe en React 19. `createRoot` es lo que activa todas las features concurrentes (transiciones, Suspense, Actions).

Para crear un proyecto nuevo:

```
# SPA con Vite (como esta guía)
npm create vite@latest mi-app -- --template react-ts

# App full stack con Next.js
npx create-next-app@latest mi-app
```

Una clase y su equivalente moderno

× Componente de clase

```
class Contador extends React.Component {
  state = { count: 0 }

  componentDidMount() {
    document.title = `Clicks: ${this.state.count}`
  }

  componentDidUpdate() {
    document.title = `Clicks: ${this.state.count}`
  }

  render() {
    return (
      <button onClick={() => this.setState({ count:
        this.state.count + 1 })}>
        {this.state.count}
      </button>
    )
  }
}
```

✓ Función con hooks

```
function Contador() {
  const [count, setCount] = useState(0)

  useEffect(() => {
    document.title = `Clicks: ${count}`
  }, [count])
```

```
  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  )
}
```

Al final de la guía hay una chuleta con la equivalencia de cada método de clase.

> **🎤 En una entrevista**
>
> "¿Qué cambió en React 19?" La respuesta corta: "La gran novedad son las Actions, funciones async dentro de transiciones que manejan solas el estado pendiente, los errores y las actualizaciones optimistas. Alrededor de eso vienen `useActionState`, `useOptimistic`, `useFormStatus` y los formularios con `action`. También se estabilizaron los Server Components, apareció `use()` para leer promesas y contexto, y `ref` pasó a ser una prop común. Y desde 2025 el React Compiler hace la memoización automáticamente."

Parte 1 · Fundamentos

Repaso rápido de lo que no cambió: componentes, props, estado y cómo pensar en React.

Capítulo 3

Componentes, JSX y props

Funciones que devuelven UI, props, children, condicionales y listas con key.

Un componente de React es **una función que devuelve UI**. Recibe datos por parámetro (las props) y devuelve JSX. Nada más: no hay clases, no hay `this`, no hay `render()`.

```
function Saludo({ nombre }: { nombre: string }) {
  return <h1>Hola, {nombre}</h1>
}

// Se usa como una etiqueta HTML:
<Saludo nombre="Ana" />
```

JSX en 6 reglas

1. Los componentes empiezan con **mayúscula** (`<Saludo />`); las etiquetas HTML, con minúscula (`<div>`).
2. Devolvés **un solo elemento raíz**. Si no querés un `div` extra, usá un Fragment: `<>...</>`.
3. Todas las etiquetas se cierran: `<img />`, `<input />`, `<br />`.
4. Los atributos van en camelCase: `className`, `onClick`, `htmlFor`, `tabIndex`.
5. Entre llaves `{}` va **cualquier expresión** de JavaScript: variables, llamadas a funciones, ternarios. No van sentencias (`if`, `for`).
6. `style` recibe un objeto: `style={{ marginTop: 8 }}`.

Props

Las props son los argumentos del componente. Son **de solo lectura**: un componente nunca modifica sus props.

- Se desestructuran en la firma de la función.
- Los **valores por defecto** se ponen en la desestructuración. `defaultProps` ya no funciona en componentes de función en React 19.

- `children` es una prop especial: lo que va entre la etiqueta de apertura y la de cierre.
- Podés pasar cualquier valor: strings, números, objetos, funciones y hasta otros componentes.

DEMO **Props, children y valores por defecto**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/ProfileCard.tsx`

```tsx
import type { ReactNode } from 'react'

// Un componente es una función que recibe props y
  devuelve JSX.
type BadgeProps = {
  label: string
  tone?: 'neutral' | 'success'
}

function Badge({ label, tone = 'neutral' }:
  BadgeProps) {
  // Valor por defecto con destructuring: reemplaza
    a defaultProps.
  return <span className={`pill ${tone === 'success'
    ? 'ok-text' : ''}`}>{label}</span>
}

type ProfileCardProps = {
  name: string
  role: string
  openToWork?: boolean
```

```
  children?: ReactNode // lo que va entre
    <ProfileCard> y </ProfileCard>
}

function ProfileCard({ name, role, openToWork =
  false, children }: ProfileCardProps) {
  return (
    <article className="card">
      <h3 style={{ margin: 0 }}>{name}</h3>
      <p className="muted">{role}</p>
      {/* Render condicional con && */}
      {openToWork && <Badge label="Open to work"
        tone="success" />}
      {children}
    </article>
  )
}

export default function ProfileCardDemo() {
  return (
    <div className="grid-2">
      <ProfileCard name="Ana Rojas" role="Frontend
        Engineer · React" openToWork>
        <p>Asunción · 100% remoto</p>
      </ProfileCard>
      <ProfileCard name="Bruno Díaz" role="Full
        Stack Engineer" />
    </div>
  )
}
```

× PropTypes y defaultProps

```
function Badge({ label, tone }) {
  return <span className={tone}>{label}</span>
}
```

```
Badge.propTypes = {
  label: PropTypes.string.isRequired,
  tone: PropTypes.oneOf(['neutral', 'success']),
}

Badge.defaultProps = { tone: 'neutral' }
```

✓ TypeScript y parámetros por defecto

```
type BadgeProps = {
  label: string
  tone?: 'neutral' | 'success'
}

function Badge({ label, tone = 'neutral' }:
  BadgeProps) {
  return <span className={tone}>{label}</span>
}
```

`propTypes` se eliminó en React 19 (se ignora en silencio). Hoy el chequeo de tipos lo hace TypeScript, en tiempo de compilación.

💡 Tipos útiles de TypeScript

- `ReactNode`: cualquier cosa que React pueda renderizar (para `children`).
- `ComponentProps<'button'>`: todas las props de un `<button>`, ideal para componentes que envuelven un elemento HTML.
- `ComponentProps<typeof MiComponente>`: las props de otro componente.

```
import type { ComponentProps } from 'react'

function Button({ variant = 'primary', ...rest
  }: ComponentProps<'button'> & { variant?:
  string }) {
  return <button className={`btn
    btn-${variant}`} {...rest} />
}
```

Renderizado condicional

Como JSX es JavaScript, usás las herramientas de siempre:

```
// 1. Return temprano
if (!user) return <Login />

// 2. Ternario: una cosa u otra
{isLoading ? <Spinner /> : <Results />}
```

```
// 3. && : algo o nada
{hasError && <ErrorMessage />}
```

> ⚠️ **El 0 que aparece de la nada**
>
> `{count && <Badge />}` renderiza un **0** cuando `count` vale 0, porque `0 && x` devuelve `0` y React lo muestra. Usá una comparación explícita: `{count > 0 && <Badge />}`.

Listas y keys

Para renderizar una lista, transformás un array en un array de elementos con `map`. Cada elemento necesita una `key` **única y estable** entre sus hermanos, para que React sepa cuál es cuál entre un render y el siguiente.

DEMO **Lista filtrada con key**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/JobList.tsx`

```
import { useState } from 'react'
```

```tsx
type Job = {
  id: number
  company: string
  role: string
  remote: boolean
  salary: number
}

const jobs: Job[] = [
  { id: 1, company: 'Nubelia', role: 'Frontend
    Engineer', remote: true, salary: 3200 },
  { id: 2, company: 'Pagos Sur', role: 'React
    Developer', remote: false, salary: 2800 },
  { id: 3, company: 'Datalab', role: 'Senior
    Frontend', remote: true, salary: 4500 },
  { id: 4, company: 'Kuarahy', role: 'UI Engineer',
    remote: true, salary: 2600 },
]

export default function JobList() {
  const [onlyRemote, setOnlyRemote] =
    useState(false)
  // Dato derivado: se calcula en cada render, no se
    guarda en otro estado.
  const visibleJobs = onlyRemote ? jobs.filter((job)
    => job.remote) : jobs

  return (
    <div className="stack">
      <label className="row">
        <input
          type="checkbox"
          checked={onlyRemote}
          onChange={(event) =>
            setOnlyRemote(event.target.checked)}
        />
        Solo remotos
      </label>
```

```
      {visibleJobs.length === 0 ? (
        <p>No hay ofertas.</p>
      ) : (
        <ul style={{ margin: 0, paddingLeft: '1.2em'
          }}>
          {visibleJobs.map((job) => (
            // key: un id estable que identifica a
              cada elemento de la lista
            <li key={job.id}>
              <strong>{job.role}</strong> en
                {job.company} · USD
                {job.salary.toLocaleString('es')}
              {job.remote ? ' · 🌎 Remoto' : ' · 🏢
                Presencial'}
            </li>
          ))}
        </ul>
      )}

      <p className="muted">
        Mostrando {visibleJobs.length} de
          {jobs.length} ofertas
      </p>
    </div>
  )
}
```

¿Por qué no usar el índice del array como key? Porque si la lista se reordena o se agrega un elemento al principio, el índice deja de identificar al mismo elemento. React reutiliza el componente equivocado y el estado (el texto de un input, por ejemplo) queda en la fila equivocada.

DEMO **El bug de usar el índice como key**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/KeysDemo.tsx`

```tsx
import { useState } from 'react'

type Company = { id: number; name: string }

const initialCompanies: Company[] = [
  { id: 1, name: 'Nubelia' },
  { id: 2, name: 'Pagos Sur' },
  { id: 3, name: 'Datalab' },
]

export default function KeysDemo() {
  const [companies, setCompanies] =
    useState(initialCompanies)
  const [indexAsKey, setIndexAsKey] = useState(true)

  function addOnTop() {
    const id = Date.now()
    setCompanies([{ id, name: `Empresa nueva
      #${companies.length + 1}` }, ...companies])
  }

  return (
    <div className="stack">
      <div className="row">
        <button onClick={addOnTop}>Agregar
          arriba</button>
        <label className="row">
          <input
            type="checkbox"
```

```
            checked={indexAsKey}
            onChange={(event) =>
              setIndexAsKey(event.target.checked)}
          />
          Usar el índice como key (mal)
        </label>
      </div>

      <ul className="stack" style={{ listStyle:
        'none', padding: 0, margin: 0 }}>
        {companies.map((company, index) => (
          <li key={indexAsKey ? index : company.id}
            className="row">
            <span style={{ minWidth: 170 }}>
              {company.name}</span>
            {/* Este input no está controlado: su
              texto vive en el DOM */}
            <input placeholder="Notas sobre la
              empresa…" />
          </li>
        ))}
      </ul>
    </div>
  )
}
```

🧪 Probá esto

Escribí algo en las notas de "Nubelia" y tocá **Agregar arriba** con la opción "índice como key" activada: la nota queda en la fila de la empresa nueva. Desactivá la opción, reiniciá la demo y repetí: ahora cada nota sigue a su empresa.

Buenas keys: el `id` que viene de la base de datos, un `crypto.randomUUID()` generado **al crear** el dato (nunca durante el render) o una combinación de campos que sea única.

Los componentes son puros

React asume que tus componentes son **funciones puras**: con las mismas props, el mismo estado y el mismo contexto, devuelven siempre el mismo JSX, y no modifican nada que exista fuera de ellos mientras renderizan.

```
let visitas = 0

function Contador() {
  visitas++ // 🔴 efecto secundario durante el
    render
  return <p>Visitas: {visitas}</p>
}
```

Los efectos secundarios (pedir datos, modificar el DOM a mano, suscribirse a algo) van en **event handlers** o, si no hay otra opción, en un **Effect**. En desarrollo, `<StrictMode>` renderiza cada componente dos veces justamente para detectar componentes impuros.

🎤 En una entrevista

"¿Qué es una key y por qué importa?" → "Es el identificador que React usa para saber qué elemento de una lista es cuál entre renders. Con keys estables React puede mover, agregar o borrar elementos sin perder su estado. Con el índice como key, si la lista se reordena, el estado queda asociado al elemento equivocado."

Capítulo 4

Estado con useState

El estado como foto, la función actualizadora e inmutabilidad en objetos y arrays.

El **estado** es la memoria de un componente: datos que cambian con el tiempo (lo que escribió el usuario, si un menú está abierto, la lista de tareas). Cuando el estado cambia, React vuelve a ejecutar el componente y actualiza la pantalla.

```
const [count, setCount] = useState(0)
//     ↑ valor   ↑ función para cambiarlo   ↑ valor
  inicial
```

¿Por qué no alcanza con una variable común? Porque una variable local se reinicia en cada render y, además, cambiarla no le avisa a React que tiene que volver a renderizar.

El estado es una foto

Esta es la idea más importante del capítulo. Dentro de un render, `count` **es una constante**. Llamar a `setCount` no cambia la variable: le pide a React un nuevo render, donde `count` va a tener el valor nuevo.

DEMO **El estado como foto y la función actualizadora**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/Counter.tsx`

```
import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  function addThreeWrong() {
    // count vale lo mismo en las 3 líneas: es una
      "foto" del render actual.
    setCount(count + 1) // setCount(0 + 1)
    setCount(count + 1) // setCount(0 + 1)
    setCount(count + 1) // setCount(0 + 1)
  }

  function addThreeRight() {
    // La función actualizadora recibe el valor más
      reciente (c).
    setCount((c) => c + 1)
```

```
    setCount((c) => c + 1)
    setCount((c) => c + 1)
  }

  return (
    <div className="stack">
      <p style={{ fontSize: '2.2rem', fontWeight:
        700, margin: 0 }}>{count}</p>
      <div className="row">
        <button onClick={() => setCount(count +
          1)}>+1</button>
        <button onClick={addThreeWrong}>+3 (mal)
          </button>
        <button onClick={addThreeRight}>+3 (bien)
          </button>
        <button onClick={() =>
          setCount(0)}>Reset</button>
      </div>
    </div>
  )
}
```

El botón "+3 (mal)" suma solo 1 porque las tres líneas leen el mismo `count` de esa foto: es como escribir `setCount(0 + 1)` tres veces. Con la **función actualizadora** `setCount(c => c + 1)`, React encola las funciones y le pasa a cada una el resultado de la anterior.

> 💡 **Regla práctica**
>
> Si el valor nuevo depende del anterior, usá la función actualizadora: `setX(prev => ...)`. Es más segura y además es la única opción correcta cuando hay varias actualizaciones seguidas o código async.

React además **agrupa (batching)** todas las actualizaciones de un mismo evento en un solo render. Desde React 18 eso pasa en todos lados: handlers, timeouts, promesas.

Objetos: nunca mutes, reemplazá

React compara el estado viejo con el nuevo usando `Object.is`. Si mutás un objeto (`form.company = 'X'`), es el mismo objeto, React no ve ningún cambio y no vuelve a renderizar. Siempre creás un objeto nuevo con spread.

DEMO **Objeto en el estado e inputs controlados**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/fundamentos/ApplicationForm.tsx

```tsx
import { useState } from 'react'

type ApplicationForm = {
  company: string
  role: string
  remote: boolean
  salary: number
}

export default function ApplicationFormDemo() {
  const [form, setForm] = useState<ApplicationForm>
    ({
    company: '',
    role: 'Frontend Engineer',
    remote: true,
    salary: 3000,
  })

  // Nunca mutes el objeto (form.company = ...).
    Creá uno nuevo con spread.
  function update<K extends keyof ApplicationForm>
    (key: K, value: ApplicationForm[K]) {
    setForm((prev) => ({ ...prev, [key]: value }))
  }

  return (
    <div className="grid-2">
      <div className="stack">
        {/* Inputs controlados: el valor sale del
          estado y onChange lo actualiza */}
        <label className="stack">
          Empresa
          <input value={form.company} onChange={(e)
            => update('company', e.target.value)} />
        </label>
```

```
        <label className="stack">
          Puesto
          <input value={form.role} onChange={(e) =>
            update('role', e.target.value)} />
        </label>
        <label className="stack">
          Salario (USD)
          <input
            type="number"
            value={form.salary}
            onChange={(e) => update('salary',
              e.target.valueAsNumber || 0)}
          />
        </label>
        <label className="row">
          <input
            type="checkbox"
            checked={form.remote}
            onChange={(e) => update('remote',
              e.target.checked)}
          />
          Remoto
        </label>
      </div>
      <pre className="log" style={{ maxHeight:
        'none' }}>
        {JSON.stringify(form, null, 2)}
      </pre>
    </div>
  )
}
```

Los inputs de esta demo están **controlados**: su `value` sale del estado y `onChange` lo actualiza. El estado es la única fuente de verdad. En React 19 vas

a ver que para muchos formularios ya no hace falta controlar cada input (capítulo de formularios).

Arrays: map, filter y spread

Las operaciones que mutan (`push` , `splice` , `sort` , `reverse` , `arr[i] = x`) están prohibidas sobre el estado. Usá las que devuelven un array nuevo:

Para...	En vez de (muta)	Usá (devuelve uno nuevo)
agregar	`push` , `unshift`	`[...arr, item]` , `[item, ...arr]`
borrar	`splice`	`arr.filter(x => x.id !== id)`
editar	`arr[i] = x`	`arr.map(x => x.id === id ? { ...x, done: true } : x)`
ordenar	`sort` , `reverse`	`arr.toSorted(...)` , `arr.toReversed()`

DEMO **Agregar, editar y borrar sin mutar**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/TodoList.tsx`

```
import { useState, type FormEvent } from 'react'
```

```tsx
type Task = { id: number; text: string; done:
  boolean }

function createInitialTasks(): Task[] {
  return [
    { id: 1, text: 'Actualizar el CV', done: true },
    { id: 2, text: 'Repasar hooks', done: false },
    { id: 3, text: 'Practicar live coding', done:
      false },
  ]
}

export default function TodoList() {
  // Inicialización perezosa: la función se ejecuta
    solo en el primer render.
  const [tasks, setTasks] =
    useState(createInitialTasks)
  const [text, setText] = useState('')

  function handleSubmit(event:
    FormEvent<HTMLFormElement>) {
    event.preventDefault()
    if (!text.trim()) return
    setTasks((prev) => [...prev, { id: Date.now(),
      text: text.trim(), done: false }]) // agregar
    setText('')
  }

  function toggle(id: number) {
    setTasks((prev) => prev.map((t) => (t.id === id
      ? { ...t, done: !t.done } : t))) // editar
  }

  function remove(id: number) {
    setTasks((prev) => prev.filter((t) => t.id !==
      id)) // borrar
  }
```

```
  const pending = tasks.filter((t) =>
    !t.done).length

  return (
    <div className="stack">
      <form onSubmit={handleSubmit} className="row">
        <input value={text} onChange={(e) =>
          setText(e.target.value)}
          placeholder="Nueva tarea" />
        <button type="submit">Agregar</button>
      </form>
      <ul className="stack" style={{ listStyle:
        'none', padding: 0, margin: 0 }}>
        {tasks.map((task) => (
          <li key={task.id} className="row">
            <input type="checkbox" checked=
              {task.done} onChange={() =>
              toggle(task.id)} />
            <span style={{ textDecoration: task.done
              ? 'line-through' : 'none', flex: 1 }}>
              {task.text}
            </span>
            <button onClick={() => remove(task.id)}
              aria-label={`Borrar ${task.text}`}>
              ×
            </button>
          </li>
        ))}
      </ul>
      <p className="muted">{pending} pendientes</p>
    </div>
  )
}
```

> **ℹ️ Inicialización perezosa**
>
> `useState(createInitialTasks)` (sin paréntesis) le pasa **la función** a React, que la ejecuta solo en el primer render. Si escribís `useState(createInitialTasks())` la función corre en **cada** render, aunque el resultado se descarte. Importa cuando el cálculo inicial es caro, como leer `localStorage` o procesar una lista grande.

El estado vive en la posición del árbol

El estado no está "dentro" del componente: React lo guarda asociado a la **posición** del componente en el árbol. Si en el mismo lugar renderizás el mismo tipo de componente, el estado se conserva aunque cambien las props.

DEMO **Reiniciar el estado con key**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/fundamentos/KeyReset.tsx

```tsx
import { useState } from 'react'

type Contact = { id: number; name: string }

const contacts: Contact[] = [
  { id: 1, name: 'Recruiter de Nubelia' },
  { id: 2, name: 'CTO de Datalab' },
  { id: 3, name: 'Referido de Pagos Sur' },
]

function MessageDraft({ contact }: { contact:
  Contact }) {
  // Este estado pertenece a la "posición" del
    componente en el árbol...
  const [draft, setDraft] = useState('')
  return (
    <div className="stack">
      <textarea
        rows={3}
        value={draft}
        onChange={(e) => setDraft(e.target.value)}
        placeholder={`Mensaje para ${contact.name}`}
      />
      <button disabled={!draft}>Enviar a
        {contact.name}</button>
    </div>
  )
}

export default function KeyReset() {
  const [selected, setSelected] =
    useState(contacts[0])
  const [useKey, setUseKey] = useState(false)

  return (
```

```
    <div className="stack">
      <div className="row">
        {contacts.map((contact) => (
          <button
            key={contact.id}
            className={contact.id === selected.id ?
              'active' : ''}
            onClick={() => setSelected(contact)}
          >
            {contact.name}
          </button>
        ))}
      </div>
      <label className="row">
        <input type="checkbox" checked={useKey}
          onChange={(e) =>
          setUseKey(e.target.checked)} />
        Usar key={'{contact.id}'} para reiniciar el
          borrador
      </label>
      {/* ...salvo que cambie su key: una key
        distinta = un componente nuevo. */}
      {useKey ? (
        <MessageDraft key={selected.id} contact=
          {selected} />
      ) : (
        <MessageDraft contact={selected} />
      )}
    </div>
  )
}
```

🧪 Probá esto

Escribí un borrador para el recruiter de Nubelia y cambiá al CTO de Datalab: sin `key`, el borrador te sigue. Activá la opción de `key` y repetí: cada contacto arranca con el borrador vacío. Cambiar la `key` es la forma idiomática de decir "esto es otro componente, empezá de cero".

Errores comunes

⚠️ Leer el estado justo después de setearlo

```
function handleClick() {
  setCount(count + 1)
  console.log(count) // 🔴 todavía muestra el
    valor viejo: es la foto de este render
}
```

Si necesitás el valor nuevo, calculalo en una variable: `const next = count + 1; setCount(next); console.log(next)`.

⚠️ Guardar en el estado algo que se puede calcular

Si un valor se puede calcular a partir de props o de otro estado, **no lo guardes en el estado**. Lo vemos en el próximo capítulo.

🎤 En una entrevista

"¿Por qué setState no actualiza el valor inmediatamente?" → "Porque el estado es una foto del render actual. `setState` agenda un nuevo render, y en ese render la variable tiene el valor nuevo. Además React agrupa varias actualizaciones en un solo render. Si el valor nuevo depende del anterior, uso la forma con función: `setCount(c => c + 1)`."

Capítulo 5

Estado derivado y levantar el estado

Guardar lo mínimo, calcular el resto y compartir estado entre componentes.

Dos ideas separan el código React prolijo del código React que se vuelve inmanejable: **guardar el mínimo estado posible** y **ponerlo en el lugar correcto**. Las dos te van a ahorrar la mayoría de los `useEffect` innecesarios.

Estado mínimo, todo lo demás se deriva

Antes de crear un `useState`, preguntate:

1. ¿Viene de las props? → No es estado.
2. ¿Se mantiene igual con el tiempo? → No es estado, es una constante.
3. ¿Se puede **calcular** con otras props o estado? → No es estado, es un **dato derivado**.

Lo que queda es el estado real. Todo lo demás se calcula **durante el render**, en variables comunes.

DEMO **Filtrar y contar sin estado extra**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/fundamentos/DerivedState.tsx`

```tsx
import { useState } from 'react'

type Status = 'aplicado' | 'entrevista' | 'oferta' |
  'rechazado'
type Application = { id: number; company: string;
  status: Status }

const applications: Application[] = [
  { id: 1, company: 'Nubelia', status: 'entrevista'
    },
  { id: 2, company: 'Pagos Sur', status: 'aplicado'
    },
  { id: 3, company: 'Datalab', status: 'oferta' },
  { id: 4, company: 'Kuarahy', status: 'rechazado'
    },
  { id: 5, company: 'Andes Cloud', status:
    'aplicado' },
]

export default function DerivedState() {
  // Estado mínimo: solo lo que el usuario puede
    cambiar.
  const [query, setQuery] = useState('')
  const [status, setStatus] = useState<Status |
    'todos'>('todos')

```

```
  // Todo lo demás se DERIVA durante el render. Sin
    useEffect, sin estado duplicado.
  const filtered = applications.filter(
    (app) =>

        app.company.toLowerCase().includes(query.toL
        owerCase()) &&
      (status === 'todos' || app.status === status),
  )
  const active = filtered.filter((app) => app.status
    !== 'rechazado').length

  return (
    <div className="stack">
      <div className="row">
        <input
          type="search"
          placeholder="Buscar empresa…"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
        />
        <select value={status} onChange={(e) =>
          setStatus(e.target.value as Status |
          'todos')}>
          <option value="todos">Todos los
            estados</option>
          <option value="aplicado">Aplicado</option>
          <option
            value="entrevista">Entrevista</option>
          <option value="oferta">Oferta</option>
          <option
            value="rechazado">Rechazado</option>
        </select>
      </div>
      <ul style={{ margin: 0, paddingLeft: '1.2em'
        }}>
        {filtered.map((app) => (
          <li key={app.id}>
```

```
            {app.company} <span className="pill">
              {app.status}</span>
          </li>
        ))}
      </ul>
      <p className="muted">
        {filtered.length} resultados · {active}
          activas
      </p>
    </div>
  )
}
```

Fijate que hay solo dos `useState` (el texto y el filtro). La lista filtrada y la cantidad de activas son variables que se recalculan en cada render. Es rápido, siempre está sincronizado y no hay forma de que quede desactualizado.

× Sincronizar con useEffect

```
const [query, setQuery] = useState('')
const [filtered, setFiltered] = useState(apps)

// 🔴 Un render extra y una fuente de bugs
useEffect(() => {
  setFiltered(apps.filter((a) =>
    a.company.includes(query)))
}, [apps, query])
```

✓ Calcular durante el render

```
const [query, setQuery] = useState('')
```

```
// ✅ Un solo render, siempre correcto
const filtered = apps.filter((a) =>
  a.company.includes(query),
)
```

> 💡 **¿Y si el cálculo es caro?**
>
> Filtrar unas cuantas centenas de elementos es instantáneo. Si de verdad el cálculo es pesado (miles de elementos, procesamiento complejo), lo memoizás con `useMemo`, o mejor, dejás que el React Compiler lo haga por vos. Nunca lo resuelvas con un Effect.

Levantar el estado

Cuando dos componentes necesitan el mismo dato, el estado se "levanta" al **padre común más cercano**, que lo pasa hacia abajo por props. Los hijos avisan cambios llamando a funciones que también reciben por props.

DEMO **Un solo panel abierto a la vez**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/fundamentos/Accordion.tsx

```tsx
import { useState, type ReactNode } from 'react'

type PanelProps = {
  title: string
  isActive: boolean
  onShow: () => void
  children: ReactNode
}

// Panel no tiene estado propio: lo controla el
  padre a través de props.
function Panel({ title, isActive, onShow, children
  }: PanelProps) {
  return (
    <section className="card">
      <div className="row" style={{ justifyContent:
        'space-between' }}>
        <strong>{title}</strong>
        {!isActive && <button onClick=
          {onShow}>Ver</button>}
      </div>
      {isActive && <div style={{ marginTop: 8 }}>
        {children}</div>}
    </section>
  )
}

export default function Accordion() {
  // El estado "subió" al padre común: así solo un
    panel puede estar abierto.
  const [activeIndex, setActiveIndex] = useState(0)

  return (
    <div className="stack">
      <Panel title="¿Qué es el estado?" isActive=
        {activeIndex === 0} onShow={() =>
```

```
          setActiveIndex(0)}>
          Datos que cambian con el tiempo y que, al
            cambiar, hacen que React vuelva a
            renderizar.
        </Panel>
        <Panel title="¿Qué son las props?" isActive=
          {activeIndex === 1} onShow={() =>
          setActiveIndex(1)}>
          Los argumentos que un componente padre le
            pasa a un hijo. Son de solo lectura.
        </Panel>
        <Panel title="¿Dónde va el estado?" isActive=
          {activeIndex === 2} onShow={() =>
          setActiveIndex(2)}>
          En el componente padre común más cercano de
            todos los que lo necesitan.
        </Panel>
      </div>
    )
}
```

Si cada `Panel` tuviera su propio `useState(isOpen)`, no habría forma de cerrar uno cuando se abre otro. Al subir el estado al padre, `Accordion` es la **única fuente de verdad** y cada `Panel` pasa a ser un componente **controlado**: no decide nada, solo muestra lo que le dicen y avisa cuando lo tocan.

Componentes controlados y no controlados

	Controlado	No controlado
¿Quién tiene el dato?	El padre (estado de React)	El propio componente o el DOM
Ejemplo con inputs	`<input value={v} onChange={...} />`	`<input defaultValue="x" />`
Ventaja	Podés validar y transformar mientras se escribe	Menos código y menos renders

En React 19 los formularios con `action` reciben un `FormData` con todos los valores, así que los inputs **no controlados** vuelven a ser una opción excelente para formularios simples.

Flujo de datos en una sola dirección

Los datos bajan por props y los eventos suben por funciones. Cuando algo se ve mal en la pantalla, seguís el dato hacia arriba hasta encontrar quién tiene el estado. Esa previsibilidad es la razón de ser de React.

🎤 En una entrevista

"¿Dónde ponés el estado?" → "Primero me fijo si realmente es estado o si se puede derivar. Si es estado, lo pongo en el componente más bajo que lo necesite; si lo necesitan varios, lo subo al padre común más cercano y lo paso por props. Si muchos niveles lo necesitan, uso Context o una librería de estado."

Parte 2 · Hooks a fondo

Todos los hooks que vas a usar en el día a día, con sus trampas más comunes.

Capítulo 6

useEffect: sincronizar con el exterior

Dependencias, cleanup, StrictMode, fetch sin race conditions y cuándo NO usarlo.

`useEffect` es el hook más usado y el peor usado. La clave es entender **para qué existe**: sincronizar tu componente con un **sistema externo** a React. Un evento del navegador, un `setInterval`, un WebSocket, una librería que no es de React, `localStorage`, una conexión de chat.

```
useEffect(() => {
  // setup: conectarse, suscribirse, arrancar algo
  return () => {
```

```
    // cleanup: desconectarse, desuscribirse, frenar
       ese algo
  }
}, [dependencias])
```

Olvidate del ciclo de vida

Con clases pensabas en `componentDidMount`, `componentDidUpdate` y `componentWillUnmount`. Con hooks conviene pensar distinto:

> "Este efecto mantiene **X** sincronizado con **Y**. Cada vez que **Y** cambia, deshago la sincronización vieja y armo la nueva."

El array de dependencias declara **de qué valores depende** la sincronización. No es un "cuándo correr", es un "con qué datos":

Dependencias	Cuándo corre el setup
Sin array	Después de **cada** render (casi nunca es lo que querés)
`[]`	Solo después del primer render (no depende de nada reactivo)
`[a, b]`	Después del primer render y cada vez que cambie `a` o `b`

Antes de cada nuevo setup y al desmontar, React ejecuta el **cleanup** del efecto anterior.

DEMO **Suscribirse a un evento del navegador**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/WindowSize.tsx`

```
import { useEffect, useState } from 'react'

export default function WindowSize() {
  const [width, setWidth] =
    useState(window.innerWidth)

  useEffect(() => {
    // Setup: nos suscribimos a un sistema externo
      (el evento resize del navegador)
    function handleResize() {
      setWidth(window.innerWidth)
    }
    window.addEventListener('resize', handleResize)

    // Cleanup: deshacemos exactamente lo que hizo
      el setup
    return () =>
      window.removeEventListener('resize',
      handleResize)
  }, []) // [] = no depende de props ni estado: se
    suscribe una vez

  return (
    <p>
```

```
      Ancho de la ventana: <strong>
        {width}px</strong>{' '}
      {width < 900 ? '📱 layout móvil' : '🖥️ layout
        escritorio'}
    </p>
  )
}
```

Setup, cleanup y StrictMode

En la siguiente demo, el reloj depende de `intervalMs`. Cambiá la velocidad y mirá el log: React limpia el intervalo viejo y crea uno nuevo. Desmontá el reloj y vas a ver el cleanup final.

DEMO **El ciclo setup → cleanup → setup**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/Clock.tsx`

```
import { useEffect, useState, type Dispatch, type
  SetStateAction } from 'react'

type ClockProps = {
  intervalMs: number
  onLog: Dispatch<SetStateAction<string[]>>
}

function Clock({ intervalMs, onLog }: ClockProps) {
  const [now, setNow] = useState(() => new Date())
```

```
  useEffect(() => {
    onLog((log) => [...log, `▶ setup: setInterval
      cada ${intervalMs} ms`])
    const id = setInterval(() => setNow(new Date()),
      intervalMs)

    return () => {
      clearInterval(id)
      onLog((log) => [...log, `■ cleanup:
        clearInterval de ${intervalMs} ms`])
    }
  }, [intervalMs, onLog]) // si cambia intervalMs:
    cleanup del viejo + setup del nuevo

  return (
    <p style={{ fontSize: '1.8rem', fontWeight: 700,
      fontVariantNumeric: 'tabular-nums' }}>
      {now.toLocaleTimeString('es', { hour12: false
        })}.
        {String(now.getMilliseconds()).padStart(3,
        '0')}
    </p>
  )
}

export default function ClockDemo() {
  const [visible, setVisible] = useState(true)
  const [intervalMs, setIntervalMs] = useState(1000)
  const [log, setLog] = useState<string[]>([])

  return (
    <div className="stack">
      <div className="row">
        <button onClick={() => setVisible((v) =>
          !v)}>{visible ? 'Desmontar' : 'Montar'}
          reloj</button>
        <select value={intervalMs} onChange={(e) =>
          setIntervalMs(Number(e.target.value))}>
```

```
          <option value={1000}>Cada 1
            segundo</option>
          <option value={100}>Cada 100 ms</option>
        </select>
        <button onClick={() => setLog([])}>Limpiar
          log</button>
      </div>
      {visible && <Clock intervalMs={intervalMs}
        onLog={setLog} />}
      <ol className="log">
        {log.map((line, i) => (
          <li key={i}>{line}</li>
        ))}
      </ol>
    </div>
  )
}
```

ℹ️ ¿Por qué el log arranca con setup, cleanup, setup?

En desarrollo, `<StrictMode>` monta cada componente, lo desmonta y lo vuelve a montar enseguida. Lo hace **a propósito** para que te des cuenta si te olvidaste del cleanup: si sin cleanup quedaran dos intervalos andando, lo notarías enseguida. En producción el efecto corre una sola vez. La solución nunca es "evitar que corra dos veces", sino escribir un cleanup correcto.

> 🧪 **Probá esto**
>
> Comentá la línea `clearInterval(id)` del cleanup en `src/demos/hooks/Clock.tsx` y cambiá varias veces la velocidad. Vas a acumular intervalos y el reloj se va a actualizar cada vez más seguido. Ese es el bug que StrictMode intenta mostrarte.

Pedir datos en un Effect (y la race condition)

Pedir datos en un Effect funciona, pero tiene trampas. La más común: si el usuario cambia rápido de selección, las respuestas pueden llegar **desordenadas** y la última en llegar pisa a la correcta.

DEMO **Race condition al pedir datos**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/FetchRace.tsx`

```
import { useEffect, useState } from 'react'
import { sleep } from '../shared/utils'
```

```
type Company = { id: number; name: string;
  employees: number; delayMs: number }

const companies: Company[] = [
  { id: 1, name: 'Nubelia', employees: 120, delayMs:
    2000 },
  { id: 2, name: 'Pagos Sur', employees: 45,
    delayMs: 300 },
  { id: 3, name: 'Datalab', employees: 300, delayMs:
    800 },
]

// "API" falsa: cada empresa tarda distinto en
  responder.
async function fetchCompany(id: number):
  Promise<Company> {
  const company = companies.find((c) => c.id ===
    id)!
  await sleep(company.delayMs)
  return company
}

export default function FetchRace() {
  const [companyId, setCompanyId] = useState(3)
  const [company, setCompany] = useState<Company |
    null>(null)
  const [isLoading, setIsLoading] = useState(true)
  const [protect, setProtect] = useState(true)

  useEffect(() => {
    let ignore = false
    fetchCompany(companyId).then((data) => {
      // Si el Effect ya se limpió (cambió
        companyId), descartamos esta respuesta
        vieja.
      if (protect && ignore) return
      setCompany(data)
      setIsLoading(false)
    })
```

```
    return () => {
      ignore = true
    }
  }, [companyId, protect])

  function select(id: number) {
    setCompanyId(id)
    setIsLoading(true)
  }

  const isWrong = !isLoading && company !== null &&
    company.id !== companyId

  return (
    <div className="stack">
      <div className="row">
        {companies.map((c) => (
          <button key={c.id} className={c.id ===
            companyId ? 'active' : ''} onClick={()
            => select(c.id)}>
            {c.name} ({c.delayMs} ms)
          </button>
        ))}
      </div>
      <label className="row">
        <input type="checkbox" checked={protect}
          onChange={(e) =>
          setProtect(e.target.checked)} />
        Ignorar respuestas viejas (flag
          <code>ignore</code> en el cleanup)
      </label>
      <div className="card">
        {isLoading ? (
          <p className="muted">Cargando…</p>
        ) : (
          company && (
            <p>
              <strong>{company.name}</strong> ·
                {company.employees} empleados
```

```
            </p>
          )
        )}
        {isWrong && <p className="error-text">⚠️
          Bug: seleccionaste otra empresa pero se
          muestra {company.name}.</p>}
      </div>
    </div>
  )
}
```

🧪 Probá esto

Desactivá "Ignorar respuestas viejas", tocá **Nubelia** (tarda 2 s) e inmediatamente **Pagos Sur** (tarda 300 ms). Primero aparece Pagos Sur, pero a los 2 segundos llega la respuesta de Nubelia y la pisa. Con el flag `ignore` activado, el cleanup marca la respuesta vieja como descartable.

Con `fetch` real también podés usar un `AbortController` y llamar a `controller.abort()` en el cleanup, lo que además cancela la request en el navegador.

> **💡 En una app real, casi nunca hagas fetch en un Effect**
>
> Además de las race conditions, el fetch en un Effect no tiene caché, no se ejecuta en el servidor y genera cascadas (el hijo empieza a pedir datos recién cuando el padre terminó). Preferí, en este orden: **Server Components** en Next.js, el loader de tu router, o una librería como **TanStack Query**. Con React 19 también podés leer una promesa con use().

Quizás no necesitás un Effect

Esta es la parte más importante del capítulo. Si no hay un sistema externo de por medio, probablemente no necesitás un Effect.

Situación	En vez de un Effect...
Calcular un dato a partir de props o estado	Calculalo durante el render (estado derivado).
Cálculo caro	`useMemo`, o dejá que lo haga el React Compiler.
Reiniciar el estado cuando cambia una prop	Pasale una `key` distinta al componente.
Reaccionar a algo que hizo el usuario	Ponelo en el **event handler** (onClick, onSubmit...).

Situación	En vez de un Effect...
Avisarle al padre que algo cambió	Llamá a la prop `onChange` desde el mismo handler.
Enviar un formulario	Event handler o, en React 19, una Action.

× Lógica de un evento dentro de un Effect

```
const [submitted, setSubmitted] = useState(false)

// 🔴 Indirecto, corre de más y es difícil de seguir
useEffect(() => {
  if (submitted) {
    sendApplication(form)
    showToast('¡Postulación enviada!')
  }
}, [submitted, form])

function handleSubmit() {
  setSubmitted(true)
}
```

✓ La lógica vive en el handler

```
// ✅ Pasó por un click: va en el handler
function handleSubmit() {
  sendApplication(form)
  showToast('¡Postulación enviada!')
}
```

Las dependencias no se eligen, se declaran

Todo valor reactivo (props, estado, y variables o funciones declaradas dentro del componente) que se lee en el Effect **tiene que estar** en las dependencias. No lo decidís vos: lo exige el código. El linter (`react-hooks/exhaustive-deps`) te avisa cuando falta alguno.

Si una dependencia hace que el Effect corra de más, no la saques del array: **cambiá el código** para no necesitarla.

- Usá la forma con función del setter: `setCount(c => c + 1)` en vez de leer `count`.
- Mové objetos y funciones **adentro** del Effect si solo se usan ahí.
- Si necesitás leer el valor más reciente sin "reaccionar" a sus cambios, usá useEffectEvent (React 19.2).

⚠️ Objetos y funciones en las dependencias

```
function ChatRoom({ roomId }: { roomId: string
  }) {
  const options = { roomId, serverUrl:
    'https://chat.ejemplo.com' } // objeto
    nuevo en cada render

  useEffect(() => {
    const connection =
      createConnection(options)
    connection.connect()
    return () => connection.disconnect()
  }, [options]) // 🔴 se reconecta en CADA
    render
}
```

Solución: creá `options` **dentro** del Effect y dependé solo de `roomId`.

useEffect vs useLayoutEffect

`useEffect` corre **después** de que el navegador pinta la pantalla, así no la bloquea.
`useLayoutEffect` corre **antes** de pintar, de forma síncrona. Usalo solo cuando necesitás medir el DOM y corregir la posición de algo antes de que el usuario lo vea (por ejemplo, un tooltip). Lo vemos en otros hooks.

🎤 En una entrevista

"¿Para qué sirve useEffect?" → "Para sincronizar un componente con un sistema externo: suscripciones, timers, APIs del navegador, librerías que no son de React. El setup arma la sincronización, el cleanup la deshace, y las dependencias le dicen a React cuándo rehacerla. Para datos derivados o lógica de eventos no uso Effects, y para pedir datos prefiero el framework o una librería como TanStack Query."

Capítulo 7

useRef: el DOM y valores que no renderizan

Acceder a nodos del DOM y guardar valores que no deben provocar renders.

`useRef` te da una "caja" (`{ current: valor }`) que **persiste entre renders** y que podés modificar sin que el componente se vuelva a renderizar. Tiene dos usos principales:

1. Guardar una referencia a un **nodo del DOM** (para enfocarlo, medirlo, hacer scroll).
2. Guardar un **valor que no se muestra** en pantalla (el id de un timer, una instancia de una librería, el valor anterior de algo).

```
const ref = useRef(valorInicial)
ref.current // leer
ref.current = nuevoValor // escribir: NO dispara un
  render
```

Referencias al DOM

Pasás el ref como prop `ref` a un elemento, y después del commit React pone el nodo en `ref.current`.

DEMO **Enfocar y seleccionar un input**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/hooks/FocusInput.tsx

```tsx
import { useRef } from 'react'

export default function FocusInput() {
  // useRef<HTMLInputElement>(null): al principio no
    apunta a nada
  const inputRef = useRef<HTMLInputElement>(null)

  return (
    <div className="row">
      {/* React asigna el nodo del DOM a
        inputRef.current después del commit */}
      <input ref={inputRef} defaultValue="Frontend
        Engineer" />
      <button onClick={() =>
        inputRef.current?.focus()}>Enfocar</button>
      <button onClick={() =>
        inputRef.current?.select()}>Seleccionar
        todo</button>
    </div>
  )
```

```
}
```

Enfocar, hacer scroll, medir tamaños, reproducir un video o integrar una librería que necesita un nodo del DOM (un mapa, un gráfico, un editor) son los casos típicos.

Ref vs estado

	`useState`	`useRef`
¿Cambiarlo dispara un render?	Sí	No
¿Se puede leer durante el render?	Sí	No conviene (solo en eventos y Effects)
¿Cómo se cambia?	Con el setter	Mutando `ref.current`
Usalo para...	Todo lo que se muestra en pantalla	Lo que el render no necesita

DEMO **El ref cambia, pero la pantalla no se entera**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/hooks/RefVsState.tsx

```tsx
import { useRef, useState } from 'react'

export default function RefVsState() {
  const [stateClicks, setStateClicks] = useState(0)
  const refClicks = useRef(0)
  const [report, setReport] = useState('')

  return (
    <div className="stack">
      <div className="row">
        <button onClick={() => setStateClicks((n) =>
          n + 1)}>Click con estado: {stateClicks}
          </button>
        <button
          onClick={() => {
            refClicks.current += 1 // cambia el
              valor, pero React no se entera
          }}
        >
          Click con ref
        </button>
        <button onClick={() => setReport(`El ref
          vale ${refClicks.current}`)}>Leer el
          ref</button>
      </div>
      {report && <p className="muted">{report}</p>}
    </div>
  )
}
```

> 🧪 **Probá esto**
>
> Tocá varias veces **Click con ref** y después **Leer el ref**: el valor se fue sumando aunque la pantalla no cambió. Ahora tocá **Click con estado**: cada click dispara un render.

Un valor que no se muestra: el id del intervalo

El cronómetro necesita guardar el id del `setInterval` para poder frenarlo después. Ese id no se muestra en pantalla, así que va en un ref.

DEMO **Cronómetro con el id del intervalo en un ref**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/Stopwatch.tsx`

```
import { useEffect, useRef, useState } from 'react'

export default function Stopwatch() {
  const [startedAt, setStartedAt] = useState<number
    | null>(null)
  const [now, setNow] = useState<number | null>
    (null)
```

```
  // El id del intervalo no se muestra en pantalla:
    no necesita ser estado.
  const intervalRef = useRef<number | undefined>
    (undefined)

  function start() {
    setStartedAt(Date.now())
    setNow(Date.now())
    clearInterval(intervalRef.current)
    intervalRef.current = window.setInterval(() =>
      setNow(Date.now()), 10)
  }

  function stop() {
    clearInterval(intervalRef.current)
  }

  // Si el componente se desmonta con el cronómetro
    andando, lo frenamos.
  useEffect(() => () =>
    clearInterval(intervalRef.current), [])

  const seconds = startedAt !== null && now !== null
    ? (now - startedAt) / 1000 : 0

  return (
    <div className="row">
      <span style={{ fontSize: '1.8rem', fontWeight:
        700, fontVariantNumeric: 'tabular-nums',
        minWidth: 110 }}>
        {seconds.toFixed(2)}s
      </span>
      <button onClick={start}>Iniciar</button>
      <button onClick={stop}>Frenar</button>
    </div>
  )
}
```

Reglas para no meterte en problemas

- **No leas ni escribas `ref.current` durante el render**, salvo para inicializarlo. El render tiene que ser puro; los refs se usan en event handlers y Effects.
- Si un valor se muestra en pantalla, es **estado**, no ref.
- Un ref al DOM vale `null` hasta que el elemento se monta. Por eso el `?.` en `inputRef.current?.focus()`.

> ℹ️ **Novedades de React 19 con refs**
>
> - `ref` ya es una **prop común** en componentes de función: chau `forwardRef`.
> - Un **ref callback** puede devolver una función de limpieza.
> - `useRef()` sin argumento ya no está permitido en los tipos de TypeScript: escribí `useRef<T>(null)` o `useRef<T>(undefined)`.
>
> Todo esto está en ref como prop. Y en React 19.3 hasta los Fragments aceptan ref: Fragment refs.

🎤 En una entrevista

"¿Diferencia entre useRef y useState?" → "Los dos persisten entre renders, pero cambiar un ref no dispara un render y cambiar el estado sí. Uso estado para todo lo que se ve en pantalla y refs para nodos del DOM o valores que el render no necesita, como el id de un timer."

Capítulo 8

useContext: datos sin prop drilling

Compartir datos con todo un subárbol sin pasar props nivel por nivel.

Pasar props de padre a hijo está bien. Pero cuando un dato lo necesitan muchos componentes en distintos niveles (el usuario logueado, el tema, el idioma), terminás pasándolo por componentes que ni lo usan. Eso se llama **prop drilling**, y Context lo resuelve: un componente **provee** un valor y cualquier descendiente lo **lee** directamente.

Los tres pasos

```
// 1. Crear el contexto (con un valor por defecto)
const ThemeContext = createContext<'claro' |
  'oscuro'>('claro')

// 2. Proveer un valor a un subárbol (React 19: sin
  .Provider)
<ThemeContext value={theme}>
```

```
  <App />
</ThemeContext>

// 3. Leerlo en cualquier componente de abajo
const theme = useContext(ThemeContext)
```

DEMO **Tema compartido sin prop drilling**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/ThemeContext.tsx`

```
import { createContext, useContext, useState } from
  'react'

type Theme = 'claro' | 'oscuro'

// 1) Crear el contexto con un valor por defecto
const ThemeContext = createContext<Theme>('claro')

const styles = {
  claro: { background: '#ffffff', color: '#23272f',
    border: '1px solid #d0d5dd' },
  oscuro: { background: '#23272f', color: '#f5f6f8',
    border: '1px solid #23272f' },
}

// 3) Leer el contexto en cualquier componente de
  abajo, sin recibir props
function ThemedButton({ label }: { label: string })
  {
  const theme = useContext(ThemeContext)
  return <button style={styles[theme]}>{label}
    ({theme})</button>
```

```tsx
}

function Toolbar() {
  // Toolbar no sabe nada del tema: no hay prop
    drilling
  return (
    <div className="row">
      <ThemedButton label="Guardar" />
      <ThemedButton label="Cancelar" />
    </div>
  )
}

export default function ThemeDemo() {
  const [theme, setTheme] = useState<Theme>('claro')

  return (
    // 2) Proveer el valor. En React 19:
      <ThemeContext value={...}> (sin .Provider)
    <ThemeContext value={theme}>
      <div className="stack">
        <label className="row">
          <input
            type="checkbox"
            checked={theme === 'oscuro'}
            onChange={(e) =>
              setTheme(e.target.checked ? 'oscuro' :
              'claro')}
          />
          Tema oscuro
        </label>
        <Toolbar />
        {/* Un provider más cercano pisa el valor
          para su subárbol */}
        <ThemeContext value="claro">
          <div className="card">
            <p className="muted">Este recuadro
              fuerza el tema claro:</p>
            <ThemedButton label="Siempre claro" />
```

```
          </div>
        </ThemeContext>
      </div>
    </ThemeContext>
  )
}
```

Fijate en `Toolbar`: renderiza los botones pero no recibe ni pasa ninguna prop de tema. Y el recuadro de abajo muestra otra regla: **gana el provider más cercano**.

× Hasta React 18

```
<ThemeContext.Provider value={theme}>
  <App />
</ThemeContext.Provider>
```

✓ React 19

```
<ThemeContext value={theme}>
  <App />
</ThemeContext>
```

`.Provider` sigue funcionando, pero la forma nueva es más corta y es la recomendada. En el futuro `.Provider` se va a deprecar.

El patrón Provider + custom hook

En una app real casi nunca se usa `useContext` "suelto". Se arma un componente que encapsula el estado y un custom hook que lo consume y valida que exista el provider:

DEMO **AuthProvider + useAuth**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/AuthContext.tsx`

```
import { createContext, useContext, useState, type
  ReactNode } from 'react'

type User = { name: string; role: 'candidato' |
  'recruiter' }
type AuthContextValue = {
  user: User | null
  login: (user: User) => void
  logout: () => void
}

// null por defecto: así detectamos si alguien lo
  usa fuera del provider
const AuthContext = createContext<AuthContextValue |
  null>(null)

// Patrón habitual: un componente Provider que
  encapsula el estado...
// (en una app real, AuthProvider y useAuth viven en
  su propio archivo y se exportan)
```

```tsx
function AuthProvider({ children }: { children:
  ReactNode }) {
  const [user, setUser] = useState<User | null>
    (null)
  const value = { user, login: setUser, logout: ()
    => setUser(null) }
  return <AuthContext value={value}>{children}
    </AuthContext>
}

// ...y un custom hook que lo consume y valida que
  haya provider.
function useAuth() {
  const context = useContext(AuthContext)
  if (!context) throw new Error('useAuth tiene que
    usarse dentro de <AuthProvider>')
  return context
}

function UserMenu() {
  const { user, login, logout } = useAuth()
  if (!user) {
    return <button onClick={() => login({ name:
      'Ana', role: 'candidato' })}>Iniciar
      sesión</button>
  }
  return (
    <div className="row">
      <span>
        Hola, <strong>{user.name}</strong>
          ({user.role})
      </span>
      <button onClick={logout}>Salir</button>
    </div>
  )
}

export default function AuthDemo() {
  return (
```

```
    <AuthProvider>
      <UserMenu />
    </AuthProvider>
  )
}
```

El `throw` dentro de `useAuth` te salva de horas de debugging: si alguien usa el hook fuera del provider, el error dice exactamente qué pasa.

Context y rendimiento

Cuando el `value` de un provider cambia, **todos** los componentes que leen ese contexto se vuelven a renderizar. Algunas recomendaciones:

- **Separá contextos** por frecuencia de cambio: no mezcles el usuario (cambia poco) con la posición del mouse (cambia todo el tiempo).
- Si el `value` es un objeto creado en el render, es nuevo en cada render del provider. Con el React Compiler eso se memoiza solo; sin él, envolvelo en `useMemo`.
- Para estado global que cambia seguido y que leen muchos componentes, una librería como **Zustand** (que usa `useSyncExternalStore`) suele ser mejor opción.

⚠️ Context no es un gestor de estado

Context es un mecanismo de **transporte**: lleva un valor a lo profundo del árbol. El estado sigue viviendo en un `useState` o un `useReducer` dentro del provider. La combinación `useReducer` + Context es un patrón clásico para estado global sin librerías.

💡 React 19: use(Context)

`use(ThemeContext)` hace lo mismo que `useContext(ThemeContext)`, pero se puede llamar **dentro de un if** o de un loop. Lo vemos en el capítulo de use().

🎤 En una entrevista

"¿Cuándo usás Context?" → "Para datos que necesitan muchos componentes en distintos niveles y que cambian poco: usuario autenticado, tema, idioma, feature flags. Lo encapsulo en un Provider con un custom hook. Para estado que cambia muy seguido o que leen muchísimos componentes, prefiero una librería como Zustand para evitar renders innecesarios."

Capítulo 9

useReducer: lógica de estado compleja

Centralizar las transiciones de estado en una función pura.

Cuando un componente tiene muchas formas de cambiar su estado (agregar, editar, mover, borrar...), los `setX` se desparraman por todos los handlers y cuesta seguir qué pasa. `useReducer` junta **toda la lógica de cambio en una sola función pura**, el reducer.

```
const [state, dispatch] = useReducer(reducer,
  initialState)

dispatch({ type: 'advanced', id: 3 }) // "pasó esto"

function reducer(state, action) {
  // "entonces el estado nuevo es este"
  return nuevoEstado
}
```

La idea es la misma de Redux, pero local a un componente y sin librerías.

Un pipeline de postulaciones

DEMO **Mover postulaciones entre etapas con un reducer**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/PipelineReducer.tsx`

```
import { useReducer, useState } from 'react'

type Stage = 'aplicado' | 'entrevista' | 'oferta' |
  'rechazado'
type Application = { id: number; company: string;
  stage: Stage }

// Todas las acciones posibles, como una unión
  discriminada por "type"
type Action =
  | { type: 'added'; id: number; company: string }
  | { type: 'advanced'; id: number }
  | { type: 'rejected'; id: number }
  | { type: 'removed'; id: number }

const nextStage: Record<Stage, Stage> = {
  aplicado: 'entrevista',
  entrevista: 'oferta',
  oferta: 'oferta',
  rechazado: 'rechazado',
```

```
}

// El reducer es una función PURA: (estado actual,
  acción) => estado nuevo.
// No hace fetch, no usa Date.now(), no muta: solo
  calcula.
function applicationsReducer(state: Application[],
  action: Action): Application[] {
  switch (action.type) {
    case 'added':
      return [...state, { id: action.id, company:
        action.company, stage: 'aplicado' }]
    case 'advanced':
      return state.map((app) => (app.id ===
        action.id ? { ...app, stage:
        nextStage[app.stage] } : app))
    case 'rejected':
      return state.map((app) => (app.id ===
        action.id ? { ...app, stage: 'rechazado' } :
        app))
    case 'removed':
      return state.filter((app) => app.id !==
        action.id)
  }
}

const initialApplications: Application[] = [
  { id: 1, company: 'Nubelia', stage: 'entrevista'
    },
  { id: 2, company: 'Pagos Sur', stage: 'aplicado'
    },
]

const stages: Stage[] = ['aplicado', 'entrevista',
  'oferta', 'rechazado']

export default function PipelineReducer() {
  const [applications, dispatch] =
    useReducer(applicationsReducer,
    initialApplications)
```

```
const [company, setCompany] = useState('')
const [lastAction, setLastAction] =
  useState<Action | null>(null)

// Los componentes solo "describen qué pasó": la
  lógica vive en el reducer.
function send(action: Action) {
  dispatch(action)
  setLastAction(action)
}

return (
  <div className="stack">
    <form
      className="row"
      onSubmit={(e) => {
        e.preventDefault()
        if (!company.trim()) return
        send({ type: 'added', id: Date.now(),
          company: company.trim() }) // el id se
          genera acá, no en el reducer
        setCompany('')
      }}
    >
      <input value={company} onChange={(e) =>
        setCompany(e.target.value)}
        placeholder="Empresa" />
      <button type="submit">Agregar</button>
    </form>

    <div className="grid-2">
      {stages.map((stage) => (
        <section key={stage} className="card">
          <strong style={{ textTransform:
            'capitalize' }}>{stage}</strong>
          {applications
            .filter((app) => app.stage === stage)
            .map((app) => (
```

```
                <div key={app.id} className="row"
                  style={{ marginTop: 6 }}>
                  <span style={{ flex: 1 }}>
                    {app.company}</span>
                  {stage !== 'oferta' && stage !==
                    'rechazado' && (
                    <button onClick={() => send({
                      type: 'advanced', id: app.id
                      })}>→</button>
                  )}
                  {stage !== 'rechazado' && <button
                    onClick={() => send({ type:
                    'rejected', id: app.id
                    })}>×</button>}
                  <button onClick={() => send({
                    type: 'removed', id: app.id })}>
                    🗑</button>
                </div>
              ))}
          </section>
        ))}
      </div>

      {lastAction && <pre
        className="log">dispatch({JSON.stringify(las
        tAction)})</pre>}
    </div>
  )
}
```

Tres cosas para notar:

1. **Las acciones describen qué pasó** (`added`, `advanced`, `rejected`), no qué hay que hacer con el estado. Los handlers quedan cortos y declarativos.

2. **El reducer es puro**: no hace requests, no genera ids aleatorios, no muta. Por eso el `id` se genera en el handler y viaja dentro de la acción.
3. **TypeScript brilla acá**: con la unión discriminada `Action`, dentro de cada `case` TypeScript sabe qué campos tiene la acción. Y si agregás un tipo nuevo y te olvidás de manejarlo, te avisa.

> ⚠️ **El reducer tiene que ser puro**
>
> En desarrollo, StrictMode llama al reducer **dos veces** para detectar impurezas. Si adentro usás `Date.now()`, `Math.random()` o mutás el estado, vas a ver comportamientos raros. Todo lo impuro va en el handler, antes del `dispatch`.

useState o useReducer

Usá `useState` si...	Usá `useReducer` si...
El estado es simple (un string, un booleano, un número).	Hay muchas formas distintas de actualizar el estado.
Las actualizaciones son independientes entre sí.	Varias partes del estado cambian juntas.

Usá `useState` si...	Usá `useReducer` si...
Hay pocos handlers.	Querés testear la lógica sin renderizar nada (el reducer es una función común).

No hay una respuesta correcta universal: muchos equipos arrancan con `useState` y migran a `useReducer` cuando el componente crece.

useReducer + Context

Para compartir este estado con un árbol grande, el patrón clásico es proveer `state` y `dispatch` en dos contextos:

```
const ApplicationsContext =
  createContext<Application[]>([])
const ApplicationsDispatchContext =
  createContext<Dispatch<Action>>(() => {})

export function ApplicationsProvider({ children }: {
  children: ReactNode }) {
  const [applications, dispatch] =
    useReducer(applicationsReducer, [])
  return (
    <ApplicationsContext value={applications}>
      <ApplicationsDispatchContext value={dispatch}>
        {children}</ApplicationsDispatchContext>
    </ApplicationsContext>
  )
}
```

`dispatch` tiene identidad estable, así que los componentes que solo despachan acciones no se re-renderizan cuando cambia la lista.

> **ℹ️ useReducer y useActionState**
>
> En React 19 existe useActionState, que se parece mucho a un `useReducer` pero cuyo reducer **puede ser async y tener efectos secundarios** (por ejemplo, llamar a una API). Cuando llegues a ese capítulo, vas a reconocer el patrón.

> **🎤 En una entrevista**
>
> "¿Cuándo usarías useReducer en vez de useState?" → "Cuando el estado tiene muchas transiciones posibles o varias partes cambian juntas. El reducer centraliza la lógica en una función pura que puedo testear sola, y los componentes solo despachan acciones que describen qué pasó."

Capítulo 10

Custom hooks

Extraer y reutilizar lógica con estado: useDebounce, useLocalStorage y más.

Un **custom hook** es una función cuyo nombre empieza con `use` y que llama a otros hooks. Sirve para **extraer lógica con estado** y reutilizarla en varios componentes, o simplemente para que un componente se lea mejor.

Es el reemplazo moderno de los HOCs (`withRouter(Component)`) y de los render props, que eran la forma de compartir lógica en la época de las clases.

```
function useOnlineStatus() {
  // ...hooks adentro...
  return isOnline
}

function StatusBar() {
  const isOnline = useOnlineStatus() // se lee como
    una frase
  return <p>{isOnline ? '🟢 En línea' : '🔴 Sin
    conexión'}</p>
```

```
}
```

Tres hooks, una demo

La demo usa tres custom hooks que vas a encontrar (con alguna variante) en casi cualquier proyecto real:

- `useDebouncedValue`: espera a que el usuario deje de escribir antes de buscar. Es lo mismo que practicaste en el ejercicio de **debounce** del live coding, pero como hook.
- `useLocalStorage`: un `useState` que sobrevive a recargar la página.
- `useOnlineStatus`: se suscribe a la conexión del navegador con `useSyncExternalStore`.

DEMO **useDebouncedValue + useLocalStorage + useOnlineStatus**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/CustomHooks.tsx`

```
import { useEffect, useState } from 'react'
import { sleep } from '../shared/utils'
import { useDebouncedValue } from
  './custom/useDebouncedValue'
```

```
import { useLocalStorage } from
  './custom/useLocalStorage'
import { useOnlineStatus } from
  './custom/useOnlineStatus'

const roles = ['Frontend Engineer', 'React
  Developer', 'Full Stack Engineer', 'UI Engineer',
  'Frontend Lead']

async function searchRoles(query: string) {
  await sleep(250) // latencia de la "API"
  return roles.filter((role) =>
    role.toLowerCase().includes(query.toLowerCase())
    )
}

export default function CustomHooksDemo() {
  const [query, setQuery] = useState('')
  const [keystrokes, setKeystrokes] = useState(0)
  const [requests, setRequests] = useState(0)
  const [results, setResults] = useState<string[]>
    (roles)
  const debouncedQuery = useDebouncedValue(query,
    500)
  const [notes, setNotes] = useLocalStorage('guia-
    react:notas', '')
  const isOnline = useOnlineStatus()

  // La búsqueda solo se dispara cuando el texto "se
    calmó"
  useEffect(() => {
    let ignore = false
    searchRoles(debouncedQuery).then((found) => {
      if (ignore) return
      setResults(found)
      setRequests((n) => n + 1)
    })
    return () => {
      ignore = true
```

```
    }
  }, [debouncedQuery])

  return (
    <div className="stack">
      <p>{isOnline ? '🟢 En línea' : '🔴 Sin
        conexión'}</p>

      <input
        type="search"
        value={query}
        placeholder="Buscar puesto…"
        onChange={(e) => {
          setQuery(e.target.value)
          setKeystrokes((n) => n + 1)
        }}
      />
      <p className="muted">
        Teclas: {keystrokes} · Requests a la API:
          {requests}
      </p>
      <ul style={{ margin: 0, paddingLeft: '1.2em'
        }}>
        {results.map((role) => (
          <li key={role}>{role}</li>
        ))}
      </ul>

      <label className="stack">
        Notas (se guardan en localStorage, recargá
          la página y siguen ahí)
        <textarea rows={2} value={notes} onChange=
          {(e) => setNotes(e.target.value)} />
      </label>
    </div>
  )
}
```

src/demos/hooks/custom/useDebouncedValue.ts

```ts
import { useEffect, useState } from 'react'

/** Devuelve `value`, pero recién cuando dejó de
  cambiar durante `delayMs`. */
export function useDebouncedValue<T>(value: T,
  delayMs = 400): T {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value),
      delayMs)
    // Si value cambia antes de que pase el tiempo,
      el cleanup cancela el timeout anterior.
    return () => clearTimeout(id)
  }, [value, delayMs])

  return debounced
}
```

src/demos/hooks/custom/useLocalStorage.ts

```ts
import { useEffect, useState } from 'react'

/** Como useState, pero el valor sobrevive a
  recargar la página. */
export function useLocalStorage<T>(key: string,
  initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    try {
      const saved = localStorage.getItem(key)
      return saved === null ? initialValue :
        (JSON.parse(saved) as T)
    } catch {
      return initialValue // modo privado, JSON
        corrupto, etc.
```

```
    }
  })

  // Sincronizamos el estado con un sistema externo:
    localStorage
  useEffect(() => {
    try {
      localStorage.setItem(key,
        JSON.stringify(value))
    } catch {
      // sin espacio o sin permisos: seguimos
        funcionando en memoria
    }
  }, [key, value])

  return [value, setValue] as const
}
```

src/demos/hooks/custom/useOnlineStatus.ts

```
import { useSyncExternalStore } from 'react'

function subscribe(onChange: () => void) {
  window.addEventListener('online', onChange)
  window.addEventListener('offline', onChange)
  return () => {
    window.removeEventListener('online', onChange)
    window.removeEventListener('offline', onChange)
  }
}

/** true si el navegador tiene conexión. */
export function useOnlineStatus() {
  return useSyncExternalStore(
    subscribe,
    () => navigator.onLine, // valor en el cliente
```

```
    () => true, // valor durante el render en el
      servidor (SSR)
  )
}
```

> **🧪 Probá esto**
>
> Escribí "frontend" rápido en el buscador y compará el contador de teclas con el de requests: con debounce, 8 teclas suman una sola request nueva. Cambiá el `500` de `useDebouncedValue(query, 500)` por `0` y repetí.

Mirá las pestañas de la demo: cada hook vive en su propio archivo y se puede testear y reutilizar por separado.

Las reglas

1. **El nombre empieza con `use`.** Así React (y el linter) saben que adentro hay hooks y aplican las reglas.
2. **Solo se llaman en el nivel superior** de un componente o de otro hook: nunca dentro de un `if`, un loop o un callback.
3. **Comparten lógica, no estado.** Cada componente que llama a `useLocalStorage` tiene su propio

estado independiente. Si querés compartir el estado mismo, necesitás levantarlo, Context o un store externo.

> **⚠️ No todo tiene que ser un hook**
>
> Si la función no llama a ningún hook, no le pongas `use` adelante: es una función común (`formatSalary`, `filterJobs`). Y no extraigas hooks "por las dudas": extraelos cuando la lógica se repite o cuando el componente se vuelve difícil de leer.

Buenos candidatos a custom hook

Hook	Qué encapsula
`useDebouncedValue`, `useThrottle`	Timers con cleanup
`useLocalStorage`, `useSessionStorage`	Persistencia en el navegador
`useMediaQuery`, `useWindowSize`	Suscripción a APIs del navegador
`useOnClickOutside`, `useHotkeys`	Event listeners globales
`useAuth`, `useCart`	Leer un Context y validar que exista el provider

Hook	Qué encapsula
`useJobs(filters)`	Un fetch con su estado (aunque en una app real usarías TanStack Query)

💡 TanStack Query: el custom hook que no vas a tener que escribir

`useQuery({ queryKey: ['jobs', filters], queryFn: fetchJobs })` te da caché, reintentos, revalidación, estados de carga y error, deduplicación de requests y mucho más. En el mercado laboral actual aparece en muchísimas ofertas de React: vale la pena conocerlo.

🎤 En una entrevista

"¿Qué es un custom hook?" → "Una función que empieza con use y llama a otros hooks, para reutilizar lógica con estado entre componentes. Reemplazó a los HOCs y render props. Importante: comparten la lógica, no el estado; cada componente que lo usa tiene su propia copia."

Capítulo 11

memo, useMemo y useCallback

Cuándo React vuelve a renderizar y cómo evitarlo (y por qué el Compiler lo cambia todo).

Antes de optimizar hay que entender **cuándo React vuelve a renderizar** un componente:

1. Cuando cambia **su estado**.
2. Cuando se renderiza **su padre** (aunque las props sean iguales).
3. Cuando cambia un **contexto** que lee.

Y un dato clave: **renderizar no es tocar el DOM**. Renderizar es ejecutar la función del componente y comparar el resultado. React solo toca el DOM si algo cambió. La mayoría de los renders "de más" son baratos y no hace falta evitarlos.

memo: saltear el render si las props no cambiaron

`memo(Componente)` hace que React se saltee el render del componente si **todas sus props son iguales** a las del render anterior (comparación superficial, con `Object.is`).

En la demo, cada fila parpadea en amarillo cuando se renderiza. Probá los tres modos y tocá **Re-renderizar el padre**:

DEMO **memo y useCallback, con resaltado de renders**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/MemoDemo.tsx`

```
import { memo, useCallback, useState } from 'react'
import { useRenderFlash } from
  '../shared/useRenderFlash'

type Job = { id: number; title: string }

const jobs: Job[] = [
  { id: 1, title: 'Frontend Engineer · Nubelia' },
  { id: 2, title: 'React Developer · Pagos Sur' },
  { id: 3, title: 'UI Engineer · Datalab' },
]
```

```
type RowProps = { job: Job; isSelected: boolean;
  onSelect: (id: number) => void }

function JobRow({ job, isSelected, onSelect }:
  RowProps) {
  const ref = useRenderFlash<HTMLLIElement>() //
    parpadea en amarillo en cada render
  return (
    <li ref={ref} className="row card" style={{
      justifyContent: 'space-between' }}>
      <span>
        {isSelected ? '★ ' : ''}
        {job.title}
      </span>
      <button onClick={() =>
        onSelect(job.id)}>Elegir</button>
    </li>
  )
}

// memo: si las props son iguales (comparación
  superficial), React se saltea el render
const MemoJobRow = memo(JobRow)

type Mode = 'sin-memo' | 'memo-inline' | 'memo-
  callback'

export default function MemoDemo() {
  const [mode, setMode] = useState<Mode>('sin-memo')
  const [renders, setRenders] = useState(0)
  const [selectedId, setSelectedId] =
    useState<number | null>(null)
  const ref = useRenderFlash<HTMLDivElement>()

  // Función nueva en cada render del padre
  const handleSelectInline = (id: number) =>
    setSelectedId(id)
  // Misma función entre renders (mientras sus
    dependencias no cambien)
```

```
  const handleSelectStable = useCallback((id:
    number) => setSelectedId(id), [])

  const Row = mode === 'sin-memo' ? JobRow :
    MemoJobRow
  const onSelect = mode === 'memo-callback' ?
    handleSelectStable : handleSelectInline

  return (
    <div ref={ref} className="stack">
      <div className="row">
        <select value={mode} onChange={(e) =>
          setMode(e.target.value as Mode)}>
          <option value="sin-memo">1. Sin
            memo</option>
          <option value="memo-inline">2. memo +
            función inline</option>
          <option value="memo-callback">3. memo +
            useCallback</option>
        </select>
        <button onClick={() => setRenders((n) => n +
          1)}>Re-renderizar el padre ({renders})
          </button>
      </div>
      <ul className="stack" style={{ listStyle:
        'none', padding: 0, margin: 0 }}>
        {jobs.map((job) => (
          <Row key={job.id} job={job} isSelected=
            {job.id === selectedId} onSelect=
            {onSelect} />
        ))}
      </ul>
    </div>
  )
}
```

1. **Sin memo**: cada render del padre renderiza las tres filas.

2. **memo + función inline**: ¡siguen renderizando! `handleSelectInline` es una función **nueva** en cada render, así que la prop `onSelect` "cambió" y memo no sirve de nada.
3. **memo + useCallback**: `useCallback` devuelve **la misma función** entre renders y memo por fin funciona. Solo parpadea la fila cuyo `isSelected` cambia de verdad.

useMemo: no repetir un cálculo caro

`useMemo(() => calcular(a, b), [a, b])` guarda el resultado y solo recalcula cuando cambian las dependencias.

DEMO **Un cálculo de 150 ms en cada tecla**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/UseMemoDemo.tsx`

```
import { useMemo, useState } from 'react'
import { blockFor } from '../shared/utils'

type Offer = { company: string; salary: number;
  remote: boolean }
```

```tsx
const offers: Offer[] = Array.from({ length: 300 },
  (_, i) => ({
  company: `Empresa ${i + 1}`,
  salary: 1800 + ((i * 137) % 3200),
  remote: i % 3 !== 0,
}))

// Cálculo "caro" a propósito: tarda unos 150 ms
function buildReport(minSalary: number) {
  blockFor(150)
  const matching = offers.filter((o) => o.remote &&
    o.salary >= minSalary)
  const average = matching.reduce((sum, o) => sum +
    o.salary, 0) / (matching.length || 1)
  return { count: matching.length, average:
    Math.round(average) }
}

function WithoutMemo() {
  const [minSalary, setMinSalary] = useState(3000)
  const [note, setNote] = useState('')
  const report = buildReport(minSalary) // 🔴 se
    recalcula en CADA render, también al escribir la
    nota
  return <ReportForm {...{ minSalary, setMinSalary,
    note, setNote, report }} />
}

function WithMemo() {
  const [minSalary, setMinSalary] = useState(3000)
  const [note, setNote] = useState('')
  const report = useMemo(() =>
    buildReport(minSalary), [minSalary]) // ✅ solo
    si cambia minSalary
  return <ReportForm {...{ minSalary, setMinSalary,
    note, setNote, report }} />
}

export default function UseMemoDemo() {
```

```
  const [memoized, setMemoized] = useState(false)
  return (
    <div className="stack">
      <label className="row">
        <input type="checkbox" checked={memoized}
          onChange={(e) =>
          setMemoized(e.target.checked)} />
        Usar useMemo
      </label>
      {memoized ? <WithMemo /> : <WithoutMemo />}
    </div>
  )
}

type ReportFormProps = {
  minSalary: number
  setMinSalary: (n: number) => void
  note: string
  setNote: (s: string) => void
  report: { count: number; average: number }
}

function ReportForm({ minSalary, setMinSalary, note,
  setNote, report }: ReportFormProps) {
  return (
    <>
      <label className="row">
        Salario mínimo: USD {minSalary}
        <input
          type="range"
          min={1800}
          max={5000}
          step={100}
          value={minSalary}
          onChange={(e) =>
            setMinSalary(Number(e.target.value))}
        />
      </label>
```

```
      <p>
        {report.count} ofertas remotas · promedio
          USD {report.average}
      </p>
      <input value={note} onChange={(e) =>
        setNote(e.target.value)}
        placeholder="Escribí una nota rápido…" />
    </>
  )
}
```

> **🧪 Probá esto**
>
> Sin `useMemo`, escribí rápido en el campo de la nota: cada tecla recalcula el reporte (150 ms) y el input se traba. Activá `useMemo` y repetí: el reporte solo se recalcula al mover el slider.

useCallback: la misma función entre renders

`useCallback(fn, deps)` es `useMemo` para funciones: `useCallback(fn, deps)` equivale a `useMemo(() => fn, deps)`. Solo tiene sentido cuando:

- pasás la función a un componente envuelto en `memo`, o

- la función es dependencia de un `useEffect` o de otro hook.

Cuándo optimizar (con o sin Compiler)

> 💡 **El React Compiler cambia las reglas**
>
> Desde octubre de 2025 el **React Compiler** es estable. Analiza tu código en el build y agrega la memoización automáticamente: valores, funciones y JSX. En un proyecto con el Compiler activado **casi nunca vas a escribir** `memo`, `useMemo` **ni** `useCallback`. Next.js lo activa con una opción y en Vite se agrega con un plugin. Tiene su propio capítulo con una demo comparativa.

Aun así, tenés que entender estas tres APIs porque:

- Vas a encontrarlas en todo el código existente (el de tu próximo trabajo, seguro).
- Son una pregunta clásica de entrevista.
- Hay casos puntuales donde el Compiler no puede optimizar y hay que hacerlo a mano.

Sin Compiler, la receta es: **medí primero** (React DevTools → Profiler, o las Performance Tracks del

navegador), optimizá solo lo que es lento, y preferí arreglos estructurales antes que memoizar:

- **Bajar el estado**: si solo un input usa un estado, que viva en ese input y no en el padre de toda la página.
- **Pasar JSX como children**: un componente que recibe `children` no re-renderiza esos children cuando cambia su propio estado.
- **Virtualizar** listas muy largas (TanStack Virtual).

> 🎤 **En una entrevista**
>
> "¿Diferencia entre useMemo y useCallback?" → "useMemo memoiza el resultado de un cálculo y useCallback memoiza la función misma. useCallback sirve cuando la función va a un componente con memo o es dependencia de un efecto. Hoy, con el React Compiler, esa memoización se hace automáticamente en el build, así que en código nuevo casi no las escribo a mano, pero las uso para entender y mantener código existente."

Capítulo 12

useTransition y useDeferredValue

Actualizaciones urgentes y no urgentes para que la UI nunca se trabe.

Desde React 18, no todas las actualizaciones tienen la misma prioridad:

- **Urgentes**: escribir en un input, hacer click, arrastrar. El usuario espera respuesta inmediata.
- **No urgentes (transiciones)**: mostrar los resultados de una búsqueda, cambiar de pestaña, navegar a otra página. Si tardan un poco, nadie se da cuenta.

Con el renderizado concurrente, React puede **interrumpir** un render no urgente para atender uno urgente. Las dos herramientas para marcar algo como no urgente son `useTransition` y `useDeferredValue`.

useTransition: marcar una actualización como no urgente

```
const [isPending, startTransition] = useTransition()

function selectTab(tab: Tab) {
  startTransition(() => {
    setTab(tab) // esta actualización puede esperar
  })
}
```

Mientras la transición se procesa, `isPending` vale `true` y la UI vieja **sigue siendo interactiva**.

DEMO **Pestañas que no traban la UI**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/TransitionTabs.tsx`

```
import { useState, useTransition } from 'react'
import { blockFor } from '../shared/utils'

type Tab = 'resumen' | 'postulaciones' | 'contacto'

function SlowRow({ index }: { index: number }) {
  blockFor(1.5) // 300 filas x 1.5 ms = casi medio
    segundo de render
  return <li>Postulación #{index + 1}</li>
}
```

```
function ApplicationsTab() {
  return (
    <ul style={{ margin: 0, paddingLeft: '1.2em',
      maxHeight: 180, overflow: 'auto' }}>
      {Array.from({ length: 300 }, (_, i) => (
        <SlowRow key={i} index={i} />
      ))}
    </ul>
  )
}

export default function TransitionTabs() {
  const [tab, setTab] = useState<Tab>('resumen')
  const [transitionOn, setTransitionOn] =
    useState(true)
  const [isPending, startTransition] =
    useTransition()

  function selectTab(next: Tab) {
    if (transitionOn) {
      // Actualización NO urgente: React puede
        interrumpirla si hacés otro click.
      startTransition(() => setTab(next))
    } else {
      setTab(next) // urgente: bloquea la UI hasta
        terminar de renderizar
    }
  }

  return (
    <div className="stack">
      <label className="row">
        <input type="checkbox" checked=
          {transitionOn} onChange={(e) =>
          setTransitionOn(e.target.checked)} />
        Usar startTransition
      </label>
      <div className="row">
```

```
        <button className={tab === 'resumen' ?
          'active' : ''} onClick={() =>
          selectTab('resumen')}>
          Resumen
        </button>
        <button className={tab === 'postulaciones' ?
          'active' : ''} onClick={() =>
          selectTab('postulaciones')}>
          Postulaciones (lenta)
        </button>
        <button className={tab === 'contacto' ?
          'active' : ''} onClick={() =>
          selectTab('contacto')}>
          Contacto
        </button>
        {isPending && <span
          className="muted">Cargando pestaña…
          </span>}
      </div>
      <div className={`card ${isPending ? 'pending'
        : ''}`}>
        {tab === 'resumen' && <p>3 entrevistas esta
          semana, 1 oferta pendiente.</p>}
        {tab === 'postulaciones' && <ApplicationsTab
          />}
        {tab === 'contacto' &&
          <p>recruiting@ejemplo.com</p>}
      </div>
    </div>
  )
}
```

🧪 Probá esto

Con `startTransition` desactivado, tocá **Postulaciones (lenta)** y enseguida **Contacto**: la UI se congela casi medio segundo antes de responder. Activalo y repetí: el click en Contacto interrumpe el render lento y responde al instante, y mientras tanto ves "Cargando pestaña...".

También existe `startTransition` importado directo de `react`, sin hook, para cuando no necesitás `isPending`.

ℹ️ Transiciones y React 19

En React 19 la función que le pasás a `startTransition` puede ser **async**. Eso se llama una **Action**, y es la base de todo lo nuevo: `useActionState`, `useOptimistic`, formularios con `action`. Lo vemos a fondo en el capítulo de Actions. Y desde React 19.3 las transiciones se renderizan de forma independiente: una transición lenta ya no frena a otras que no tienen nada que ver.

useDeferredValue: una copia "atrasada" de un valor

A veces no controlás el `setState` (viene de una prop, o el input tiene que actualizarse sí o sí al instante). `useDeferredValue` te da una versión del valor que **se queda atrás** mientras haya trabajo urgente:

```
const [query, setQuery] = useState('')
const deferredQuery = useDeferredValue(query)

// El input usa query (urgente); la lista lenta usa
  deferredQuery (puede esperar)
<input value={query} onChange={(e) =>
  setQuery(e.target.value)} />
<SlowList query={deferredQuery} />
```

DEMO **Búsqueda fluida sobre una lista lenta**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/DeferredSearch.tsx`

```
import { memo, useDeferredValue, useState } from
  'react'
import { blockFor } from '../shared/utils'

const companies = Array.from({ length: 250 }, (_, i)
  => `Empresa ${String(i + 1).padStart(3, '0')}`)
```

```
function SlowItem({ text }: { text: string }) {
  blockFor(1) // cada ítem tarda 1 ms en renderizar:
    250 ítems = 250 ms
  return <li>{text}</li>
}

// memo es clave: así la lista se saltea el render
  urgente (con el valor diferido viejo).
const SlowList = memo(function SlowList({ query }: {
  query: string }) {
  const items = companies.filter((c) =>
    c.includes(query))
  return (
    <ul style={{ margin: 0, paddingLeft: '1.2em',
      columns: 3 }}>
      {items.map((c) => (
        <SlowItem key={c} text={c} />
      ))}
    </ul>
  )
})

export default function DeferredSearch() {
  const [query, setQuery] = useState('')
  const [deferOn, setDeferOn] = useState(true)
  const deferredQuery = useDeferredValue(query)
  const listQuery = deferOn ? deferredQuery : query
  const isStale = listQuery !== query

  return (
    <div className="stack">
      <label className="row">
        <input type="checkbox" checked={deferOn}
          onChange={(e) =>
          setDeferOn(e.target.checked)} />
        Usar useDeferredValue
      </label>
      <input
```

```
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Escribí números rápido: 1, 12,
          123…"
      />
      <div className={isStale ? 'pending' : ''}
        style={{ maxHeight: 220, overflow: 'auto'
        }}>
        <SlowList query={listQuery} />
      </div>
    </div>
  )
}
```

🧪 Probá esto

Desactivá `useDeferredValue` y escribí "123" rápido: cada tecla tarda un cuarto de segundo en aparecer. Activalo y repetí: el input responde al instante y la lista se actualiza cuando puede (mientras está desactualizada se ve más clara, gracias a `isStale`).

> ⚠️ **memo es parte del truco**
>
> `SlowList` está envuelta en `memo`. En el render urgente, `deferredQuery` todavía tiene el valor viejo, así que memo se saltea la lista y el render es instantáneo. Sin `memo`, la lista se renderizaría igual en el render urgente y no ganarías nada. Con el React Compiler esa memoización es automática.

¿Debounce o useDeferredValue?

	Debounce	useDeferredValue
Qué resuelve	Hacer **menos requests** a una API	Mantener fluida una UI con **render lento**
Cuánto espera	Un tiempo fijo (por ejemplo 400 ms)	Lo que tarde el render, adaptado al dispositivo
Se puede interrumpir	No	Sí, React descarta el render viejo

Muchas veces se combinan: debounce para la request y `useDeferredValue` para pintar los resultados.

🎤 En una entrevista

"¿Qué es una transición en React?" → "Una actualización marcada como no urgente con startTransition o useTransition. React la puede interrumpir para atender interacciones urgentes como escribir o hacer click, y mientras tanto muestra la UI anterior con un isPending. useDeferredValue es la versión para valores: una copia atrasada que se actualiza con baja prioridad."

Capítulo 13

Suspense, lazy y Error Boundaries

Estados de carga declarativos, code splitting y cómo atrapar errores de render.

Dos componentes especiales manejan los dos "estados raros" de cualquier UI: **cargando** y **error**. En vez de llenar cada componente de `if (isLoading)` y `if (error)`, los declarás una vez, más arriba en el árbol.

```
<ErrorBoundary fallback={<p>Algo salió mal</p>}>
  <Suspense fallback={<Spinner />}>
    <Perfil />
  </Suspense>
</ErrorBoundary>
```

Suspense: estados de carga declarativos

Cuando un componente **suspende** (porque está esperando código o datos), React muestra el `fallback` del `<Suspense>` más cercano hasta que todo lo de adentro esté listo.

Un componente puede suspender por:

- **Código**: un componente cargado con `lazy()` que todavía se está descargando.
- **Datos**: una promesa leída con use() (React 19), o datos de un framework compatible con Suspense (Next.js, React Router, TanStack Query con `useSuspenseQuery`).

> ⚠️ **Suspense no detecta fetch en useEffect**
>
> Suspense solo funciona con fuentes de datos preparadas para él. Si pedís datos en un `useEffect` y guardás un `isLoading`, Suspense no se entera.

lazy: cargar componentes cuando hacen falta

`lazy()` divide tu bundle: el código del componente se descarga recién la primera vez que se renderiza. Ideal para pantallas pesadas, modales o editores que no todos los usuarios abren.

DEMO **Code splitting con lazy y Suspense**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/LazyDemo.tsx`

```
import { lazy, Suspense, useState } from 'react'
import { sleep } from '../shared/utils'

// lazy recibe una función que devuelve un import()
  dinámico.
// Le sumamos 1,5 s de espera para simular una red
  lenta.
const SalaryChart = lazy(() => sleep(1500).then(()
  => import('./SalaryChart')))

export default function LazyDemo() {
  const [showChart, setShowChart] = useState(false)

  return (
    <div className="stack">
      <button onClick={() => setShowChart(true)}
        disabled={showChart}>
        Ver gráfico de salarios
```

```
      </button>
      {showChart && (
        // Mientras el código del componente se
          descarga, se muestra el fallback
        <Suspense fallback={<p className="muted">⏳
          Descargando el componente…</p>}>
          <SalaryChart />
        </Suspense>
      )}
    </div>
  )
}
```

src/demos/hooks/SalaryChart.tsx

```
// Componente "pesado" que se carga con lazy()
  recién cuando hace falta.
const data = [
  { role: 'Junior', salary: 1500 },
  { role: 'Semi Senior', salary: 2800 },
  { role: 'Senior', salary: 4200 },
  { role: 'Lead', salary: 5500 },
]

export default function SalaryChart() {
  const max = Math.max(...data.map((d) => d.salary))
  return (
    <div className="stack" aria-label="Salarios
      remotos en USD por seniority">
      {data.map((d) => (
        <div key={d.role} className="row" style={{
          flexWrap: 'nowrap' }}>
          <span style={{ width: 100, flex: 'none'
            }}>{d.role}</span>
          <div
            style={{
              height: 18,
              width: `${(d.salary / max) * 100}%`,
```

```
              background: 'var(--accent)',
              borderRadius: 4,
            }}
          />
          <span style={{ flex: 'none' }}>USD
            {d.salary}</span>
        </div>
      ))}
    </div>
  )
}
```

Si reiniciás la demo y volvés a tocar el botón, el gráfico aparece al instante: `lazy` recuerda el módulo que ya descargó.

> ℹ️ **Nota**
>
> El archivo de `lazy()` tiene que tener un `export default` con el componente. En Next.js no hace falta usar `lazy` para las páginas: el framework divide el código por ruta automáticamente.

Suspense y transiciones juntas

Si un contenido que **ya se estaba mostrando** vuelve a suspender (por ejemplo, porque cambiaste de pestaña y la nueva pide datos), React reemplaza todo por el fallback. Eso suele verse mal: la pantalla

"parpadea". Si el cambio ocurre dentro de una **transición**, React mantiene la UI vieja visible mientras prepara la nueva. Lo vas a ver en acción en el capítulo de use().

Error Boundaries: atrapar errores de render

Si un componente lanza un error mientras renderiza y nadie lo atrapa, React desmonta **toda la app** y queda la pantalla en blanco. Un Error Boundary atrapa los errores de su subárbol y muestra un fallback en su lugar.

DEMO **Un componente que explota**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/ErrorBoundaryDemo.tsx`

```
import { useState } from 'react'
import { ErrorBoundary } from
  '../shared/ErrorBoundary'

function OfferCounter() {
  const [offers, setOffers] = useState(0)
  if (offers === 3) {
    // Un error durante el render: lo atrapa el
      Error Boundary más cercano
```

```
    throw new Error('¡Demasiadas ofertas! (error de
      render simulado)')
  }
  return (
    <div className="row">
      <span>Ofertas: {offers}</span>
      <button onClick={() => setOffers((n) => n +
        1)}>+1 (en la 3 explota)</button>
    </div>
  )
}

export default function ErrorBoundaryDemo() {
  return (
    <ErrorBoundary
      fallback={(error, reset) => (
        <div className="card stack">
          <p className="error-text">Algo salió mal:
            {error.message}</p>
          <button onClick=
            {reset}>Reintentar</button>
        </div>
      )}
    >
      <OfferCounter />
    </ErrorBoundary>
  )
}
```

src/demos/shared/ErrorBoundary.tsx

```
import { Component, type ErrorInfo, type ReactNode }
  from 'react'

type Props = {
  fallback: (error: Error, reset: () => void) =>
    ReactNode
  children: ReactNode
```

```tsx
}
type State = { error: Error | null }

/**
 * Los Error Boundaries siguen siendo el único caso
   donde necesitás una clase
 * (o una librería como react-error-boundary, que la
   trae hecha).
 */
export class ErrorBoundary extends Component<Props,
  State> {
  state: State = { error: null }

  // 1) Durante el render: actualiza el estado para
    mostrar el fallback.
  static getDerivedStateFromError(error: Error):
    State {
    return { error }
  }

  // 2) Después del commit: ideal para loguear el
    error.
  componentDidCatch(error: Error, info: ErrorInfo) {
    console.info('Error atrapado:', error.message,
      info.componentStack)
  }

  reset = () => this.setState({ error: null })

  render() {
    if (this.state.error) return
      this.props.fallback(this.state.error,
      this.reset)
    return this.props.children
  }
}
```

Abrí la pestaña `ErrorBoundary.tsx`: es una **clase**. Los Error Boundaries siguen siendo el único caso que todavía necesita una clase, porque dependen de `getDerivedStateFromError` y `componentDidCatch`, que no tienen equivalente en hooks. En la práctica casi todos usan la librería **react-error-boundary**, que ya la trae hecha:

```
import { ErrorBoundary } from 'react-error-boundary'

<ErrorBoundary fallbackRender={({ error,
  resetErrorBoundary }) => (
  <div>
    <p>Error: {error.message}</p>
    <button onClick=
      {resetErrorBoundary}>Reintentar</button>
  </div>
)}>
  <Dashboard />
</ErrorBoundary>
```

Qué atrapa y qué no

Lo atrapa ✅	No lo atrapa ❌
Errores durante el render	Errores en event handlers (`onClick`)
Errores en Effects y lifecycle	Código async suelto (`setTimeout`, promesas sin esperar)

Lo atrapa ✅	No lo atrapa ❌
Promesas rechazadas leídas con `use()`	Errores en el propio Error Boundary
Errores lanzados dentro de una **Action** (React 19)	Errores del servidor en SSR (los maneja el framework)

Para los errores de event handlers usás `try/catch` y guardás el error en el estado. En React 19, las Actions simplifican esto: si una Action lanza un error, va al Error Boundary más cercano.

> **💡 ¿Dónde poner los boundaries?**
>
> No uno solo en la raíz (todo se cae junto) ni uno por componente (demasiado). Lo habitual: uno general en la raíz, uno por sección o widget independiente (el sidebar, el feed, un gráfico) y uno por ruta. En Next.js, el archivo `error.tsx` de cada carpeta de rutas crea un Error Boundary automáticamente, y `loading.tsx` crea un `<Suspense>`.

🎤 En una entrevista

"¿Qué es Suspense?" → "Un componente que muestra un fallback mientras algo de su subárbol está esperando código o datos. Funciona con lazy para code splitting y con fuentes de datos compatibles, como use() en React 19 o los frameworks. Combinado con transiciones evita que la UI ya visible vuelva al spinner. Para los errores se complementa con los Error Boundaries."

Capítulo 14

useId, useLayoutEffect, portales y más

Los hooks menos frecuentes que igual conviene conocer.

Estos hooks y APIs se usan menos, pero aparecen en código real, en librerías y en entrevistas. Conviene saber qué hace cada uno y cuándo tiene sentido.

useId: ids únicos para accesibilidad

Para conectar un `<label>` con su `<input>`, o un input con su texto de ayuda (`aria-describedby`), necesitás ids únicos. Si el componente se usa varias veces, un id fijo como `"email"` se repetiría. `useId` genera uno distinto por instancia, estable entre el servidor y el cliente.

DEMO **Un id distinto por cada campo**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/UseIdDemo.tsx`

```tsx
import { useId } from 'react'

function Field({ label, hint }: { label: string;
  hint: string }) {
  // Un id único y estable por cada instancia del
    componente (también con SSR)
  const id = useId()
  return (
    <div className="stack" style={{ gap: 4 }}>
      <label htmlFor={id}>{label}</label>
      <input id={id} aria-describedby={`${id}-hint`}
        />
      <small id={`${id}-hint`} className="muted">
        {hint} · id generado: <code>{id}</code>
      </small>
    </div>
  )
}

export default function UseIdDemo() {
  return (
    <div className="grid-2">
      <Field label="Empresa" hint="Nombre legal o
        comercial" />
      <Field label="Puesto" hint="Como figura en la
        oferta" />
    </div>
  )
```

```
}
```

> ⚠️ **Ojo**
>
> `useId` no sirve para las `key` de una lista: las keys tienen que salir de tus datos.

createPortal: renderizar en otro lugar del DOM

Modales, tooltips y menús desplegables tienen que escaparse de contenedores con `overflow: hidden` o `z-index` bajos. `createPortal(jsx, nodo)` renderiza el JSX en otro nodo del DOM (casi siempre `document.body`), pero **sigue siendo parte del árbol de React**: recibe contexto, mantiene su estado y los eventos burbujean por el árbol de React, no por el del DOM.

DEMO **Un modal que escapa del overflow: hidden**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/hooks/PortalModal.tsx

```tsx
import { useEffect, useState, type ReactNode } from
  'react'
import { createPortal } from 'react-dom'

function Modal({ title, onClose, children }: {
  title: string; onClose: () => void; children:
  ReactNode }) {
  // Cerrar con Escape: sincronizar con un evento
    del navegador
  useEffect(() => {
    function onKeyDown(e: KeyboardEvent) {
      if (e.key === 'Escape') onClose()
    }
    window.addEventListener('keydown', onKeyDown)
    return () =>
      window.removeEventListener('keydown',
      onKeyDown)
  }, [onClose])

  // createPortal: el JSX se renderiza en
    document.body, fuera de este div,
  // pero sigue siendo parte del árbol de React
    (contexto, eventos, estado).
  return createPortal(
    <div
      role="dialog"
      aria-modal="true"
      aria-label={title}
      onClick={onClose}
      style={{ position: 'fixed', inset: 0,
        background: 'rgba(0,0,0,.45)', display:
        'grid', placeItems: 'center', zIndex: 100 }}
    >
      <div className="card stack" onClick={(e) =>
        e.stopPropagation()} style={{ width:
        'min(90vw, 360px)' }}>
```

```
        <strong>{title}</strong>
        {children}
        <button className="ui-button" onClick=
          {onClose}>
          Cerrar
        </button>
      </div>
    </div>,
    document.body,
  )
}

export default function PortalModal() {
  const [open, setOpen] = useState(false)
  return (
    // overflow: hidden recortaría un modal normal;
      el portal lo evita
    <div style={{ overflow: 'hidden', border: '1px
      dashed var(--border)', borderRadius: 8,
      padding: 12 }}>
      <button onClick={() => setOpen(true)}>Abrir
        modal</button>
      {open && (
        <Modal title="¿Postularte a Nubelia?"
          onClose={() => setOpen(false)}>
          <p>Este modal vive en document.body
            gracias a createPortal.</p>
        </Modal>
      )}
    </div>
  )
}
```

> 💡 **El elemento dialog**
>
> El elemento nativo `<dialog>` con `showModal()` ya resuelve el apilamiento, el foco y la tecla Escape, y desde React 19.1 React soporta sus eventos `toggle` y `beforetoggle`. Para muchos modales simples es una gran alternativa.

useSyncExternalStore: suscribirse a un store externo

Es la forma correcta de leer datos que viven **fuera de React** y cambian con el tiempo: APIs del navegador (`navigator.onLine`, `matchMedia`) o stores de librerías. Es exactamente lo que usan Zustand, Redux y Jotai por dentro.

```
const value = useSyncExternalStore(subscribe,
  getSnapshot, getServerSnapshot?)
```

- `subscribe(callback)`: se suscribe a los cambios y devuelve la función para desuscribirse.
- `getSnapshot()`: devuelve el valor actual. Tiene que devolver el **mismo** valor (o el mismo objeto) si nada cambió.

DEMO **Un store de 15 líneas compartido por dos componentes**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/ExternalStore.tsx`

```tsx
import { useSyncExternalStore } from 'react'

// Un "store" mínimo fuera de React (así funcionan
  Zustand, Redux, etc. por dentro)
function createStore<T>(initial: T) {
  let state = initial
  const listeners = new Set<() => void>()
  return {
    getState: () => state,
    setState(next: T) {
      state = next
      listeners.forEach((listener) => listener())
    },
    subscribe(listener: () => void) {
      listeners.add(listener)
      return () => listeners.delete(listener)
    },
  }
}

const savedJobsStore = createStore<string[]>
  (['Nubelia'])

function useSavedJobs() {
  return
    useSyncExternalStore(savedJobsStore.subscribe,
    savedJobsStore.getState)
}
```

```
// Dos componentes que NO comparten padre ni props,
  pero leen el mismo store
function SaveButtons() {
  const saved = useSavedJobs()
  return (
    <div className="row">
      {['Nubelia', 'Pagos Sur',
        'Datalab'].map((company) => (
        <button
          key={company}
          className={saved.includes(company) ?
            'active' : ''}
          onClick={() =>
            savedJobsStore.setState(
              saved.includes(company) ?
                saved.filter((c) => c !== company) :
                [...saved, company],
            )
          }
        >
          {saved.includes(company) ? '★' : '☆'}
            {company}
        </button>
      ))}
    </div>
  )
}

function SavedCounter() {
  const saved = useSavedJobs()
  return <p className="muted">Guardadas:
    {saved.length ? saved.join(', ') : 'ninguna'}
    </p>
}

export default function ExternalStoreDemo() {
  return (
    <div className="stack">
```

```
      <SaveButtons />
      <SavedCounter />
    </div>
  )
}
```

Este proyecto lo usa en la vida real: `src/lib/useColorScheme.ts` lee la preferencia de tema oscuro del sistema con `useSyncExternalStore`.

useImperativeHandle: exponer una API acotada por ref

Normalmente, si un padre le pasa un `ref` a tu componente, termina apuntando a un nodo del DOM. Con `useImperativeHandle` decidís **qué métodos** recibe el padre en vez del nodo completo.

DEMO **Un input que solo expone focus() y clear()**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/hooks/ImperativeHandle.tsx`

```
import { useImperativeHandle, useRef, type Ref }
  from 'react'
```

```tsx
// Lo único que el padre va a poder hacer con el ref
type SearchHandle = { focus: () => void; clear: ()
  => void }

// React 19: ref es una prop más (sin forwardRef)
function SearchInput({ ref, placeholder }: { ref?:
  Ref<SearchHandle>; placeholder: string }) {
  const inputRef = useRef<HTMLInputElement>(null)

  // En vez de exponer el <input> entero, exponemos
    una API acotada
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current?.focus(),
    clear: () => {
      if (inputRef.current) inputRef.current.value =
        ''
    },
  }))

  return <input ref={inputRef} placeholder=
    {placeholder} />
}

export default function ImperativeHandleDemo() {
  const searchRef = useRef<SearchHandle>(null)
  return (
    <div className="row">
      <SearchInput ref={searchRef}
        placeholder="Buscar ofertas…" />
      <button onClick={() =>
        searchRef.current?.focus()}>Enfocar</button>
      <button onClick={() =>
        searchRef.current?.clear()}>Limpiar</button>
    </div>
  )
}
```

Fijate que `SearchInput` recibe `ref` como una prop común: en React 19 ya no hace falta `forwardRef`. Usalo con moderación: la mayoría de las cosas se resuelven mejor con props.

useLayoutEffect: medir antes de pintar

Igual que `useEffect`, pero corre **antes** de que el navegador pinte. Se usa para medir el DOM y corregir algo sin que el usuario vea el "salto":

```
function Tooltip({ targetRect, children }:
  TooltipProps) {
  const ref = useRef<HTMLDivElement>(null)
  const [height, setHeight] = useState(0)

  useLayoutEffect(() => {
    // Medimos el tooltip ya renderizado, antes de
      que se vea en pantalla

      setHeight(ref.current!.getBoundingClientRect()
      .height)
  }, [])

  // Si no entra arriba del elemento, lo ponemos
    abajo
  const top = targetRect.top - height < 0 ?
    targetRect.bottom : targetRect.top - height
  return <div ref={ref} style={{ position: 'fixed',
    top }}>{children}</div>
}
```

Bloquea el pintado, así que usalo solo cuando `useEffect` produce un parpadeo visible.

Los que casi no vas a usar

API	Para qué
`useInsertionEffect`	Para autores de librerías CSS-in-JS: insertar estilos antes que los layout effects.
`useDebugValue`	Mostrar una etiqueta en React DevTools para un custom hook.
`flushSync`	Forzar que una actualización se aplique al DOM de forma síncrona (por ejemplo, antes de hacer scroll).
`<Profiler>`	Medir tiempos de render por código.
`cloneElement`, `Children`	APIs legacy para manipular children. Hoy se prefieren props explícitas o render props.

🎤 En una entrevista

"¿Para qué sirve createPortal?" → "Para renderizar un componente en otro nodo del DOM, como document.body, sin sacarlo del árbol de React. Se usa en modales, tooltips y dropdowns que tienen que escaparse de un overflow hidden o de un z-index. Sigue recibiendo contexto y los eventos burbujean por el árbol de React."

Parte 3 · React 19 a fondo

Actions, formularios, estado optimista, use() y todos los cambios de React 19.

Capítulo 15

Actions: la gran idea de React 19

React 19.0

Transiciones async que manejan pending, errores y formularios por vos.

Si tuvieras que explicar React 19 en una sola palabra, sería **Actions**. Casi todo lo nuevo (`useActionState`, `useOptimistic`, `useFormStatus`, `<form action>`) gira alrededor de esta idea.

El problema: siempre el mismo código repetido

Cada vez que el usuario hace algo que llama a una API, escribíamos lo mismo: un `isPending`, un `error`, un `try/catch/finally`, deshabilitar el botón y, si es un formulario, `preventDefault` y resetear los campos.

× React 18: todo a mano

```
function UpdateName() {
  const [name, setName] = useState('')
  const [error, setError] = useState(null)
  const [isPending, setIsPending] = useState(false)

  async function handleSubmit() {
    setIsPending(true)
    try {
      const error = await updateName(name)
      if (error) {
        setError(error)
        return
      }
      redirect('/perfil')
    } finally {
      setIsPending(false)
    }
  }
  // ...
}
```

✓ React 19: una Action

```
function UpdateName() {
  const [name, setName] = useState('')
  const [error, setError] = useState(null)
  const [isPending, startTransition] =
    useTransition()

  function handleSubmit() {
    startTransition(async () => {
      const error = await updateName(name)
      if (error) {
        setError(error)
        return
      }
      redirect('/perfil')
    })
  }
  // ...
}
```

Qué es una Action

> Una **Action** es una función que se ejecuta dentro de una transición. En React 19 esa función puede ser **async**.

Cuando una Action está en curso, React se encarga de:

- **Estado pendiente**: `isPending` es `true` desde que la Action empieza hasta que termina toda la

función, incluidos los `await`.

- **Errores**: si la Action lanza un error, lo podés mostrar con un Error Boundary.
- **Actualizaciones optimistas**: con useOptimistic mostrás el resultado esperado al instante.
- **Formularios**: `<form action={fn}>` ejecuta `fn` como Action y resetea el formulario al terminar.
- **UI responsiva**: como es una transición, la interfaz sigue respondiendo mientras tanto.

DEMO **Una Action con useTransition**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/UpdateName.tsx`

```
import { startTransition, useState, useTransition }
  from 'react'
import { updateName } from './api'

export default function UpdateName() {
  const [name, setName] = useState('')
  const [savedName, setSavedName] = useState('Ana')
  const [error, setError] = useState<string | null>
    (null)
  const [isPending, startUpdate] = useTransition()

  function handleSubmit() {
    // Una Action: una función async dentro de una
      transición.
```

```
    // isPending es true desde que empieza hasta que
      termina TODA la función.
    startUpdate(async () => {
      setError(null)
      const error = await updateName(name)
      // Después de un await, envolvé los setState
        en startTransition
      // (limitación actual de React, documentada en
        react.dev).
      startTransition(() => {
        if (error) {
          setError(error)
        } else {
          setSavedName(name)
          setName('')
        }
      })
    })
  }
```

```
  return (
    <div className="stack">
      <p>
        Nombre guardado: <strong>{savedName}
          </strong>
      </p>
      <div className="row">
        <input value={name} onChange={(e) =>
          setName(e.target.value)}
          placeholder="Nuevo nombre" />
        <button onClick={handleSubmit} disabled=
          {isPending}>
          {isPending ? 'Guardando…' : 'Actualizar'}
        </button>
      </div>
      {error && <p className="error-text">{error}
        </p>}
    </div>
  )
```

```
}
```

src/demos/react19/api.ts

```
import { sleep } from '../shared/utils'

// "Servidor" falso para las demos de React 19. Cada
  función tarda como una API real.

export async function updateName(name: string):
  Promise<string | null> {
  await sleep(1000)
  if (name.trim().length < 3) return 'El nombre
    tiene que tener al menos 3 letras.'
  return null // null = sin error
}

export async function saveOffer(company: string) {
  await sleep(900)
  return `Oferta de ${company} guardada a las ${new
    Date().toLocaleTimeString('es')}`
}

export async function updateInterviews(current:
  number, delta: number) {
  await sleep(600)
  return Math.max(0, current + delta)
}
```

🧪 **Probá esto**

Escribí "AI" (2 letras) y tocá **Actualizar**: el botón queda deshabilitado durante el segundo que tarda la "API" y después aparece el error. Con un nombre más largo, se guarda. Fijate que no hay ningún `setIsPending` en el código.

⚠️ **setState después de un await**

Hoy, las actualizaciones de estado que hacés **después de un** `await` no quedan automáticamente dentro de la transición. La documentación oficial recomienda envolverlas en otro `startTransition`, como en la demo. Es una limitación conocida que el equipo de React planea resolver; mientras tanto, es el patrón correcto.

Action props: componentes que aceptan Actions

Una convención nueva: si una prop de tu componente se ejecuta dentro de una transición, la llamás `action` (o `algoAction`, como `submitAction` o `changeAction`). El componente se encarga del

estado pendiente y quien lo usa solo dice **qué** hacer.

DEMO **Un botón de design system con prop action**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/ActionButton.tsx`

```tsx
import { startTransition, useState, useTransition,
  type ReactNode } from 'react'
import { saveOffer } from './api'

type ActionButtonProps = {
  /** Convención: las props que se ejecutan dentro
    de una transición se llaman "action" */
  action: () => void | Promise<void>
  children: ReactNode
}

// Componente de "design system": maneja su propio
  estado pendiente.
function ActionButton({ action, children }:
  ActionButtonProps) {
  const [isPending, startTransition] =
    useTransition()
  return (
    <button
      disabled={isPending}
      onClick={() => {
        startTransition(async () => {
          await action() // puede ser sync o async
        })
```

```
      }}
    >
      {isPending ? '⏳ ' : ''}
      {children}
    </button>
  )
}

export default function ActionButtonDemo() {
  const [message, setMessage] = useState('')

  return (
    <div className="stack">
      <div className="row">
        {/* El padre solo dice QUÉ hacer; el botón
          se encarga del pending */}
        <ActionButton
          action={async () => {
            const result = await
              saveOffer('Nubelia')
            startTransition(() =>
              setMessage(result))
          }}
        >
          Guardar oferta de Nubelia
        </ActionButton>
        <ActionButton
          action={async () => {
            const result = await
              saveOffer('Datalab')
            startTransition(() =>
              setMessage(result))
          }}
        >
          Guardar oferta de Datalab
        </ActionButton>
      </div>
      {message && <p className="ok-text">{message}
        </p>}
```

```
    </div>
  )
}
```

src/demos/react19/api.ts

```
import { sleep } from '../shared/utils'

// "Servidor" falso para las demos de React 19. Cada
  función tarda como una API real.

export async function updateName(name: string):
  Promise<string | null> {
  await sleep(1000)
  if (name.trim().length < 3) return 'El nombre
    tiene que tener al menos 3 letras.'
  return null // null = sin error
}

export async function saveOffer(company: string) {
  await sleep(900)
  return `Oferta de ${company} guardada a las ${new
    Date().toLocaleTimeString('es')}`
}

export async function updateInterviews(current:
  number, delta: number) {
  await sleep(600)
  return Math.max(0, current + delta)
}
```

Esto es lo que van a hacer las librerías de componentes: botones, tabs, selects y date pickers que aceptan una `action` y muestran solos su estado de carga.

> **ℹ️ Actions ≠ Server Actions**
>
> Una **Action** es cualquier función en una transición: corre en el navegador. Una **Server Function** (antes llamada Server Action) es una función marcada con `"use server"` que corre en el servidor y que podés usar *como* Action. Las ves en la Parte 5.

Mapa de las APIs nuevas

API	Para qué
`startTransition(async () => …)`	Ejecutar una Action y obtener `isPending` (con `useTransition`).
`useActionState`	Estado que se actualiza con el resultado de una Action.
`<form action={fn}>`	Enviar formularios con Actions, con reset automático.
`useFormStatus`	Leer desde un hijo si el form padre se está enviando.
`useOptimistic`	Mostrar el resultado esperado mientras la Action termina.

🎤 En una entrevista

"¿Qué es una Action en React 19?" → "Una función, normalmente async, que corre dentro de una transición. React maneja automáticamente el estado pendiente, los errores, las actualizaciones optimistas y el reset de formularios. Es la base de useActionState, useOptimistic y los formularios con action, y elimina el boilerplate de isLoading y try/finally."

Capítulo 16

useActionState

React 19.0

Estado que se actualiza con el resultado de una Action: ideal para formularios.

`useActionState` junta en un solo hook lo que casi toda Action necesita: **el resultado** de la última ejecución (un mensaje, errores de validación, datos guardados) y **si está pendiente**.

```
const [state, dispatchAction, isPending] =
  useActionState(reducerAction, initialState)
```

- `reducerAction(previousState, payload)`: tu función. Recibe el estado anterior y lo que le pases al despacharla; devuelve el estado nuevo. **Puede ser async.**
- `initialState`: el estado antes de la primera ejecución.
- `dispatchAction`: la función que dispara la Action. Se la pasás a un `<form action>` o la

llamás dentro de `startTransition`.

- `isPending`: `true` mientras haya una ejecución en curso.

Fijate el nombre `reducerAction`: es literalmente un **reducer que puede ser async y tener efectos secundarios** (como llamar a una API). Si entendiste useReducer, ya entendiste la mitad.

El caso estrella: un formulario con validación

Este es el formulario "Nueva postulación" del Job Tracker, sin un solo `useState`:

DEMO **Formulario con validación del lado del 'servidor'**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/NewApplicationForm.tsx`

```
import { useActionState } from 'react'
import { createApplication, initialFormState } from
  './applications-api'

export default function NewApplicationForm() {
  // [estado, acción para el form, ¿está pendiente?]
```

```
  const [state, formAction, isPending] =
    useActionState(createApplication,
    initialFormState)

  return (
    <div className="grid-2">
      {/* Sin onSubmit, sin preventDefault, sin
        useState por campo */}
      <form action={formAction} className="stack">
        <label className="stack">
          Empresa
          <input name="company" defaultValue=
            {state.values.company} />
          {state.errors.company && <span
            className="error-text">
            {state.errors.company}</span>}
        </label>
        <label className="stack">
          Puesto
          <input name="role" defaultValue=
            {state.values.role} />
          {state.errors.role && <span
            className="error-text">
            {state.errors.role}</span>}
        </label>
        <label className="stack">
          Salario mensual (USD)
          <input name="salary" type="number"
            defaultValue={state.values.salary} />
          {state.errors.salary && <span
            className="error-text">
            {state.errors.salary}</span>}
        </label>
        <button type="submit" disabled={isPending}>
          {isPending ? 'Guardando…' : 'Guardar
            postulación'}
        </button>
        {state.message && (
          <p className={state.status === 'error' ?
            'error-text' : 'ok-text'}>
```

```
            {state.message}</p>
        )}
      </form>

      <div className="card">
        <strong>Guardadas ({state.saved.length})
          </strong>
        <ul style={{ margin: '6px 0 0', paddingLeft:
          '1.2em' }}>
          {state.saved.map((app) => (
            <li key={app.id}>
              {app.company} · {app.role} · USD
                {app.salary}
            </li>
          ))}
        </ul>
      </div>
    </div>
  )
}
```

src/demos/react19/applications-api.ts

```
import { sleep } from '../shared/utils'

// Imaginá que esto es una Server Function en
  Next.js ("use server").
// Recibe el estado anterior y el FormData, valida y
  devuelve el estado nuevo.

export type Application = { id: number; company:
  string; role: string; salary: number }

export type FormState = {
  status: 'idle' | 'error' | 'success'
  message: string
  errors: Partial<Record<'company' | 'role' |
    'salary', string>>
```

```
  values: { company: string; role: string; salary:
    string } // para no perder lo escrito si hay
    error
  saved: Application[]
}

export const initialFormState: FormState = {
  status: 'idle',
  message: '',
  errors: {},
  values: { company: '', role: '', salary: '' },
  saved: [],
}

export async function createApplication(prevState:
  FormState, formData: FormData): Promise<FormState>
  {
  const values = {
    company: String(formData.get('company') ??
      '').trim(),
    role: String(formData.get('role') ?? '').trim(),
    salary: String(formData.get('salary') ??
      '').trim(),
  }

  await sleep(1000) // latencia de red

  const errors: FormState['errors'] = {}
  if (!values.company) errors.company = 'Falta la
    empresa.'
  if (!values.role) errors.role = 'Falta el puesto.'
  if (Number(values.salary) < 1000) errors.salary =
    'El salario tiene que ser de al menos USD 1000.'

  if (Object.keys(errors).length > 0) {
    return { ...prevState, status: 'error', message:
      'Revisá los campos marcados.', errors, values
      }
  }
```

```
  const application: Application = { id: Date.now(),
    ...values, salary: Number(values.salary) }
  return {
    status: 'success',
    message: `¡Postulación a ${values.company}
      guardada!`,
    errors: {},
    values: initialFormState.values,
    saved: [...prevState.saved, application], //
      usamos el estado anterior, como un reducer
  }
}
```

> **🧪 Probá esto**
>
> 1. Tocá **Guardar** con todo vacío: aparecen los tres errores.
> 2. Completá empresa y puesto, poné 500 de salario y guardá: el error de salario aparece y **lo que escribiste se mantiene** (gracias a `values` y `defaultValue`).
> 3. Corregí el salario y guardá: el formulario se vacía solo y la postulación aparece en la lista.

Lo que hay que entender de esta demo:

- `<form action={formAction}>`: React llama a tu función con el `FormData` del formulario. No hay

`onSubmit`, ni `preventDefault`, ni estado por input.

- **Los inputs no son controlados**: usan `name` y `defaultValue`. Los valores se leen del `FormData` en la Action.
- **El reset automático**: cuando una Action de formulario termina, React resetea el form. Por eso, si hay errores, la función devuelve los `values` y el form los usa como `defaultValue` para no perder lo que escribió el usuario.
- `saved` **se construye con el estado anterior** (`prevState.saved`), igual que en un reducer.
- `createApplication` **es una función async común.** En Next.js le agregarías `"use server"` arriba y correría en el servidor sin cambiar nada más. Lo vemos en Server Functions.

× React 18

```
function NewApplication() {
  const [company, setCompany] = useState('')
  const [errors, setErrors] = useState({})
  const [isPending, setIsPending] = useState(false)

  async function handleSubmit(e) {
    e.preventDefault()
    setIsPending(true)
    try {
      const result = await create({ company })
      setErrors(result.errors ?? {})
      if (!result.errors) setCompany('')
    } finally {
```

```
      setIsPending(false)
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={company} onChange={(e) =>
        setCompany(e.target.value)} />
      {/* ... */}
    </form>
  )
}
```

✓ React 19 con useActionState

```
function NewApplication() {
  const [state, formAction, isPending] =
    useActionState(
    createApplication,
    initialFormState,
  )

  return (
    <form action={formAction}>
      <input name="company" defaultValue=
        {state.values.company} />
      {state.errors.company && <p>
        {state.errors.company}</p>}
      <button disabled={isPending}>Guardar</button>
    </form>
  )
}
```

Fuera de un formulario: la cola de Actions

`dispatchAction` también se puede llamar con cualquier payload, por ejemplo desde un botón. La regla: tiene que ejecutarse **dentro de una transición**, así que lo envolvés en `startTransition`.

DEMO **Llamadas encoladas en orden**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/InterviewCounter.tsx`

```tsx
import { startTransition, useActionState } from
  'react'
import { updateInterviews } from './api'

type Payload = { type: 'add' } | { type: 'remove' }

// reducerAction: como un reducer, pero puede ser
  async y tener efectos secundarios
async function interviewsAction(current: number,
  payload: Payload) {
  return updateInterviews(current, payload.type ===
    'add' ? 1 : -1)
}

export default function InterviewCounter() {
  const [count, dispatch, isPending] =
    useActionState(interviewsAction, 0)
```

```
  function send(payload: Payload) {
    // dispatch tiene que llamarse dentro de una
      Action (transición)
    startTransition(() => dispatch(payload))
  }

  return (
    <div className="row">
      <button onClick={() => send({ type: 'remove'
        })}>-</button>
      <span style={{ fontSize: '1.6rem', fontWeight:
        700, minWidth: 40, textAlign: 'center' }}>
        {count}</span>
      <button onClick={() => send({ type: 'add'
        })}>+</button>
      <span className={isPending ? 'muted' : 'ok-
        text'}>
        {isPending ? '⏳ Procesando la cola…' :
          'Entrevistas agendadas'}
      </span>
    </div>
  )
}
```

src/demos/react19/api.ts

```
import { sleep } from '../shared/utils'

// "Servidor" falso para las demos de React 19. Cada
  función tarda como una API real.

export async function updateName(name: string):
  Promise<string | null> {
  await sleep(1000)
  if (name.trim().length < 3) return 'El nombre
    tiene que tener al menos 3 letras.'
  return null // null = sin error
}
```

```
export async function saveOffer(company: string) {
  await sleep(900)
  return `Oferta de ${company} guardada a las ${new
    Date().toLocaleTimeString('es')}`
}

export async function updateInterviews(current:
  number, delta: number) {
  await sleep(600)
  return Math.max(0, current + delta)
}
```

🧪 Probá esto

Tocá **+** cinco veces muy rápido. Cada llamada a la "API" tarda 600 ms y React las **encola**: se ejecutan una por una, en orden, y cada una recibe el resultado de la anterior como `current`. Mientras la cola avanza, `isPending` queda en `true` y el número no se mueve: React agrupa las Actions en curso y muestra el resultado final (5) cuando termina toda la cola, unos 3 segundos después. Ningún click se pierde ni se desordena.

Esa garantía de orden es la gran diferencia con hacer el fetch a mano: con `useState` + `fetch`, cinco clicks rápidos pueden terminar en cualquier orden. La contracara es la espera: el número recién

cambia al final. Para que suba al instante con cada click, se combina con useOptimistic.

useActionState vs useReducer

	`useReducer`	`useActionState`
La función puede ser async	No	Sí
Puede tener efectos secundarios	No (tiene que ser pura)	Sí (llamar a la API es la idea)
Te da `isPending`	No	Sí
Se conecta directo a `<form action>`	No	Sí
Funciona con Server Functions	No	Sí

⚠️ Errores comunes

- **Llamar a `dispatchAction` fuera de una transición**: en desarrollo vas a ver un error. Usá `<form action>`, `startTransition` o una prop `action`.
- **Esperar que el form no se resetee**: si necesitás conservar los valores ante un error, devolvelos en el estado y usá `defaultValue`.
- **Que el tipo del estado devuelto no coincida con `initialState`**: TypeScript te va a marcar el error; tipá el estado explícitamente (como `FormState` en la demo).

ℹ️ El nombre viejo

Durante la beta de React 19 este hook se llamaba `useFormState` y vivía en `react-dom`. Si lo ves en algún tutorial, es lo mismo: hoy se usa `useActionState` de `react`.

🎤 En una entrevista

"¿Para qué sirve useActionState?" → "Para guardar el resultado de una Action, como los errores de validación o un mensaje de éxito, junto con su estado pendiente. Funciona como un reducer que puede ser async: recibe el estado anterior y el payload, que en un formulario es el FormData. Se conecta directo a form action y encola las ejecuciones en orden."

Capítulo 17

Formularios con action y useFormStatus

React 19.0

<form action={fn}>, FormData, reset automático y botones que saben si se está enviando.

En React 19 los formularios volvieron a ser simples. Le pasás una **función** a la prop `action` del `<form>` y React se encarga del resto.

```
async function subscribe(formData: FormData) {
  const email = formData.get('email')
  await api.subscribe(email)
}

<form action={subscribe}>
  <input name="email" type="email" />
  <button type="submit">Suscribirme</button>
</form>
```

Cuando el usuario envía el formulario, React:

1. Evita la recarga de la página (no hace falta `preventDefault`).
2. Arma un `FormData` con todos los campos que tienen `name`.
3. Ejecuta tu función como una **Action** (dentro de una transición).
4. Cuando termina sin errores, **resetea el formulario** (los campos no controlados).

useFormStatus: un botón que sabe si el form se está enviando

`useFormStatus()` (de `react-dom`) devuelve el estado del `<form>` **padre** de un componente: `pending`, `data` (el FormData que se está enviando), `method` y `action`. Es ideal para botones reutilizables: no hace falta pasarles `isPending` por props.

DEMO **Form action + SubmitButton con useFormStatus**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/SubscribeForm.tsx`

```
import { startTransition, useState } from 'react'
```

```tsx
import { useFormStatus } from 'react-dom'
import { sleep } from '../shared/utils'

// Un botón reutilizable que "sabe" si su <form> se
  está enviando.
// Tiene que renderizarse DENTRO del form:
  useFormStatus lee el form padre.
function SubmitButton() {
  const { pending, data } = useFormStatus()
  return (
    <button type="submit" disabled={pending}>
      {pending ? `Suscribiendo
        ${data?.get('email')}…` : 'Suscribirme'}
    </button>
  )
}

export default function SubscribeForm() {
  const [subscribed, setSubscribed] =
    useState<string[]>([])

  // La action recibe el FormData del formulario: no
    hace falta estado por input
  async function subscribe(formData: FormData) {
    const email = String(formData.get('email'))
    await sleep(1200) // "llamada a la API"
    startTransition(() => setSubscribed((list) =>
      [...list, email]))
    // Al terminar con éxito, React resetea el form
      solo (los inputs no controlados)
  }

  return (
    <div className="stack">
      <form action={subscribe} className="row">
        <input name="email" type="email" required
          placeholder="tu@email.com" />
        <SubmitButton />
      </form>
```

```
      {subscribed.length > 0 && (
        <p className="ok-text">Alertas de empleo
          activadas para: {subscribed.join(', ')}
          </p>
      )}
    </div>
  )
}
```

> ⚠️ **useFormStatus lee el form PADRE**
>
> `useFormStatus` tiene que llamarse en un componente que se renderiza **adentro** del `<form>`. Si lo llamás en el mismo componente que renderiza el form, siempre te va a devolver `pending: false`. Por eso `SubmitButton` es un componente aparte.

Varios botones, distintas acciones

Un `<button>` (o un `<input type="submit">`) puede tener su propia `formAction` que pisa la `action` del form. Útil para "Guardar borrador" vs "Publicar", o "Aprobar" vs "Rechazar".

DEMO **formAction por botón**

▶ Demo interactiva: corré `npm run dev` y abrí

este capítulo para probarla.

src/demos/react19/DraftOrPublish.tsx

```tsx
import { startTransition, useState } from 'react'
import { useFormStatus } from 'react-dom'
import { sleep } from '../shared/utils'

function FormStatusLine() {
  const { pending } = useFormStatus()
  return <span className="muted">{pending ? '⏳
    Enviando…' : ''}</span>
}

export default function DraftOrPublish() {
  const [log, setLog] = useState<string[]>([])

  async function saveDraft(formData: FormData) {
    await sleep(700)
    startTransition(() => setLog((l) => [...l, `📝
      Borrador guardado:
      "${formData.get('note')}"`]))
  }

  async function publish(formData: FormData) {
    await sleep(700)
    startTransition(() => setLog((l) => [...l, `🚀
      Publicado: "${formData.get('note')}"`]))
  }

  return (
    <div className="stack">
      {/* formAction en cada botón pisa el action
        del form */}
      <form action={publish} className="stack">
        <textarea name="note" rows={2} required
          defaultValue="Busco rol Frontend remoto"
          />
```

```
        <div className="row">
          <button formAction={saveDraft}>Guardar
            borrador</button>
          <button type="submit" className="primary">
            Publicar
          </button>
          <FormStatusLine />
        </div>
      </form>
      <ul className="log">
        {log.map((line, i) => (
          <li key={i}>{line}</li>
        ))}
      </ul>
    </div>
  )
}
```

Otra opción es usar un solo `action` y darle a cada botón `name="intent"` con un `value` distinto: el botón que se usó para enviar viaja dentro del `FormData`.

Controlados o no controlados

Con `<form action>`, los inputs **no controlados** (`name` + `defaultValue`) son la opción más simple: no hay estado por campo y los valores llegan en el `FormData`. Usá inputs **controlados** cuando necesitás reaccionar mientras el usuario escribe: validación en vivo, formatear un número, habilitar un botón según lo escrito.

> 💡 **Resetear a mano**
>
> Si necesitás resetear un form desde otro lugar, `requestFormReset(formElement)` de `react-dom` lo hace respetando el momento correcto dentro de la transición. Y desde React 19.3 el reset automático también dispara el evento `onReset` del form.

Lo que no cambió

- La validación nativa del navegador (`required`, `type="email"`, `minLength`) sigue funcionando y corre **antes** de tu Action.
- `onSubmit` sigue existiendo y es válido. Usá `action` cuando quieras aprovechar las Actions (pending, reset, Server Functions); usá `onSubmit` si necesitás algo muy específico del evento.
- Para formularios grandes con validación compleja, librerías como **React Hook Form** con **Zod** siguen siendo muy usadas y se pueden combinar con Actions.

ℹ️ Funciona sin JavaScript (en Next.js)

Si la `action` es una **Server Function**, el formulario funciona incluso antes de que cargue el JavaScript de la página: el navegador hace un POST normal. Eso se llama **progressive enhancement** y es una de las razones por las que este modelo existe. Lo vemos en Server Functions.

🎤 En una entrevista

"¿Cómo manejás formularios en React 19?" → "Le paso una función a la prop action del form, que recibe el FormData y corre como una Action: React maneja el pending, evita el reload y resetea el form al terminar. Para mostrar el estado de envío uso useFormStatus en el botón, y para errores de validación o mensajes uso useActionState. Con Next.js esa misma función puede ser una Server Function."

Capítulo 18

useOptimistic

React 19.0

Mostrar el resultado al instante mientras la Action termina en segundo plano.

Cuando le das "me gusta" a algo en una red social, el corazón se pinta **al instante**, aunque el servidor todavía no confirmó nada. Eso es una **actualización optimista**: asumís que la operación va a salir bien y mostrás el resultado ya. Si falla, volvés atrás.

Hacerlo a mano era delicado (guardar el valor anterior, revertir en el `catch`, cuidar los casos de carrera). `useOptimistic` lo resuelve:

```
const [optimisticValue, setOptimistic] =
  useOptimistic(value, reducer?)
```

- `value`: el valor **real** (de tu estado o de las props).

- `optimisticValue`: igual a `value`, **salvo mientras haya una Action en curso**, en cuyo caso refleja lo que hayas seteado con `setOptimistic`.
- Cuando la Action termina, el valor optimista **se descarta solo** y se vuelve a mostrar `value`. Si la Action actualizó `value`, ves el dato real; si falló, ves el valor de antes.

Mensajes que aparecen al instante

DEMO **Lista optimista con manejo de errores**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/OptimisticNotes.tsx`

```
import { startTransition, useOptimistic, useState }
  from 'react'
import { sleep } from '../shared/utils'

type Note = { id: string; text: string; sending?:
  boolean }

async function sendNote(text: string, shouldFail:
  boolean): Promise<Note> {
  await sleep(1500)
  if (shouldFail) throw new Error(`No se pudo enviar
    "${text}". Probá de nuevo.`)
  return { id: crypto.randomUUID(), text }
}
```

```
export default function OptimisticNotes() {
  const [notes, setNotes] = useState<Note[]>([{ id:
    '1', text: 'Hola, vi la oferta de Frontend 👋'
    }])
  const [error, setError] = useState<string | null>
    (null)
  const [shouldFail, setShouldFail] =
    useState(false)

  // optimisticNotes = notes + lo que agregamos
    mientras la Action está en curso
  const [optimisticNotes, addOptimisticNote] =
    useOptimistic(notes, (current: Note[], newNote:
    Note) => [
    ...current,
    newNote,
  ])

  async function sendAction(formData: FormData) {
    const text = String(formData.get('text') ??
      '').trim()
    if (!text) return
    setError(null)
    addOptimisticNote({ id: crypto.randomUUID(),
      text, sending: true }) // aparece YA
    try {
      const saved = await sendNote(text, shouldFail)
      startTransition(() => setNotes((list) =>
        [...list, saved])) // el dato real
    } catch (e) {
      startTransition(() => setError((e as
        Error).message))
    }
    // Cuando la Action termina, el estado optimista
      se descarta solo.
  }

  return (
    <div className="stack">
```

```
      <ul className="stack" style={{ listStyle:
        'none', padding: 0, margin: 0 }}>
        {optimisticNotes.map((note) => (
          <li key={note.id} className={`card
            ${note.sending ? 'pending' : ''}`}>
            {note.text} {note.sending && <small
              className="muted">· enviando…</small>}
          </li>
        ))}
      </ul>
      <form action={sendAction} className="row">
        <input name="text" placeholder="Escribile al
          recruiter…" autoComplete="off" />
        <button type="submit">Enviar</button>
      </form>
      <label className="row">
        <input type="checkbox" checked={shouldFail}
          onChange={(e) =>
          setShouldFail(e.target.checked)} />
        Simular error del servidor
      </label>
      {error && <p className="error-text">{error}
        </p>}
    </div>
  )
}
```

🧪 **Probá esto**

1. Enviá un mensaje: aparece al instante, semitransparente y con "enviando...". Al segundo y medio queda confirmado.
2. Activá **Simular error del servidor** y enviá otro: aparece, y cuando la API falla desaparece solo y se muestra el error. No escribiste ningún código para "deshacer".
3. Enviá tres mensajes seguidos rápido: los tres aparecen enseguida.

Este hook usa la forma con **reducer**: `useOptimistic(notes, (current, newNote) => [...current, newNote])`. El segundo argumento de `addOptimisticNote` se pasa a ese reducer. Es la forma ideal para listas, donde querés "agregar" sobre el estado actual.

La forma simple: un valor

Para un valor suelto (un booleano, un contador, un texto) no hace falta reducer:

DEMO

Guardar una oferta con feedback instantáneo

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/react19/OptimisticLike.tsx

```tsx
import { startTransition, useOptimistic, useState }
  from 'react'
import { sleep } from '../shared/utils'

export default function OptimisticLike() {
  const [saved, setSaved] = useState(false)
  const [optimisticSaved, setOptimisticSaved] =
    useOptimistic(saved)

  function toggle() {
    startTransition(async () => {
      setOptimisticSaved(!saved) // la UI cambia al
        instante
      await sleep(1000) // la API tarda
      startTransition(() => setSaved(!saved))
    })
  }

  return (
    <div className="row">
      <button onClick={toggle} className=
        {optimisticSaved ? 'active' : ''}>
        {optimisticSaved ? '★ Guardada' : '☆ Guardar
          oferta'}
      </button>
      <span className="muted">
        Valor real: {saved ? 'guardada' : 'no
          guardada'}
```

```
        {optimisticSaved !== saved && '
          (sincronizando…)'}
      </span>
    </div>
  )
}
```

Reglas

- `setOptimistic` **se llama dentro de una Action**: en un `<form action>`, en `startTransition` o en una prop `action`. Fuera de una Action vas a ver una advertencia y el valor optimista se va a ver solo por un instante.
- **No reemplaza al estado real**: siempre necesitás actualizar el `value` de verdad (con `setState`, revalidando datos, etc.). El optimista es solo la "vista previa".
- **Tiene que ser rápido y puro**: el reducer se puede ejecutar varias veces; no hagas efectos secundarios adentro.

💡 ¿Cuándo vale la pena?

En acciones frecuentes, de bajo riesgo y que casi siempre salen bien: likes, marcar como leído, reordenar, agregar a favoritos, enviar mensajes. Para cosas críticas (pagos, borrar algo importante) mejor mostrá un estado pendiente claro y esperá la confirmación.

× Optimista a mano

```
async function toggle() {
  const previous = saved
  setSaved(!saved) // optimista
  try {
    await api.toggleSaved(jobId)
  } catch {
    setSaved(previous) // revertir a mano
  }
}
```

✓ useOptimistic

```
const [optimisticSaved, setOptimisticSaved] =
  useOptimistic(saved)

function toggle() {
  startTransition(async () => {
    setOptimisticSaved(!saved)
    await api.toggleSaved(jobId)
    startTransition(() => setSaved(!saved))
  })
```

```
}
```

🎤 En una entrevista

"¿Qué hace useOptimistic?" → "Muestra un valor provisional mientras una Action está en curso, por ejemplo un mensaje que aparece al instante antes de que el servidor confirme. Cuando la Action termina, React descarta el valor optimista y muestra el real, así que si hubo un error vuelve solo al estado anterior, sin código para revertir."

Capítulo 19

use(): promesas y contexto

React 19.0

Leer una promesa con Suspense y leer contexto incluso dentro de un if.

`use` es una API nueva de React 19 que **lee el valor de un recurso** durante el render. Hoy acepta dos tipos de recursos: **promesas** y **contexto**.

```
const job = use(jobPromise) // espera la promesa
  (con Suspense)
const theme = use(ThemeContext) // lee un contexto
```

No se llama `useAlgo` porque no es un hook común: **se puede llamar dentro de un `if` o de un loop**. Lo que sí mantiene es que solo se usa durante el render de un componente o de un hook.

Leer una promesa

Cuando le pasás una promesa a `use`:

- Si todavía no se resolvió, el componente **suspende** y se muestra el fallback del `<Suspense>` más cercano.
- Cuando se resuelve, React vuelve a renderizar y `use` devuelve el valor.
- Si se rechaza, el error va al **Error Boundary** más cercano.

DEMO **use(promise) con Suspense, Error Boundary y transiciones**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/UsePromise.tsx`

```
import { Suspense, use, useState, useTransition }
  from 'react'
import { ErrorBoundary } from
  '../shared/ErrorBoundary'
import { clearJobsCache, fetchJob, type Job } from
  './jobs-api'

function JobDetails({ jobPromise }: { jobPromise:
  Promise<Job> }) {
  // use() "desenvuelve" la promesa: si no se
    resolvió, el componente suspende
  const job = use(jobPromise)
```

```
  return (
    <div className="card">
      <strong>
        {job.title} · {job.company}
      </strong>
      <p>Salario: {job.salary}</p>
      <p className="muted">Stack: {job.stack.join(',
        ')}</p>
    </div>
  )
}

export default function UsePromise() {
  const [selectedId, setSelectedId] = useState(1)
  // Guardamos la PROMESA en el estado (se crea en
    un handler, no en el render)
  const [jobPromise, setJobPromise] = useState(() =>
    fetchJob(1))
  const [withTransition, setWithTransition] =
    useState(false)
  const [isPending, startTransition] =
    useTransition()

  function select(id: number) {
    if (withTransition) {
      // Con transición: React mantiene visible el
        contenido anterior hasta que llega el nuevo
      startTransition(() => {
        setSelectedId(id)
        setJobPromise(fetchJob(id))
      })
    } else {
      // Sin transición: vuelve a mostrarse el
        fallback de Suspense
      setSelectedId(id)
      setJobPromise(fetchJob(id))
    }
  }
```

```
  return (
    <div className="stack">
      <div className="row">
        {[1, 2, 3, 4].map((id) => (
          <button key={id} className={id ===
            selectedId ? 'active' : ''} onClick={()
            => select(id)}>
            Oferta {id}
            {id === 4 ? ' (rota)' : ''}
          </button>
        ))}
        <button onClick={clearJobsCache}>Vaciar
          caché</button>
      </div>
      <label className="row">
        <input type="checkbox" checked=
          {withTransition} onChange={(e) =>
          setWithTransition(e.target.checked)} />
        Cambiar dentro de startTransition
      </label>

      <div className={isPending ? 'pending' : ''}>
        {/* Suspense va AFUERA del boundary con key:
          así no se crea uno nuevo en cada cambio
          */}
        <Suspense fallback={<p className="muted">⏳
          Cargando oferta…</p>}>
          {/* key: al elegir otra oferta, el Error
            Boundary se reinicia si había un error
            */}
          <ErrorBoundary key={selectedId} fallback=
            {(error) => <p className="error-text">❌
            {error.message}</p>}>
            <JobDetails jobPromise={jobPromise} />
          </ErrorBoundary>
        </Suspense>
      </div>
    </div>
  )
```

```
}
```

src/demos/react19/jobs-api.ts

```ts
import { sleep } from '../shared/utils'

export type Job = { id: number; title: string;
  company: string; salary: string; stack: string[] }

const jobs: Job[] = [
  { id: 1, title: 'Frontend Engineer', company:
    'Nubelia', salary: 'USD 3.200', stack: ['React',
    'TypeScript', 'Next.js'] },
  { id: 2, title: 'React Developer', company: 'Pagos
    Sur', salary: 'USD 2.800', stack: ['React',
    'Redux', 'Jest'] },
  { id: 3, title: 'UI Engineer', company: 'Datalab',
    salary: 'USD 4.100', stack: ['React',
    'Tailwind', 'Storybook'] },
  { id: 4, title: 'Senior Frontend', company:
    'Kuarahy', salary: '—', stack: [] },
]

// Caché de promesas: use() necesita recibir LA
  MISMA promesa en cada render.
const cache = new Map<number, Promise<Job>>()

export function fetchJob(id: number): Promise<Job> {
  if (!cache.has(id)) {
    cache.set(
      id,
      sleep(1200).then(() => {
        if (id === 4) throw new Error('Esta oferta
          ya no existe (404).')
        return jobs.find((job) => job.id === id)!
      }),
    )
  }
```

```
  return cache.get(id)!
}

export function clearJobsCache() {
  cache.clear()
}
```

🧪 Probá esto

1. Tocá **Oferta 2** y **Oferta 3**: ves el fallback "Cargando oferta..." durante 1,2 segundos. Volvé a la 2: aparece al instante, porque la promesa ya está en la caché.
2. Tocá **Oferta 4 (rota)**: la promesa se rechaza y el Error Boundary muestra el error.
3. Tocá **Vaciar caché**, activá **Cambiar dentro de startTransition** y cambiá de oferta: en vez del fallback, la oferta anterior queda visible (atenuada) hasta que llega la nueva. Así se evita el parpadeo. Fijate que el botón también se marca recién al final: `setSelectedId` está dentro de la misma transición, y React muestra todo junto cuando está listo.

Un detalle del código: el `<Suspense>` está **afuera** del `ErrorBoundary` que tiene `key`. Si estuviera adentro, cada cambio de oferta crearía un `<Suspense>` nuevo, y un Suspense recién montado

siempre muestra su fallback, aunque estés en una transición. La transición solo evita **volver a ocultar** contenido que ya estaba en pantalla.

La regla de oro: la promesa tiene que ser estable

`use` necesita recibir **la misma promesa** en cada render. Si creás la promesa durante el render, cada render crea una nueva, el componente vuelve a suspender y nunca sale del fallback.

```
function JobDetails({ id }: { id: number }) {
  const job = use(fetch(`/api/jobs/${id}`).then((r)
    => r.json())) // 🔴 promesa nueva en cada render
}
```

Las promesas tienen que venir de un lugar estable:

- **Creadas en un event handler** y guardadas en el estado (como en la demo).
- De una **caché** (como `fetchJob` en `jobs-api.ts`, que guarda una promesa por id).
- De un **Server Component** que la pasa como prop a un Client Component (el caso ideal en Next.js, lo vemos en la Parte 5).
- De un **framework o librería** compatible con Suspense (React Router, TanStack Query, Relay).

> **⚠️ use no es el nuevo fetch en un Effect**
>
> `use` no pide datos: **lee** una promesa que alguien más creó. En una SPA con Vite, armar tu propia caché de promesas es delicado (invalidación, reintentos, refetch). Para producción usá TanStack Query (`useSuspenseQuery`) o un framework. Donde `use` brilla es con Server Components.

Leer contexto, incluso de forma condicional

`use(Context)` hace lo mismo que `useContext(Context)`, con una diferencia: como no es un hook común, lo podés llamar **después de un return temprano o dentro de un if**.

DEMO **use(Context) después de un return temprano**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/UseContextConditional.tsx`

```
import { createContext, use, useState } from 'react'
```

```tsx
const CurrencyContext = createContext<'USD' | 'PYG'>
  ('USD')

function Salary({ amountUsd, visible }: { amountUsd:
  number; visible: boolean }) {
  if (!visible) return <span
    className="muted">Salario oculto</span>

  // ✅ use() se puede llamar después de un return
    temprano o dentro de un if.
  // Con useContext esto rompería las reglas de los
    hooks.
  const currency = use(CurrencyContext)
  const amount = currency === 'USD' ? amountUsd :
    amountUsd * 7300
  return (
    <strong>
      {currency} {amount.toLocaleString('es')}
    </strong>
  )
}

export default function UseContextConditional() {
  const [currency, setCurrency] = useState<'USD' |
    'PYG'>('USD')
  const [visible, setVisible] = useState(true)

  return (
    <CurrencyContext value={currency}>
      <div className="stack">
        <div className="row">
          <button onClick={() =>
            setCurrency(currency === 'USD' ? 'PYG' :
            'USD')}>Moneda: {currency}</button>
          <label className="row">
            <input type="checkbox" checked={visible}
              onChange={(e) =>
              setVisible(e.target.checked)} />
            Mostrar salario
```

```
          </label>
        </div>
        <p>
          Frontend Engineer en Nubelia · <Salary
            amountUsd={3200} visible={visible} />
        </p>
      </div>
    </CurrencyContext>
  )
}
```

Con `useContext` en esa misma posición, el linter te marcaría un error: los hooks tienen que llamarse siempre, en el mismo orden, en cada render.

> ### ℹ️ ¿Y useContext? ¿Queda obsoleto?
>
> No. `useContext` sigue funcionando y no se va a deprecar. `use(Context)` es más flexible, así que en código nuevo podés usar cualquiera de los dos; muchos equipos usan `use` solo cuando necesitan llamarlo de forma condicional.

⚠️ Lo que use no puede hacer

- No se puede llamar dentro de un `try/catch`: para los errores, usá un Error Boundary.
- No se usa en event handlers ni en Effects: solo durante el render.
- Desde React 19.3, en desarrollo vas a ver una advertencia si un componente parece haberse "desbloqueado" llamando a `use()` de forma condicional según el estado de la promesa.

🎤 En una entrevista

"¿Qué es use() en React 19?" → "Una API para leer recursos durante el render: promesas y contexto. Con una promesa, el componente suspende hasta que se resuelve, se integra con Suspense y los errores van al Error Boundary. La promesa tiene que ser estable, por ejemplo creada en un Server Component o en una caché. Y a diferencia de los hooks, se puede llamar dentro de condicionales."

Capítulo 20

ref como prop y <Context> como provider

React 19.0

Adiós forwardRef y .Provider, y los refs con función de limpieza.

React 19 trae varios cambios chicos que, sumados, eliminan mucho código ceremonial. Si venís de un proyecto legacy, estos son los que más vas a notar al leer código nuevo.

ref como prop: chau forwardRef

Hasta React 18, un componente de función **no podía recibir** un `ref`: había que envolverlo en `forwardRef`. En React 19, `ref` es una prop más.

× React 18

```
const TextField = forwardRef<HTMLInputElement,
  Props>(
  function TextField({ label, ...props }, ref) {
    return (
      <label>
        {label}
        <input ref={ref} {...props} />
      </label>
    )
  },
)
```

✓ **React 19**

```
function TextField({ label, ref, ...props }: Props)
  {
  return (
    <label>
      {label}
      <input ref={ref} {...props} />
    </label>
  )
}
```

DEMO **Enfocar el primer campo vacío con ref como prop**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/react19/RefAsProp.tsx

```tsx
import { useRef, useState, type ComponentProps }
  from 'react'

// React 19: `ref` llega como cualquier otra prop.
  Sin forwardRef.
// ComponentProps<'input'> ya incluye `ref` en sus
  tipos.
function TextField({ label, ref, ...inputProps }:
  ComponentProps<'input'> & { label: string }) {
  return (
    <label className="stack">
      {label}
      <input ref={ref} {...inputProps} />
    </label>
  )
}

export default function RefAsProp() {
  const companyRef = useRef<HTMLInputElement>(null)
  const roleRef = useRef<HTMLInputElement>(null)
  const [message, setMessage] = useState('')

  function validate() {
    // Enfocamos el primer campo vacío
    const empty = [companyRef.current,
      roleRef.current].find((input) =>
      !input?.value)
    if (empty) {
      empty.focus()
      setMessage(`Completá el campo
        "${empty.name}"`)
    } else {
      setMessage('Todo completo ✅')
    }
  }
```

```
  return (
    <div className="stack">
      <div className="grid-2">
        <TextField label="Empresa" name="empresa"
          ref={companyRef} />
        <TextField label="Puesto" name="puesto" ref=
          {roleRef} />
      </div>
      <div className="row">
        <button onClick={validate}>Validar</button>
        <span className="muted">{message}</span>
      </div>
    </div>
  )
}
```

`forwardRef` sigue funcionando, pero se va a deprecar en una versión futura. Hay un codemod que lo migra automáticamente.

Ref callbacks con limpieza

Un **ref callback** es una función que pasás como `ref`: React la llama con el nodo cuando se monta. En React 19 esa función puede **devolver un cleanup**, igual que un Effect. Ideal para conectar observers, librerías o event listeners a un nodo puntual.

DEMO **ResizeObserver conectado con un ref callback**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/RefCleanup.tsx`

```tsx
import { useCallback, useState } from 'react'

export default function RefCleanup() {
  const [visible, setVisible] = useState(true)
  const [width, setWidth] = useState(0)
  const [log, setLog] = useState<string[]>([])

  // React 19: un ref callback puede devolver una función de limpieza.
  // (useCallback para que React no lo vuelva a llamar en cada render;
  // con el React Compiler ni siquiera haría falta)
  const measureRef = useCallback((node: HTMLTextAreaElement) => {
    const observer = new ResizeObserver(([entry]) =>
      setWidth(Math.round(entry.contentRect.width)))
    observer.observe(node)
    setLog((l) => [...l, '▶ ref: ResizeObserver conectado'])

    return () => {
      observer.disconnect()
      setLog((l) => [...l, '■ cleanup: ResizeObserver desconectado'])
    }
  }, [])

  return (
```

```
    <div className="stack">
      <button onClick={() => setVisible((v) => !v)}>
        {visible ? 'Desmontar' : 'Montar'} el
        textarea</button>
      {visible && (
        <>
          <textarea ref={measureRef} rows={2} style=
            {{ resize: 'horizontal', width: 260 }}
            defaultValue="Estirame desde la esquina
            ↘" />
          <p className="muted">Ancho medido:
            {width}px</p>
        </>
      )}
      <ol className="log">
        {log.map((line, i) => (
          <li key={i}>{line}</li>
        ))}
      </ol>
    </div>
  )
}
```

🧪 Probá esto

Estirá el textarea desde la esquina y mirá el ancho medido. Después tocá **Desmontar**: el log muestra que el cleanup desconectó el observer. Al montar ves "conectado, desconectado, conectado": como con los Effects, en desarrollo StrictMode ejecuta setup, cleanup y setup para verificar que la limpieza funciona.

> **⚠️ TypeScript: cuidado con el return implícito**
>
> Como ahora un ref callback puede devolver una función, TypeScript rechaza los que devuelven otra cosa:

```
<div ref={(node) => (instance = node)} />    //
  🔴 devuelve el nodo
<div ref={(node) => { instance = node }} />  //
  ✅ con llaves no devuelve nada
```

Context como provider

Ya lo usamos en el capítulo de useContext: en vez de `<ThemeContext.Provider value={...}>` escribís `<ThemeContext value={...}>`.

Otros cambios que vas a cruzarte

Cambio	Detalle
`useDeferredValue(value, initialValue)`	Nuevo segundo argumento: el valor que se usa en el primer render.
`useRef` requiere argumento	En los tipos: `useRef<T>(null)` en vez de `useRef<T>()`.
`element.ref` deprecado	Para leer el ref de un elemento se usa `element.props.ref`.

Cambio	Detalle
`ReactElement` tipado con `unknown`	Las props de un `ReactElement` sin tipo ahora son `unknown` en vez de `any`.
JSX global	El namespace `JSX` global se reemplaza por `React.JSX`.
Custom elements	Soporte completo de Web Components: las props se asignan como propiedades.

🎤 En una entrevista

"¿Qué pasó con forwardRef?" → "En React 19 ref es una prop común en componentes de función, así que ya no hace falta forwardRef; se va a deprecar más adelante. Además los ref callbacks pueden devolver una función de limpieza, y los contextos se proveen directamente con Context en vez de Context.Provider."

Capítulo 21

Metadatos, estilos y recursos

React 19.0

<title> y <meta> desde cualquier componente, hojas de estilo y precarga de recursos.

Antes, para cambiar el `<title>` de la página según el componente había que usar `document.title` en un Effect o una librería como `react-helmet`. En React 19 podés renderizar `<title>`, `<meta>` y `<link>` **en cualquier componente**, y React los mueve al `<head>` del documento.

```
function JobPage({ job }: { job: Job }) {
  return (
    <article>
      <title>{`${job.title} en ${job.company}`}
        </title>
      <meta name="description" content={job.summary}
        />
      <link rel="canonical" href=
        {`https://ejemplo.com/jobs/${job.id}`} />
```

```
      <h1>{job.title}</h1>
    </article>
  )
}
```

Funciona en el cliente y también en el servidor (SSR y streaming), que es donde más importa para SEO.

DEMO **El título de la pestaña desde un componente**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/react19/DocumentMetadata.tsx`

```
import { useState } from 'react'

export default function DocumentMetadata() {
  const [unread, setUnread] = useState(3)
  const title = `${unread > 0 ? `(${unread}) ` :
    ''}Mensajes · Job Tracker`

  return (
    <div className="stack">
      {/* React 19 sube estas etiquetas al <head>
        del documento */}
      <title>{title}</title>
      <meta name="description" content={`Tenés
        ${unread} mensajes sin leer de recruiters`}
        />

      <p>
```

```
        Mirá el título de la pestaña del navegador:
          <strong>{title}</strong>
      </p>
      <div className="row">
        <button onClick={() => setUnread((n) => n +
          1)}>Llega un mensaje</button>
        <button onClick={() => setUnread(0)}>Marcar
          todo como leído</button>
      </div>
    </div>
  )
}
```

🧪 Probá esto

Tocá **Llega un mensaje** y mirá la pestaña del navegador: el contador de no leídos aparece en el título, como en Gmail. Toda esta guía usa la misma técnica: cada capítulo renderiza su `<title>` (mirá `src/components/DocPage.tsx`).

> **⚠️ Un solo title a la vez**
>
> Si dos componentes renderizan un `<title>` al mismo tiempo, React mete los dos en el `<head>` y el resultado depende del navegador. Por eso, en este capítulo la guía **no** renderiza su propio título y deja que la demo lo controle (fijate en `ownsTitle` dentro de `src/chapters.ts`). En Next.js usás la API de `metadata` del framework, que resuelve esto por vos.

Hojas de estilo con precedencia

Podés renderizar `<link rel="stylesheet">` dentro de un componente y darle una `precedence`. React:

- Inserta la hoja en el `<head>` respetando el orden de precedencia.
- **No la duplica** si varios componentes la renderizan.
- Espera a que cargue antes de mostrar el contenido que la usa (con Suspense), para evitar el "flash" sin estilos.

```
function Chart() {
  return (
    <>
```

```
      <link rel="stylesheet" href="/chart.css"
        precedence="default" />
      <link rel="stylesheet" href="/chart-dark.css"
        precedence="high" />
      <div className="chart">…</div>
    </>
  )
}
```

Lo mismo pasa con los scripts async: `<script async src="https://widget.ejemplo.com/embed.js" />` se carga una sola vez aunque lo rendericen varios componentes.

Precargar recursos

`react-dom` exporta funciones para avisarle al navegador qué va a necesitar pronto, desde cualquier componente o event handler:

```
import { preconnect, prefetchDNS, preinit, preload }
  from 'react-dom'

function JobDetailsPage() {
  preinit('https://cdn.ejemplo.com/editor.js', { as:
    'script' }) // descarga y ejecuta
  preload('https://cdn.ejemplo.com/inter.woff2', {
    as: 'font' }) // solo descarga
  preconnect('https://api.ejemplo.com') // abre la
    conexión antes
  prefetchDNS('https://analytics.ejemplo.com') //
    resuelve el DNS antes
  return <div>…</div>
```

```
}
```

Los frameworks como Next.js ya hacen mucho de esto por vos, pero es bueno saber de dónde sale.

> **ℹ️ ¿Y react-helmet?**
>
> Para casos simples ya no hace falta. Las librerías de metadatos siguen teniendo sentido si necesitás que un componente **pise** los metadatos de otro según especificidad de la ruta. En Next.js usás `export const metadata` o `generateMetadata` en cada `page.tsx` y `layout.tsx`.

> **🎤 En una entrevista**
>
> "¿Cómo manejás el título y los meta tags en React 19?" → "Puedo renderizar title, meta y link directamente en el componente y React los sube al head, también con SSR. En Next.js uso la API de metadata del framework. Para estilos, React 19 permite renderizar link de stylesheets con precedence: los ordena, los deduplica y espera a que carguen."

Capítulo 22

Errores y migración desde código legacy

React 19.0

Mejor reporte de errores, APIs eliminadas y cómo migrar un proyecto viejo.

Este capítulo es para cuando te toque **mantener o migrar** un proyecto viejo, algo muy probable en tu próximo trabajo. Primero, una mejora que vas a querer usar en cualquier proyecto nuevo: el reporte de errores.

Mejor manejo de errores

En React 18, un error atrapado por un Error Boundary se logueaba **dos veces** y con información confusa. React 19 lo ordena y agrega tres callbacks en `createRoot` para centralizar el reporte (por ejemplo, a Sentry):

Opción	Cuándo se llama
`onCaughtError`	Un Error Boundary atrapó el error. La app sigue funcionando.
`onUncaughtError`	Nadie lo atrapó. React desmonta la app.
`onRecoverableError`	React se recuperó solo (por ejemplo, un error de hidratación que resolvió renderizando en el cliente).

Así lo usa esta misma guía:

src/main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import '@fontsource-variable/inter'
import '@fontsource-variable/jetbrains-mono'
import './styles/global.css'
import App from './App.tsx'

// React 19: createRoot acepta callbacks para
  centralizar el reporte de errores.
// En una app real, acá mandarías los errores a
  Sentry, Datadog, etc.
createRoot(document.getElementById('root')!, {
  // Errores que atrapó un Error Boundary (la UI
    muestra su fallback).
  onCaughtError(error, errorInfo) {
    console.warn('[Error atrapado por un Error
      Boundary]', error, errorInfo.componentStack)
  },
  // Errores que ningún Error Boundary atrapó: React
    desmonta toda la app.
```

```
  onUncaughtError(error, errorInfo) {
    console.error('[Error no atrapado]', error,
      errorInfo.componentStack)
  },
}).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Abrí la consola del navegador y hacé explotar la demo del capítulo de Error Boundaries: vas a ver el `console.warn` de `onCaughtError`.

Además, los errores de hidratación (cuando el HTML del servidor no coincide con el del cliente) ahora muestran **un solo mensaje con el diff** de lo que no coincide, en vez de varios errores crípticos.

APIs eliminadas en React 19

Eliminado	Usá en su lugar
`ReactDOM.render`, `ReactDOM.hydrate`	`createRoot`, `hydrateRoot` de `react-dom/client`
`unmountComponentAtNode`	`root.unmount()`
`ReactDOM.findDOMNode`	Refs
`propTypes` en componentes de función	TypeScript

Eliminado	Usá en su lugar
`defaultProps` en componentes de función	Parámetros por defecto en el destructuring
Refs de tipo string (`ref="input"`)	`useRef` o ref callbacks
Legacy Context (`contextTypes`, `getChildContext`)	`createContext`
`React.createFactory`	JSX
`react-test-renderer/shallow`	`react-shallow-renderer`, o mejor Testing Library
`act` desde `react-dom/test-utils`	`act` desde `react`
Builds UMD	Un CDN con ESM, como esm.sh

Deprecado (funciona, pero se va)

- `forwardRef` → `ref` como prop.
- `<Context.Provider>` → `<Context>`.
- `element.ref` → `element.props.ref`.

Cómo migrar un proyecto viejo, paso a paso

1. **Pasá a React 18.3 primero.** Es igual a 18.2 pero agrega advertencias para todo lo que se rompe

en 19. Corré la app y los tests, y arreglá los warnings.

2. **Actualizá** `react`, `react-dom`, `@types/react` y `@types/react-dom` a la última versión 19.
3. **Corré los codemods** oficiales, que automatizan la mayoría de los cambios:

```
# Cambios de React 19 (render → createRoot, string
  refs, act, etc.)
npx codemod@latest react/19/migration-recipe

# Cambios en los tipos de TypeScript
npx types-react-codemod@latest preset-19 ./src
```

4. **Si el proyecto usa Create React App**, migrá a Vite. CRA está deprecado y no se actualiza. El cambio suele ser mover `index.html` a la raíz, cambiar los scripts del `package.json` y renombrar las variables `REACT_APP_*` a `VITE_*`.
5. **Revisá las librerías**: las que dependen de `findDOMNode` o de APIs eliminadas necesitan una versión nueva.
6. **Activá el React Compiler** al final, cuando todo funcione, y con su regla de lint para detectar código que rompe las reglas de React.

> **💡 No hace falta reescribir las clases**
>
> Los componentes de clase **siguen funcionando** en React 19. No hace falta migrarlos todos de golpe: convertí a funciones los que tengas que tocar por otro motivo. La chuleta tiene la equivalencia de cada método.

Cambios de TypeScript que te vas a cruzar

```
// useRef necesita un argumento
const ref = useRef<HTMLInputElement>(null)

// Los ref callbacks no pueden devolver cualquier
  cosa
<div ref={(node) => { instance = node }} />

// El namespace JSX global ahora es React.JSX
function Icon(): React.JSX.Element { /* ... */ }

// Las props de un ReactElement sin tipar son
  unknown (antes any)
const element: ReactElement<{ title: string }> =
  <Card title="Hola" />
```

🎤 En una entrevista

"¿Cómo migrarías una app de React 17 a 19?" → "Primero subo a React 18.3, que avisa todo lo que se rompe en 19, y arreglo esos warnings. Después actualizo a 19 con los codemods oficiales para createRoot, string refs y tipos. Si usa Create React App, aprovecho para pasar a Vite. Las clases las dejo como están salvo que las tenga que tocar, y al final activo el React Compiler con su regla de lint."

Parte 4 · React 19.1 → 19.3 y el Compiler

Lo que llegó después de React 19: Activity, useEffectEvent, View Transitions, Fragment refs y el React Compiler.

Capítulo 23

<Activity>

React 19.2

Ocultar partes de la UI sin perder su estado.

Hasta ahora, para ocultar una parte de la UI había dos opciones, y las dos tenían un costo:

- **Render condicional** (`{show && <Panel />}`): el componente se desmonta y **pierde su estado**.
- **Ocultarlo con CSS** (`display: none`): conserva el estado, pero **sus Effects siguen corriendo** (timers, suscripciones, videos) aunque nadie lo vea.

React 19.2 agregó un tercer camino: `<Activity>`.

```
<Activity mode={isVisible ? 'visible' : 'hidden'}>
  <Sidebar />
</Activity>
```

Cuando `mode` es `"hidden"`, React:

1. Oculta los hijos con `display: none`.
2. **Destruye sus Effects** (ejecuta los cleanups): se cortan timers y suscripciones.
3. **Conserva su estado** (y el del DOM, como el texto de un textarea).
4. Los sigue renderizando en segundo plano, con baja prioridad, si cambian sus props.

Cuando vuelve a `"visible"`, React muestra los hijos con el estado intacto y vuelve a crear sus Effects.

DEMO **Pestañas que recuerdan su estado**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/ActivityTabs.tsx`

```
import { Activity, useEffect, useState, type
  Dispatch, type SetStateAction } from 'react'
```

```tsx
type Tab = 'notas' | 'preguntas' | 'timer'
type Log = Dispatch<SetStateAction<string[]>>

function NotesTab() {
  const [text, setText] = useState('')
  return <textarea rows={3} value={text} onChange=
    {(e) => setText(e.target.value)}
    placeholder="Notas de la entrevista…" />
}

function QuestionsTab() {
  const [count, setCount] = useState(0)
  return <button onClick={() => setCount((n) => n +
    1)}>Preguntas practicadas: {count}</button>
}

function TimerTab({ onLog }: { onLog: Log }) {
  const [seconds, setSeconds] = useState(0)
  useEffect(() => {
    onLog((log) => [...log, '▶ effect: el timer
      arranca'])
    const id = setInterval(() => setSeconds((s) => s
      + 1), 1000)
    return () => {
      clearInterval(id)
      onLog((log) => [...log, '■ cleanup: el timer
        se detiene'])
    }
  }, [onLog])
  return <p>Tiempo practicando: {seconds} s</p>
}

export default function ActivityTabs() {
  const [tab, setTab] = useState<Tab>('notas')
  const [useActivity, setUseActivity] =
    useState(true)
  const [log, setLog] = useState<string[]>([])
```

```
  const mode = (t: Tab) => (tab === t ? 'visible' :
    'hidden')

  return (
    <div className="stack">
      <label className="row">
        <input type="checkbox" checked={useActivity}
          onChange={(e) =>
          setUseActivity(e.target.checked)} />
        Usar {'<Activity>'} (si no, render
          condicional)
      </label>
      <div className="row">
        {(['notas', 'preguntas', 'timer'] as
          const).map((t) => (
          <button key={t} className={tab === t ?
            'active' : ''} onClick={() =>
            setTab(t)}>
            {t}
          </button>
        ))}
      </div>

      <div className="card">
        {useActivity ? (
          <>
            {/* Ocultas: display none + Effects
              destruidos, pero el ESTADO se conserva
              */}
            <Activity mode={mode('notas')}>
              <NotesTab />
            </Activity>
            <Activity mode={mode('preguntas')}>
              <QuestionsTab />
            </Activity>
            <Activity mode={mode('timer')}>
              <TimerTab onLog={setLog} />
            </Activity>
          </>
```

```
        ) : (
          <>
            {/* Render condicional: al salir de la
              pestaña, el componente se desmonta */}
            {tab === 'notas' && <NotesTab />}
            {tab === 'preguntas' && <QuestionsTab
              />}
            {tab === 'timer' && <TimerTab onLog=
              {setLog} />}
          </>
        )}
      </div>
      <ol className="log">
        {log.map((line, i) => (
          <li key={i}>{line}</li>
        ))}
      </ol>
    </div>
  )
}
```

🧪 Probá esto

1. Con `<Activity>` activado, escribí algo en **notas**, sumá preguntas en **preguntas** y dejá correr el **timer** unos segundos. Cambiá de pestaña y volvé: todo sigue ahí. En el log ves que al salir del timer su Effect se limpia (el timer se pausa) y al volver arranca de nuevo desde donde estaba.
2. Desactivá `<Activity>` y repetí: cada vez que salís de una pestaña, su estado se pierde y el timer vuelve a 0.

Para qué sirve

Caso de uso	Por qué Activity
Pestañas	Volver a una pestaña sin perder el scroll, los filtros ni lo escrito.
Sidebars y paneles	Cerrar y abrir sin reiniciar su contenido.
Navegación "atrás"	Volver a la pantalla anterior al instante, con su estado.
Pre-renderizar	Renderizar en `hidden` una pantalla que el usuario probablemente visite después, para que aparezca al instante.

> 💡 **Pre-renderizar lo próximo**
>
> Como el contenido oculto se renderiza con baja prioridad, podés montar en `hidden` la siguiente pantalla (y empezar a cargar sus datos con Suspense) sin afectar a la pantalla actual. Cuando el usuario navega, ya está lista.

Detalles a tener en cuenta

- Los Effects se destruyen al ocultar: si un componente oculto tiene que seguir haciendo algo (por ejemplo, reproducir música), Activity no es la herramienta.
- Mantener muchas pantallas ocultas consume memoria. Usalo para lo que es probable que el usuario vuelva a ver.
- Un `<Activity>` oculto que solo renderiza texto no muestra nada, porque no hay un elemento DOM al que aplicarle `display: none`.
- Dentro de un `<ViewTransition>`, pasar de oculto a visible dentro de una transición activa la animación de entrada, y al revés la de salida.

ℹ️ El nombre

Durante años esta API se desarrolló con el nombre `Offscreen`. Si lo ves en algún artículo viejo, es lo mismo.

🎤 En una entrevista

"¿Qué es Activity en React 19.2?" → "Un componente para ocultar parte de la UI sin desmontarla. Con mode hidden, React aplica display none, destruye los Effects para que no consuman recursos, pero conserva el estado. Al volver a visible, restaura todo y recrea los Effects. Sirve para pestañas, sidebars, volver atrás rápido y pre-renderizar la próxima pantalla."

Capítulo 24

useEffectEvent

React 19.2

Leer el valor más reciente dentro de un Effect sin volver a ejecutarlo.

Este hook resuelve uno de los problemas más molestos de `useEffect`: necesitás **leer el valor más reciente** de algo dentro de un Effect, pero **no querés que el Effect se vuelva a ejecutar** cada vez que ese valor cambia.

El problema

Un chat se conecta a una sala. Cuando la conexión se establece, muestra una notificación con el tema actual (claro u oscuro). El Effect depende de `roomId`, pero como lee `theme`, el linter te obliga a agregarlo a las dependencias:

```
useEffect(() => {
  const connection = createConnection(roomId)
```

```
  connection.on('connected', () => {
    showNotification('¡Conectado!', theme) // lee
      theme…
  })
  connection.connect()
  return () => connection.disconnect()
}, [roomId, theme]) // …entonces cambiar el tema
  RECONECTA el chat 🔴
```

Cambiar de tema no tiene nada que ver con la conexión. Antes había que elegir entre desactivar el linter (y arriesgarse a bugs) o hacer malabares con refs.

La solución: un Effect Event

`useEffectEvent` crea una función que **siempre ve los valores más recientes** de props y estado, pero que **no es reactiva**: no va en las dependencias y no hace que el Effect se re-ejecute.

```
const onConnected = useEffectEvent(() => {
  showNotification('¡Conectado!', theme) // siempre
    el theme actual
})

useEffect(() => {
  const connection = createConnection(roomId)
  connection.on('connected', () => onConnected())
  connection.connect()
  return () => connection.disconnect()
```

```
}, [roomId]) // ✅ solo se reconecta si cambia la
  sala
```

DEMO **Cambiar el tema sin reconectar**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/ChatRoomEffectEvent.tsx`

```
import { useEffect, useEffectEvent, useState, type
  Dispatch, type SetStateAction } from 'react'

type Log = Dispatch<SetStateAction<string[]>>
type Theme = 'claro' | 'oscuro'
type RoomProps = { roomId: string; theme: Theme;
  onLog: Log }

// Conexión simulada: avisa "connected" a los 400 ms
function createConnection(roomId: string) {
  let timer: number | undefined
  let onConnected = () => {}
  return {
    on(_event: 'connected', callback: () => void) {
      onConnected = callback
    },
    connect() {
      timer = window.setTimeout(() => onConnected(),
        400)
    },
    disconnect() {
      clearTimeout(timer)
      void roomId
    },
  }
```

```
}

// 🔴 Sin useEffectEvent: theme es dependencia, así
  que cambiar el tema RECONECTA
function ChatRoomWithThemeDependency({ roomId,
  theme, onLog }: RoomProps) {
  useEffect(() => {
    onLog((log) => [...log, `▶ conectando a
      #${roomId}…`])
    const connection = createConnection(roomId)
    connection.on('connected', () => {
      onLog((log) => [...log, `🔔 [tema ${theme}]
        Conectado a #${roomId}`])
    })
    connection.connect()
    return () => {
      connection.disconnect()
      onLog((log) => [...log, `■ desconectado de
        #${roomId}`])
    }
  }, [roomId, theme, onLog])
  return null
}

// ✅ Con useEffectEvent: lee el theme más reciente
  sin ser dependencia
function ChatRoomWithEffectEvent({ roomId, theme,
  onLog }: RoomProps) {
  const onConnected = useEffectEvent(() => {
    onLog((log) => [...log, `🔔 [tema ${theme}]
      Conectado a #${roomId}`])
  })

  useEffect(() => {
    onLog((log) => [...log, `▶ conectando a
      #${roomId}…`])
    const connection = createConnection(roomId)
    connection.on('connected', () => onConnected())
    connection.connect()
```

```
    return () => {
      connection.disconnect()
      onLog((log) => [...log, `■ desconectado de
        #${roomId}`])
    }
  }, [roomId, onLog]) // theme ya no está acá

  return null
}

export default function ChatRoomEffectEvent() {
  const [roomId, setRoomId] = useState('frontend')
  const [theme, setTheme] = useState<Theme>('claro')
  const [withEffectEvent, setWithEffectEvent] =
    useState(true)
  const [log, setLog] = useState<string[]>([])

  const Room = withEffectEvent ?
    ChatRoomWithEffectEvent :
    ChatRoomWithThemeDependency

  return (
    <div className="stack">
      <label className="row">
        <input type="checkbox" checked=
          {withEffectEvent} onChange={(e) =>
          setWithEffectEvent(e.target.checked)} />
        Usar useEffectEvent
      </label>
      <div className="row">
        <select value={roomId} onChange={(e) =>
          setRoomId(e.target.value)}>
          <option
            value="frontend">#frontend</option>
          <option value="react">#react</option>
          <option value="empleos">#empleos</option>
        </select>
        <button onClick={() => setTheme(theme ===
          'claro' ? 'oscuro' : 'claro')}>Tema:
```

```
          {theme}</button>
        <button onClick={() => setLog([])}>Limpiar
          log</button>
      </div>
      <Room roomId={roomId} theme={theme} onLog=
        {setLog} />
      <ol className="log">
        {log.map((line, i) => (
          <li key={i}>{line}</li>
        ))}
      </ol>
    </div>
  )
}
```

🧪 Probá esto

1. Con `useEffectEvent` activado, cambiá el **tema** varias veces: no aparece ningún "desconectado" en el log. Cambiá la **sala**: ahí sí se reconecta, y la notificación muestra el tema actual.
2. Desactivá `useEffectEvent` y cambiá el tema: cada click desconecta y reconecta el chat. Ese es el bug.

Una forma de pensarlo

	Reactivo	No reactivo
Qué es	Código que tiene que re-sincronizarse cuando un valor cambia	Código que reacciona a un **evento**, leyendo los valores de ese momento
Dónde va	El cuerpo del Effect, con sus dependencias	Un Effect Event
Ejemplo	Conectarse a `roomId`	Mostrar una notificación con el `theme` actual

Por eso se llama "Effect **Event**": se parece más a un event handler (que siempre ve los valores actuales) que a un Effect.

Reglas

- **Solo se llama desde adentro de Effects** (o de otros Effect Events del mismo componente). No lo pases a otros componentes ni lo llames durante el render.
- **No lo pongas en las dependencias**. Su identidad cambia a propósito en cada render, justamente para que no se use como dependencia. El linter de React lo sabe.
- **No lo uses para "silenciar" el linter.** Si un valor tiene que re-sincronizar el Effect, va en las

dependencias. Usá Effect Events solo para lógica que realmente es un evento.

💡 Casos típicos

- Analytics: registrar una visita a una página con el carrito actual, pero solo cuando cambia la URL.
- Notificaciones o toasts que leen el idioma o el tema actual.
- Callbacks que llegan por props (`onTick` , `onMessage`) y que no querés que reinicien una suscripción.

🎤 En una entrevista

"¿Para qué sirve useEffectEvent?" → "Para separar la lógica no reactiva de un Effect. Crea una función que siempre ve las props y el estado más recientes, pero que no va en las dependencias, así que no hace que el Effect se vuelva a ejecutar. El ejemplo clásico es un chat que se reconecta solo cuando cambia la sala, aunque la notificación de conexión use el tema actual."

Capítulo 25

<ViewTransition> y addTransitionType

React 19.3

Animaciones de entrada, salida y reordenamiento con la View Transitions API.

React 19.3 incorpora **animaciones** de forma nativa. `<ViewTransition>` usa la View Transitions API del navegador para animar componentes cuando entran, salen, se mueven o cambian, y React decide cuándo y qué animar.

```
import { ViewTransition, startTransition } from
  'react'

<ViewTransition>
  <JobCard job={job} />
</ViewTransition>
```

Qué activa una animación

Esta es la regla más importante: **un `setState` común no anima nada**. Solo animan los cambios que ocurren dentro de:

- una **Transition** (`startTransition`, `useTransition`, una Action, un `<form action>`),
- un `<Suspense>` que revela su contenido,
- o un `useDeferredValue`.

Tiene sentido: las transiciones son las actualizaciones "no urgentes", y una animación de 300 ms es justamente algo que no querés en una tecla del input.

Los cuatro tipos de animación

Tipo	Cuándo se activa
`enter`	El `<ViewTransition>` se monta en la transición.
`exit`	El `<ViewTransition>` se desmonta en la transición.
`update`	Cambia algo adentro, o el boundary cambia de tamaño o posición (por ejemplo, al reordenar una lista).
`share`	Un `<ViewTransition>` con `name` se desmonta y otro con el mismo `name` se monta: "elemento compartido" que vuela de un lugar a otro.

Por defecto todas hacen un **cross-fade**. Para personalizarlas, pasás una **clase** a cada prop (`enter="vt-slide-in"`) y la estilás con los pseudo-elementos de la View Transitions API.

DEMO **Agregar, quitar y reordenar con animación**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/ViewTransitionList.tsx`

```
import { startTransition, useState, ViewTransition }
  from 'react'
import './view-transitions.css'

type Job = { id: number; company: string; role:
  string }

const pool = ['Nubelia', 'Pagos Sur', 'Datalab',
  'Kuarahy', 'Andes Cloud', 'Yvága', 'Mburucuyá
  Tech']

export default function ViewTransitionList() {
  const [jobs, setJobs] = useState<Job[]>([
    { id: 1, company: 'Nubelia', role: 'Frontend
      Engineer' },
    { id: 2, company: 'Pagos Sur', role: 'React
      Developer' },
    { id: 3, company: 'Datalab', role: 'UI Engineer'
      },
  ])
```

```
// Las animaciones solo se activan con updates
  dentro de una Transition
function add() {
  startTransition(() => {
    const company = pool[Math.floor(Math.random()
      * pool.length)]
    setJobs((list) => [{ id: Date.now(), company,
      role: 'Frontend' }, ...list])
  })
}

function shuffle() {
  startTransition(() => setJobs((list) =>
    list.toSorted(() => Math.random() - 0.5)))
}

function remove(id: number) {
  startTransition(() => setJobs((list) =>
    list.filter((job) => job.id !== id)))
}

return (
  <div className="stack">
    <div className="row">
      <button onClick={add}>➕ Agregar</button>
      <button onClick={shuffle}>🔀
        Mezclar</button>
    </div>
    <div className="stack">
      {jobs.map((job) => (
        // enter / exit: clases CSS para entrar y
          salir. Al reordenar se anima solo
          ("update").
        <ViewTransition key={job.id} enter="vt-
          slide-in" exit="vt-slide-out">
          <div className="card row" style={{
            justifyContent: 'space-between' }}>
            <span>
```

```
                <strong>{job.company}</strong> ·
                  {job.role}
              </span>
              <button onClick={() => remove(job.id)}
                aria-label={`Quitar
                ${job.company}`}>
                ×
              </button>
            </div>
          </ViewTransition>
        ))}
      </div>
    </div>
  )
}
```

src/demos/novedades/view-transitions.css

```
/* Clases de View Transition que usan las demos de
  <ViewTransition>.
   React las aplica a los elementos cuando la
     animación se activa. */

/* Entrada y salida de un elemento de la lista */
::view-transition-new(.vt-slide-in):only-child {
  animation: 320ms ease-out both vt-slide-in;
}
::view-transition-old(.vt-slide-out):only-child {
  animation: 220ms ease-in both vt-slide-out;
}

/* Carrusel: la dirección depende del Transition
  Type */
::view-transition-new(.vt-from-right) {
  animation: 300ms ease-out both vt-from-right;
}
::view-transition-old(.vt-to-left) {
  animation: 300ms ease-out both vt-to-left;
```

```
}
::view-transition-new(.vt-from-left) {
  animation: 300ms ease-out both vt-from-left;
}
::view-transition-old(.vt-to-right) {
  animation: 300ms ease-out both vt-to-right;
}

@keyframes vt-slide-in {
  from {
    opacity: 0;
    transform: translateX(-32px);
  }
}
@keyframes vt-slide-out {
  to {
    opacity: 0;
    transform: translateX(32px) scale(0.96);
  }
}
@keyframes vt-from-right {
  from {
    transform: translateX(60%);
    opacity: 0;
  }
}
@keyframes vt-to-left {
  to {
    transform: translateX(-60%);
    opacity: 0;
  }
}
@keyframes vt-from-left {
  from {
    transform: translateX(-60%);
    opacity: 0;
  }
}
```

```css
@keyframes vt-to-right {
  to {
    transform: translateX(60%);
    opacity: 0;
  }
}

/* Siempre respetá a quien prefiere menos movimiento
  */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-old(*),
  ::view-transition-new(*),
  ::view-transition-group(*) {
    animation: none !important;
  }
}
```

🧪 Probá esto

Agregá un par de ofertas, quitá alguna y tocá **Mezclar**: las tarjetas se deslizan a su nueva posición. Ahora, en el código, sacá el `startTransition` de la función `shuffle` y probá de nuevo: el orden cambia de golpe, sin animación.

Transition Types: animar distinto según la causa

Una misma actualización puede necesitar animaciones distintas según **por qué** ocurrió: avanzar o retroceder en un carrusel, navegar hacia adelante o hacia atrás. `addTransitionType` le agrega un "tipo" a la transición y `<ViewTransition>` acepta un objeto que mapea cada tipo a una clase.

DEMO **Carrusel que se desliza según la dirección**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/TransitionTypes.tsx`

```
import { addTransitionType, startTransition,
  useState, ViewTransition } from 'react'
import './view-transitions.css'

const offers = [
  { company: 'Nubelia', role: 'Frontend Engineer',
    salary: 'USD 3.200' },
  { company: 'Datalab', role: 'UI Engineer', salary:
    'USD 4.100' },
  { company: 'Pagos Sur', role: 'React Developer',
    salary: 'USD 2.800' },
  { company: 'Andes Cloud', role: 'Frontend Lead',
    salary: 'USD 5.000' },
]
```

```
export default function TransitionTypes() {
  const [index, setIndex] = useState(0)
  const offer = offers[index]

  function go(direction: 1 | -1) {
    startTransition(() => {
      // Le decimos a React POR QUÉ ocurre esta
        transición
      addTransitionType(direction === 1 ?
        'siguiente' : 'anterior')
      setIndex((i) => (i + direction +
        offers.length) % offers.length)
    })
  }

  return (
    <div className="stack">
      <div style={{ overflow: 'hidden',
        borderRadius: 10 }}>
        {/* key distinta = componente nuevo: el
          viejo sale (exit) y el nuevo entra (enter)
          */}
        <ViewTransition
          key={index}
          enter={{ siguiente: 'vt-from-right',
            anterior: 'vt-from-left', default:
            'none' }}
          exit={{ siguiente: 'vt-to-left', anterior:
            'vt-to-right', default: 'none' }}
        >
          <div className="card" style={{ minHeight:
            96 }}>
            <p className="muted">
              Oferta destacada {index + 1} de
                {offers.length}
            </p>
            <strong>
              {offer.role} · {offer.company}
```

```
            </strong>
            <p>{offer.salary}</p>
          </div>
        </ViewTransition>
      </div>
      <div className="row">
        <button onClick={() => go(-1)}>←
          Anterior</button>
        <button onClick={() => go(1)}>Siguiente
          →</button>
      </div>
    </div>
  )
}
```

src/demos/novedades/view-transitions.css

```
/* Clases de View Transition que usan las demos de
  <ViewTransition>.
   React las aplica a los elementos cuando la
     animación se activa. */

/* Entrada y salida de un elemento de la lista */
::view-transition-new(.vt-slide-in):only-child {
  animation: 320ms ease-out both vt-slide-in;
}
::view-transition-old(.vt-slide-out):only-child {
  animation: 220ms ease-in both vt-slide-out;
}

/* Carrusel: la dirección depende del Transition
  Type */
::view-transition-new(.vt-from-right) {
  animation: 300ms ease-out both vt-from-right;
}
::view-transition-old(.vt-to-left) {
  animation: 300ms ease-out both vt-to-left;
```

```
}
::view-transition-new(.vt-from-left) {
  animation: 300ms ease-out both vt-from-left;
}
::view-transition-old(.vt-to-right) {
  animation: 300ms ease-out both vt-to-right;
}

@keyframes vt-slide-in {
  from {
    opacity: 0;
    transform: translateX(-32px);
  }
}
@keyframes vt-slide-out {
  to {
    opacity: 0;
    transform: translateX(32px) scale(0.96);
  }
}
@keyframes vt-from-right {
  from {
    transform: translateX(60%);
    opacity: 0;
  }
}
@keyframes vt-to-left {
  to {
    transform: translateX(-60%);
    opacity: 0;
  }
}
@keyframes vt-from-left {
  from {
    transform: translateX(-60%);
    opacity: 0;
  }
}
```

```
@keyframes vt-to-right {
  to {
    transform: translateX(60%);
    opacity: 0;
  }
}

/* Siempre respetá a quien prefiere menos movimiento
  */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-old(*),
  ::view-transition-new(*),
  ::view-transition-group(*) {
    animation: none !important;
  }
}
```

Esta guía lo usa

El cambio de capítulo de esta misma app es un `<ViewTransition>` con una `key` por capítulo: el capítulo viejo sale con un fade y el nuevo entra desde abajo.

src/App.tsx

```
import { startTransition, useEffect,
  useLayoutEffect, useState, ViewTransition } from
  'react'
import { chapters, findChapter } from './chapters'
import { DocPage } from './components/DocPage'
import { PrintBook } from './components/PrintBook'
import { Sidebar } from './components/Sidebar'
```

```
import { IS_PRINT } from './lib/print'

function readSlug() {
  return window.location.hash.replace(/^#\/?/, '')
}

export default function App() {
  return IS_PRINT ? <PrintBook /> : <Docs />
}

function Docs() {
  const [slug, setSlug] = useState(readSlug)
  const [menuOpen, setMenuOpen] = useState(false)
  const chapter = findChapter(slug) ?? chapters[0]

  // Router mínimo con el hash de la URL (#/use-
    state).
  useEffect(() => {
    function onHashChange() {
      setMenuOpen(false)
      // El cambio de capítulo es una Transition:
        así se activa <ViewTransition>.
      startTransition(() => setSlug(readSlug()))
    }
    window.addEventListener('hashchange',
      onHashChange)
    return () =>
      window.removeEventListener('hashchange',
      onHashChange)
  }, [])

  useLayoutEffect(() => {
    window.scrollTo(0, 0)
  }, [chapter.slug])

  return (
    <div className="layout">
      <header className="topbar">
```

```
        <button
          type="button"
          className="menu-button"
          aria-label="Abrir índice"
          aria-expanded={menuOpen}
          onClick={() => setMenuOpen((open) =>
            !open)}
        >
          ☰
        </button>
        <a href="#/" className="brand">
          <span className="brand-logo" aria-
            hidden="true">⚛</span>
          Guía de React moderno
        </a>
        <span className="topbar-versions">React 19.3
          · Vite 8</span>
        <a className="topbar-print" href="?print"
          target="_blank" rel="noreferrer">
          Versión imprimible
        </a>
      </header>

      <Sidebar current={chapter.slug} open=
        {menuOpen} />
      {menuOpen && <div className="backdrop"
        onClick={() => setMenuOpen(false)} />}

      <main className="main">
        {/* Cada capítulo entra y sale con una
          animación (React 19.3). default="none"
            evita que se anime la página entera
              cuando una demo interna cambia. */}
        <ViewTransition key={chapter.slug}
          enter="page-enter" exit="page-exit"
          default="none">
          <DocPage chapter={chapter} />
        </ViewTransition>
      </main>
```

```
    </div>
  )
}
```

> **⚠️ default='none' para no animar de más**
>
> Fijate en `default="none"` del `App.tsx`. Sin eso, cualquier transición **dentro** de un capítulo (por ejemplo, las demos de Actions) haría que toda la página se anime con un cross-fade. Con `default="none"` solo se animan la entrada y la salida del capítulo.

Buenas prácticas

- **Respetá `prefers-reduced-motion`**: React no desactiva las animaciones solo. Agregá una media query que las apague (mirá el final de `view-transitions.css`).
- **No llames a `document.startViewTransition`** por tu cuenta: React lo hace por vos y coordina las animaciones. Si ya usabas la API a mano, migrá a `<ViewTransition>`.
- **Usá `name` solo para elementos compartidos.** Para todo lo demás, React genera nombres únicos automáticamente.

- **Animaciones con JavaScript**: los eventos `onEnter`, `onExit`, `onUpdate` y `onShare` te dan acceso a la animación para controlarla con la Web Animations API.

> ℹ️ **Soporte**
>
> Funciona en los navegadores con View Transitions de mismo documento (Chrome, Edge, Safari y Firefox recientes). Donde no hay soporte, el cambio se aplica sin animación: nada se rompe. Por ahora solo funciona en el DOM, no en React Native.

> 🎤 **En una entrevista**
>
> "¿Cómo se animan componentes en React 19.3?" → "Con ViewTransition, que usa la View Transitions API del navegador. Envuelvo el componente y React activa animaciones de enter, exit, update o share cuando el cambio ocurre dentro de una transición, un Suspense o un useDeferredValue. Las personalizo con clases CSS, y con addTransitionType puedo animar distinto según la causa, por ejemplo la dirección de navegación."

Capítulo 26

Fragment refs

React 19.3

Eventos, foco y observers sobre un grupo de elementos sin un div envoltorio.

Los Fragments (`<>...</>`) agrupan elementos sin agregar un nodo al DOM. El problema aparecía cuando necesitabas **interactuar** con ese grupo: escuchar clicks, mover el foco, medirlo u observar si es visible. La única salida era envolverlo en un `<div>`, y ese div extra rompía layouts con flexbox o grid, o el HTML semántico.

React 19.3 permite pasarle un `ref` **a un Fragment**. Lo que recibís es un `FragmentInstance`: un objeto con métodos que se aplican a todos los elementos del grupo.

```
import { Fragment, useRef, type FragmentInstance }
  from 'react'
```

```tsx
const groupRef = useRef<FragmentInstance>(null)

<Fragment ref={groupRef}>
  <input />
  <input />
</Fragment>

groupRef.current?.focus() // enfoca el primer
  elemento enfocable del grupo
```

> ⚠️ **Ojo**
>
> Para pasar un `ref` (o una `key`) tenés que usar la forma larga `<Fragment>`. La forma corta `<>...</>` no acepta props.

Eventos sin un div envoltorio

`addEventListener` agrega el listener a **cada hijo DOM de primer nivel** del Fragment. Si después se agregan o quitan hijos, el Fragment actualiza los listeners solo.

DEMO **Un listener para varios botones, sin wrapper**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

src/demos/novedades/FragmentRefEvents.tsx

```tsx
import { Fragment, useEffect, useRef, useState, type
  FragmentInstance, type ReactNode } from 'react'

// Escucha los clicks de todos sus hijos SIN agregar
  un <div> envoltorio
function ClickTracker({ children, onAnyClick }: {
  children: ReactNode; onAnyClick: (e: Event) =>
  void }) {
  const fragmentRef = useRef<FragmentInstance>(null)

  useEffect(() => {
    const fragment = fragmentRef.current
    if (!fragment) return
    // addEventListener se aplica a cada hijo DOM de
      primer nivel del Fragment
    fragment.addEventListener('click', onAnyClick)
    return () =>
      fragment.removeEventListener('click',
      onAnyClick)
  }, [onAnyClick])

  return <Fragment ref={fragmentRef}>{children}
    </Fragment>
}

export default function FragmentRefEvents() {
  const [clicks, setClicks] = useState<string[]>([])

  return (
    <div className="stack">
      <div className="row">
        <ClickTracker
          onAnyClick={(e) => {
            const label = (e.currentTarget as
              HTMLElement).textContent ?? ''
            setClicks((list) => [...list, label])
```

```
          }}
        >
          <button>Nubelia</button>
          <button>Pagos Sur</button>
          <button>Datalab</button>
        </ClickTracker>
      </div>
      <p className="muted">
        Clicks registrados: {clicks.length ?
          clicks.join(', ') : 'ninguno'} ·
          Inspeccioná el DOM: no hay ningún
        div extra alrededor de los botones.
      </p>
    </div>
  )
}
```

Foco y medidas de un grupo

DEMO **Enfocar y medir un grupo de inputs**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/FragmentRefFocus.tsx`

```
import { Fragment, useRef, useState, type
  FragmentInstance } from 'react'

export default function FragmentRefFocus() {
  const groupRef = useRef<FragmentInstance>(null)
  const [info, setInfo] = useState('')
```

```
  return (
    <div className="stack">
      <div className="grid-2">
        {/* Los inputs quedan como hijos directos de
          la grilla: el Fragment no agrega DOM */}
        <Fragment ref={groupRef}>
          <input placeholder="Empresa" />
          <input placeholder="Puesto" />
          <input placeholder="Salario" />
          <input placeholder="Link de la oferta" />
        </Fragment>
      </div>
      <div className="row">
        <button onClick={() =>
          groupRef.current?.focus()}>Enfocar el
          primero</button>
        <button onClick={() =>
          groupRef.current?.focusLast()}>Enfocar el
          último</button>
        <button
          onClick={() => {
            const rects =
              groupRef.current?.getClientRects() ??
              []
            setInfo(`${rects.length} elementos, el
              primero mide
              ${Math.round(rects[0]?.width ?? 0)}px
              de ancho`)
          }}
        >
          Medir
        </button>
      </div>
      {info && <p className="muted">{info}</p>}
    </div>
  )
}
```

Los inputs son hijos directos de la grilla (`grid-2`), así que el layout no cambia por usar el Fragment. Con un `<div>` envoltorio, los cuatro inputs quedarían dentro de una sola celda.

Los métodos de FragmentInstance

Método	Qué hace
`addEventListener`, `removeEventListener`	Agregar o quitar listeners en todos los hijos de primer nivel.
`dispatchEvent`	Disparar un evento desde el Fragment; burbujea hacia el padre.
`focus`, `focusLast`, `blur`	Manejar el foco buscando en todos los hijos, en profundidad.
`observeUsing`, `unobserveUsing`	Conectar un `IntersectionObserver` o un `ResizeObserver` a todos los hijos.
`getClientRects`	Las medidas de cada hijo de primer nivel.
`scrollIntoView`	Hacer scroll hasta el grupo.
`compareDocumentPosition`, `getRootNode`	Ubicar el grupo dentro del documento.

ℹ️ ¿Qué hijos cuenta?

Los "hijos de primer nivel" son los nodos DOM más cercanos, atravesando componentes. En `<Fragment ref={r}><div id="A" /><Wrapper><div id="B"><div id="C" /></div></Wrapper></Fragment>`, los hijos son A y B (Wrapper es un componente, así que React mira adentro), pero no C, que está dentro de B.

💡 ¿Quién lo va a usar más?

Sobre todo las **librerías de componentes**: un `<Tooltip>`, un `<FocusTrap>` o un `<Draggable>` que envuelven a sus children sin agregar DOM. En tu código de aplicación lo vas a usar de vez en cuando, por ejemplo para trackear la visibilidad de un grupo de elementos con un `IntersectionObserver`.

🎤 En una entrevista

"¿Qué son los Fragment refs?" → "Desde React 19.3 un Fragment puede recibir un ref, que devuelve un FragmentInstance con métodos para trabajar con todo el grupo de hijos: agregar listeners, manejar el foco, observarlos con IntersectionObserver o medirlos. Sirve para no tener que agregar un div envoltorio que rompa el layout."

Capítulo 27

React Compiler

React Compiler 1.0

Memoización automática en tiempo de build: menos useMemo, menos useCallback.

El **React Compiler** es una herramienta de build (no de runtime) que analiza tus componentes y hooks y les agrega **memoización automática**: guarda en caché valores calculados, funciones y JSX, y los reutiliza mientras sus datos no cambien. Es como si alguien hubiera escrito `useMemo`, `useCallback` y `memo` en todos los lugares correctos por vos.

Salió estable (**v1.0**) en octubre de 2025, después de años usándose en producción en Meta. Funciona con React 17, 18 y 19.

La demo: el mismo componente, compilado y sin compilar

Los dos paneles tienen **exactamente el mismo código**. La única diferencia es que el de la derecha empieza con la directiva `'use memo'`, que en este proyecto le indica al Compiler que lo compile. Tocá los botones y mirá qué lista parpadea:

DEMO **Sin compilar vs. con React Compiler**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/novedades/CompilerDemo.tsx`

```
import { useState } from 'react'
import { useRenderFlash } from
  '../shared/useRenderFlash'

type Job = { id: number; title: string; remote:
  boolean }

const jobs: Job[] = [
  { id: 1, title: 'Frontend Engineer · Nubelia',
    remote: true },
  { id: 2, title: 'React Developer · Pagos Sur',
    remote: false },
  { id: 3, title: 'UI Engineer · Datalab', remote:
    true },
]
```

```
// Componente hijo SIN memo: parpadea en amarillo
  cada vez que se renderiza
function RemoteJobs({ items }: { items: Job[] }) {
  const ref = useRenderFlash<HTMLUListElement>()
  return (
    <ul ref={ref} style={{ margin: 0, paddingLeft:
      '1.2em' }}>
      {items.map((job) => (
        <li key={job.id}>{job.title}</li>
      ))}
    </ul>
  )
}

// Sin compilar: cada click crea un array nuevo y un
  elemento nuevo, así que RemoteJobs se renderiza
function BoardWithoutCompiler() {
  const [clicks, setClicks] = useState(0)
  const remoteJobs = jobs.filter((job) =>
    job.remote)
  return (
    <div className="card stack">
      <strong>Sin compilar</strong>
      <button onClick={() => setClicks((n) => n +
        1)}>Clicks: {clicks}</button>
      <RemoteJobs items={remoteJobs} />
    </div>
  )
}

// Compilado: el React Compiler memoiza remoteJobs y
  el JSX de <RemoteJobs />
function BoardWithCompiler() {
  'use memo' // en modo "annotation" solo se
    compilan las funciones con esta directiva
  const [clicks, setClicks] = useState(0)
  const remoteJobs = jobs.filter((job) =>
    job.remote)
  return (
```

```
    <div className="card stack">
      <strong>Con React Compiler</strong>
      <button onClick={() => setClicks((n) => n +
        1)}>Clicks: {clicks}</button>
      <RemoteJobs items={remoteJobs} />
    </div>
  )
}

export default function CompilerDemo() {
  return (
    <div className="grid-2">
      <BoardWithoutCompiler />
      <BoardWithCompiler />
    </div>
  )
}
```

- **Sin compilar**: cada click crea un array `remoteJobs` nuevo y un elemento `<RemoteJobs>` nuevo, así que la lista se vuelve a renderizar aunque no cambió nada.
- **Compilado**: el Compiler detecta que `remoteJobs` y el JSX de la lista no dependen de `clicks` y los guarda en caché. React recibe el mismo elemento y **se saltea** el render de la lista. Sin escribir `memo` ni `useMemo`.

Si querés ver lo que genera, con `npm run dev` corriendo abrí en el navegador `http://localhost:5173/src/demos/novedades/CompilerDemo.tsx`: es el archivo ya transformado por Vite.

Dentro de `BoardWithCompiler` vas a encontrar `const $ = _c(9)`, la caché que agrega el Compiler. `BoardWithoutCompiler`, en cambio, no tiene ninguna caché.

Cómo se activa

En **Next.js** es una opción de configuración:

```
// next.config.ts
const nextConfig = {
  reactCompiler: true,
}
export default nextConfig
```

En **Vite** (con `@vitejs/plugin-react` 6 o superior) se agrega con Babel. Así está configurado este proyecto:

vite.config.ts

```
import mdx from '@mdx-js/rollup'
import babel from '@rolldown/plugin-babel'
import react, { reactCompilerPreset } from
  '@vitejs/plugin-react'
import remarkGfm from 'remark-gfm'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
```

```
  // Rutas relativas: el build funciona abierto
    desde cualquier carpeta o subruta.
  base: './',
  plugins: [
    // MDX: los capítulos de la guía son archivos
      .mdx (Markdown + JSX).
    { enforce: 'pre', ...mdx({ remarkPlugins:
      [remarkGfm] }) },
    react({ include: /\.(mdx|js|jsx|ts|tsx)$/ }),
    // React Compiler en modo "annotation": solo
      compila los componentes que
    // empiezan con la directiva "use memo". Así
      podemos comparar en la guía
    // un componente compilado contra uno sin
      compilar (capítulo React Compiler).
    // En un proyecto real usarías
      reactCompilerPreset() sin opciones.
    babel({ presets: [reactCompilerPreset({
      compilationMode: 'annotation' })] }),
  ],
  build: {
    // La guía completa (31 capítulos + demos) es un
      solo bundle de ~200 kB con gzip.
    // Para una app de documentación está bien, así
      que subimos el límite del aviso.
    chunkSizeWarningLimit: 1000,
  },
})
```

> **ℹ️ ¿Por qué este proyecto usa el modo annotation?**
>
> Con `compilationMode: 'annotation'` solo se compilan los componentes que tienen `'use memo'`. Lo usamos para que las demos de rendimiento muestren el comportamiento **sin** Compiler y podamos compararlo. En un proyecto real usás el modo por defecto, `reactCompilerPreset()` sin opciones, que compila todo automáticamente.

Las directivas

Directiva	Efecto
`'use memo'`	Pide compilar esta función (necesaria en modo `annotation`).
`'use no memo'`	Excluye esta función del Compiler. Es una salida de emergencia temporal si algo se rompe.

Requisito: seguir las Reglas de React

El Compiler asume que tu código respeta las reglas de siempre: componentes **puros**, props y estado **inmutables**, hooks en el nivel superior. Si un

componente las rompe, el Compiler simplemente **no lo optimiza** (no rompe nada), y el linter te avisa por qué.

```
# Las reglas del Compiler vienen en el plugin
  oficial de ESLint
npm install -D eslint-plugin-react-hooks@latest
```

La configuración recomendada de `eslint-plugin-react-hooks` (v6 y v7) incluye reglas nuevas basadas en el Compiler: detectan mutaciones durante el render, lecturas de refs en el render, componentes creados dentro de otros componentes, `setState` síncrono dentro de Effects y más. Este proyecto usa **oxlint**, que trae reglas equivalentes.

> **💡 ¿Entonces tiro todos mis useMemo?**
>
> No hace falta. El Compiler convive con la memoización manual. En código nuevo, escribí sin `useMemo` ni `useCallback` y dejá que el Compiler optimice. En código existente, podés ir sacándolos de a poco, con tests. Y para dependencias de Effects, a veces conviene mantener `useMemo` o `useCallback` explícitos para controlar exactamente cuándo se re-ejecutan.

🎤 En una entrevista

"¿Qué es el React Compiler?" → "Una herramienta de build, estable desde octubre de 2025, que memoiza automáticamente componentes y hooks: valores, callbacks y JSX. En la práctica reemplaza a escribir useMemo, useCallback y memo a mano. Requiere que el código respete las reglas de React, y si un componente no las respeta, simplemente no lo optimiza. En Next.js se activa con reactCompiler: true."

Capítulo 28

Otras novedades de 19.1 a 19.3

React 19.1+

Owner Stack, Performance Tracks, prerender parcial, browser() y más.

Además de las features grandes, cada versión de React 19 trajo mejoras más chicas. No hace falta dominarlas todas, pero conviene saber que existen: algunas aparecen en entrevistas y otras te van a ahorrar horas de debugging.

React 19.1 (marzo 2025)

Owner Stack

Cuando algo falla, el **component stack** clásico te dice la jerarquía de componentes hasta el error (padre, abuelo...). El **Owner Stack** te dice **quién**

renderizó cada componente, que muchas veces es lo que realmente buscás. Es solo para desarrollo:

```
import { captureOwnerStack } from 'react'

function reportProblem(message: string) {
  const ownerStack = captureOwnerStack() // null en
    producción
  console.error(message, ownerStack)
}
```

Las herramientas de error (como el overlay de Next.js) ya lo usan para mostrar mejores mensajes.

Suspense en todos lados

Se mejoró Suspense para que funcione de forma consistente en el cliente, en el servidor y durante la hidratación, y se arreglaron casos donde un fallback quedaba "congelado".

React 19.2 (octubre 2025)

Además de `<Activity>` y `useEffectEvent`:

Performance Tracks

En el panel **Performance** de las DevTools de Chrome aparecen pistas propias de React: qué se está renderizando, con qué prioridad (bloqueante o

transición), cuánto tardó cada componente y cuándo corrieron los Effects. Es la herramienta para responder "¿por qué esta pantalla es lenta?".

Prerender parcial

Una app puede **prerenderizar** la parte estática de una página (por ejemplo, en el build o en un CDN) y después **reanudar** el render para completar la parte dinámica:

```
// En el build: renderiza lo que se puede y
  "pospone" lo dinámico
const { prelude, postponed } = await prerender(<App
  />, { signal: controller.signal })

// Más tarde, por cada request: completa lo que
  faltaba
const stream = await resume(<App />, postponed)
```

Vos no vas a llamar a estas APIs directamente: son la base de features como el **Partial Prerendering** de Next.js.

cacheSignal (Server Components)

`cacheSignal()` devuelve un `AbortSignal` que se aborta cuando termina la vida útil del `cache()` del render en el servidor. Sirve para cancelar fetches que ya no hacen falta.

Detalles

- **Suspense en SSR revela en lote**: el contenido que llega por streaming se muestra en grupos, igual que en el cliente. Prepara el terreno para animar las revelaciones con `<ViewTransition>`.
- **Web Streams en Node**: `renderToReadableStream` y `prerender` también funcionan en Node.js.
- **Nuevo formato de `useId`**: los ids pasan a usar el prefijo `_r_` para ser selectores CSS válidos.
- **eslint-plugin-react-hooks 6**: soporta la "flat config" de ESLint y suma reglas basadas en el React Compiler.

React 19.3 (septiembre 2026)

Además de `<ViewTransition>`, `addTransitionType` y los Fragment refs:

browser(): contenido solo para el navegador

Algunos componentes no se pueden renderizar en el servidor porque usan APIs del navegador (`localStorage`, `window`). Antes se resolvía con un Effect y un estado `isMounted`, o desactivando el SSR. Ahora:

```
import { Suspense, use, useState } from 'react'
import { browser } from 'react-dom'

function SavedDraft() {
  use(browser('El borrador vive en localStorage'))
    // en el servidor: se muestra el fallback
  const [draft] = useState(() =>
    localStorage.getItem('draft') ?? '')
  return <p>{draft}</p>
}

<Suspense fallback={<p>Cargando borrador…</p>}>
  <SavedDraft />
</Suspense>
```

En el servidor, `use(browser())` corta el render y deja el fallback del `<Suspense>` más cercano. En el navegador devuelve `undefined` y el componente renderiza normal. En una SPA con Vite (como esta guía) no hace ninguna diferencia; en Next.js reemplaza al patrón de `useEffect(() => setMounted(true), [])`.

Transiciones independientes

Antes, varias transiciones al mismo tiempo se "enredaban" en un solo render: una transición lenta podía demorar a otra que no tenía nada que ver. Desde 19.3 cada transición se renderiza de forma independiente.

Otros cambios

- Advertencia en desarrollo si un componente parece haberse "desbloqueado" llamando a `use()` de forma condicional.
- Integración con **Trusted Types**, para apps con políticas de seguridad estrictas.
- Formularios: el `FormData` incluye el botón que envió el form (`submitter`) y el reset automático dispara `onReset`.
- Soporte para los eventos `onFullscreenChange` y `onFullscreenError`.

> **ℹ️ ¿Dónde enterarte de lo nuevo?**
>
> El blog oficial (react.dev/blog) anuncia cada versión, y el CHANGELOG del repositorio tiene el detalle completo. Para ver qué viene, las publicaciones "React Labs" cuentan en qué está trabajando el equipo.

🎤 En una entrevista

"¿Qué hay de nuevo después de React 19?" → "En 19.2 llegaron Activity, para ocultar UI conservando el estado, y useEffectEvent, para lógica no reactiva en Effects, además de las Performance Tracks y el prerender parcial. En octubre de 2025 el React Compiler llegó a 1.0. Y en 19.3 se estabilizaron ViewTransition para animaciones, los Fragment refs y browser() para contenido solo del cliente."

Parte 5 · Puente a Next.js

Lo que no se puede correr en Vite pero vas a usar todos los días en Next.js, más una chuleta final.

Capítulo 29

Server Components

React 19.0

Componentes que corren en el servidor, async, sin JavaScript en el navegador.

Todo lo que viste hasta acá corre en el navegador. Los **React Server Components** (RSC) son componentes que corren **en el servidor** (en cada request o durante el build) y cuyo código **nunca llega al navegador**. Son estables desde React 19 y son la base del App Router de Next.js.

Este capítulo no tiene demos en vivo: los Server Components necesitan un framework con servidor,

y Vite solo arma una SPA. Pero todo lo que aprendiste aplica tal cual.

Un componente async que lee datos directamente

```
// app/postulaciones/page.tsx  (Server Component: es
  el default en Next.js)
import { db } from '@/lib/db'

export default async function ApplicationsPage() {
  const applications = await
    db.application.findMany({ orderBy: { createdAt:
    'desc' } })

  return (
    <main>
      <h1>Mis postulaciones ({applications.length})
        </h1>
      <ul>
        {applications.map((app) => (
          <li key={app.id}>
            {app.company} · {app.role}
          </li>
        ))}
      </ul>
    </main>
  )
}
```

Fijate lo que **no** hay: ni `useEffect`, ni `useState`, ni `isLoading`, ni un endpoint de API intermedio. El

componente es una función `async` que espera los datos y devuelve JSX.

> 💡 **La respuesta a la Misión 1**
>
> ¿Por qué podés hacer `await` dentro del componente? Porque es un **Server Component**: corre en el servidor una sola vez por request, así que puede ser `async` y esperar datos antes de devolver el JSX. React manda el resultado ya renderizado al navegador. Un Client Component no puede ser `async`, porque se vuelve a renderizar muchas veces en el navegador.

Server vs Client Components

En Next.js, **todo componente es de servidor por defecto**. Para que uno corra en el navegador, el archivo empieza con la directiva `'use client'`:

```
'use client' // este archivo, y todo lo que importa,
  va al navegador

import { useState } from 'react'

export function StatusFilter({ onChange }: {
  onChange?: (s: string) => void }) {
  const [status, setStatus] = useState('todos')
  // ...
```

```
}
```

Necesitás...	Server Component	Client Component
Leer la base de datos, archivos o secretos	✅	❌
Que el código no pese en el bundle	✅	❌
`useState`, `useReducer`, `useEffect`	❌	✅
Event handlers (`onClick`, `onChange`)	❌	✅
APIs del navegador (`localStorage`, `window`)	❌	✅
Context (proveer o leer)	❌	✅
Ser `async`	✅	❌

La estrategia: **componentes de servidor por defecto**, y `'use client'` solo en las "hojas" interactivas (un botón, un filtro, un formulario), lo más abajo posible en el árbol.

Las reglas del límite

1. **Las props que van de Server a Client se serializan.** Pueden ser strings, números, booleanos, arrays, objetos planos, `Date`, `Map`,

`Set`, JSX, **promesas** y Server Functions. No pueden ser funciones comunes ni instancias de clases.

2. **Un Client Component no puede importar un Server Component**, pero sí recibirlo como `children` o como prop. Así podés meter contenido del servidor dentro de un layout interactivo:

```
// app/layout.tsx (Server)
<Sidebar>            {/* 'use client': tiene estado
  para abrirse y cerrarse */}
  <RecentJobs />     {/* Server Component: lee la
    base de datos */}
</Sidebar>
```

3. **Lo que es solo del servidor, que no se escape.** Importá el paquete `server-only` en los módulos con secretos o acceso a la base: si alguien los importa desde un Client Component, el build falla.

Streaming con Suspense

Si una parte de la página tarda, no hace falta esperarla para mostrar el resto. Envolvela en `<Suspense>` y React manda primero lo que está listo y después, por el mismo stream, lo que faltaba:

```
export default function DashboardPage() {
  return (
    <>
      <h1>Dashboard</h1>
      <Suspense fallback={<p>Cargando estadísticas…
        </p>}>
        <SlowStats /> {/* async: tarda 2 segundos
          */}
      </Suspense>
    </>
  )
}
```

En Next.js, el archivo `loading.tsx` de una carpeta envuelve la página en un `<Suspense>` automáticamente. Es lo que pusiste (o vas a poner) en la Misión 1.

Pasar una promesa al cliente y leerla con use()

El patrón más elegante para combinar los dos mundos: el servidor **empieza** el fetch sin esperarlo y le pasa la promesa a un Client Component, que la lee con use().

```
// app/postulaciones/[id]/page.tsx (Server)
export default async function Page({ params }: {
  params: Promise<{ id: string }> }) {
  const { id } = await params
```

```
  const notesPromise = getNotes(id) // sin await:
    arranca ya
  return (
    <Suspense fallback={<p>Cargando notas…</p>}>
      <Notes notesPromise={notesPromise} />
    </Suspense>
  )
}

// app/postulaciones/[id]/notes.tsx
'use client'
import { use } from 'react'

export function Notes({ notesPromise }: {
  notesPromise: Promise<Note[]> }) {
  const notes = use(notesPromise) // estable: viene
    del servidor
  // ...interactividad con estado
}
```

ℹ️ params es una promesa

Desde Next.js 15, `params` y `searchParams` llegan como **promesas**: hay que hacer `await params`. Es un error muy común al seguir tutoriales viejos.

⚠️ Errores típicos al empezar

- Poner `'use client'` en todos lados "para que funcione": perdés los beneficios del servidor.
- Usar `useState` o `onClick` en un Server Component: el error dice que falta `'use client'`.
- Pasar una función común como prop de Server a Client: no se puede serializar. Usá una Server Function o mové la lógica al cliente.
- Pensar que `'use server'` marca Server Components: **no**. `'use server'` marca **Server Functions** (siguiente capítulo). Los Server Components no llevan ninguna directiva.

🎤 En una entrevista

"¿Qué son los Server Components?" → "Componentes que se renderizan en el servidor y no mandan su JavaScript al navegador. Pueden ser async y acceder directo a la base de datos o a secretos. En Next.js son el default; uso 'use client' solo en las partes interactivas, que necesitan estado, efectos o eventos. Las props que cruzan al cliente se serializan, y con Suspense puedo hacer streaming de las partes lentas."

Capítulo 30

Server Functions y formularios

React 19.0

"use server", formularios que funcionan sin JavaScript y cómo validar en el servidor.

Una **Server Function** es una función async marcada con `'use server'` que **corre en el servidor** pero que podés llamar desde el cliente como si fuera una función común. React y el framework se encargan del resto: crean el endpoint, serializan los argumentos, hacen el POST y te devuelven el resultado.

Cuando una Server Function se usa como `action` de un formulario o dentro de una Action, se la suele llamar **Server Action** (así la vas a ver en la documentación de Next.js).

Del demo de esta guía a Next.js

¿Te acordás de `createApplication` en el capítulo de useActionState? Para que corra en el servidor de verdad, casi no cambia:

```
// app/postulaciones/actions.ts
'use server' // todas las funciones exportadas de
  este archivo son Server Functions

import { revalidatePath } from 'next/cache'
import { z } from 'zod'
import { db } from '@/lib/db'
import { getCurrentUser } from '@/lib/auth'

const ApplicationSchema = z.object({
  company: z.string().min(1, 'Falta la empresa.'),
  role: z.string().min(1, 'Falta el puesto.'),
  salary: z.coerce.number().min(1000, 'El salario
    tiene que ser de al menos USD 1000.'),
})

export async function createApplication(prevState:
  FormState, formData: FormData): Promise<FormState>
  {
  const user = await getCurrentUser()
  if (!user) return { ...prevState, status: 'error',
    message: 'Tenés que iniciar sesión.' }

  const parsed =
    ApplicationSchema.safeParse(Object.fromEntries(f
    ormData))
  if (!parsed.success) {
    // fieldErrors: { company?: string[], role?:
      string[], ... } → en el form mostrás
      errors.company?.[0]
```

```
    return { ...prevState, status: 'error', errors:
      z.flattenError(parsed.error).fieldErrors }
  }

  await db.application.create({ data: {
    ...parsed.data, userId: user.id } })
  revalidatePath('/postulaciones') // que la lista
    del Server Component se actualice
  return { ...initialFormState, status: 'success',
    message: '¡Guardada!' }
}
```

Y el formulario es el mismo Client Component que ya conocés:

```
// app/postulaciones/new-application-form.tsx
'use client'

import { useActionState } from 'react'
import { createApplication } from './actions'

export function NewApplicationForm() {
  const [state, formAction, isPending] =
    useActionState(createApplication,
    initialFormState)
  return <form action={formAction}>{/* los mismos
    inputs con name y defaultValue */}</form>
}
```

Eso es todo. No hay `fetch`, no hay `app/api/route.ts`, no hay `JSON.stringify`.

Formularios que funcionan sin JavaScript

Si un `<form>` tiene como `action` una Server Function, el formulario funciona **incluso antes de que cargue el JavaScript** de la página (o si falla): el navegador hace un POST normal y el servidor responde. Cuando el JavaScript carga, React toma el control y el envío pasa a ser sin recarga. Esto se llama **progressive enhancement**.

Desde un Server Component ni siquiera necesitás un Client Component:

```
// app/postulaciones/[id]/page.tsx (Server
  Component)
import { archiveApplication } from '../actions'

export default async function Page({ params }: {
  params: Promise<{ id: string }> }) {
  const { id } = await params
  return (
    <form action={archiveApplication}>
      <input type="hidden" name="id" value={id} />
      <button type="submit">Archivar</button>
    </form>
  )
}
```

Seguridad: son endpoints públicos

> ⚠️ **Lo más importante de este capítulo**
>
> Una Server Function es, en la práctica, **un endpoint HTTP público**. Cualquiera puede llamarla con cualquier argumento, aunque tu UI nunca lo haga. Por eso, en cada Server Function:
>
> 1. **Verificá la sesión y los permisos** (¿este usuario puede archivar *esta* postulación?).
> 2. **Validá todos los datos** que llegan (con Zod o similar). Nunca confíes en el `FormData`.
> 3. **No devuelvas datos de más**: lo que retorna la función llega al navegador.

Después de una mutación

Querés...	En Next.js
Que una página muestre los datos nuevos	`revalidatePath('/postulaciones')`
Invalidar todo lo que usa un dato con tag	`updateTag('applications')` (Next 16)
Llevar al usuario a otra página	`redirect('/postulaciones')`

Querés...	En Next.js
Mostrar el cambio antes de que confirme el servidor	`useOptimistic` en el Client Component
Mostrar errores o mensajes	El estado de `useActionState`

El detalle de la caché de Next.js 16 (`'use cache'`, `cacheTag`, `updateTag`) lo vemos en la Misión 4 del Job Tracker.

Lo que ya sabés y vas a reutilizar

En esta guía	En Next.js
`createApplication` en `applications-api.ts`	La misma función con `'use server'`
`useActionState(createApplication, initialFormState)`	Igual
`<form action={formAction}>` con `name` y `defaultValue`	Igual, y además funciona sin JavaScript
`SubmitButton` con `useFormStatus`	Igual
`useOptimistic` para la lista de notas	Igual, combinado con `revalidatePath`

💡 Próximo paso: Misión 2 del Job Tracker

Con esta guía ya tenés todo para la Misión 2: el formulario "Nueva postulación" con `useActionState` y una Server Function que valida, guarda y revalida la lista. Empezá copiando `NewApplicationForm.tsx` y `applications-api.ts` de esta guía y adaptándolos.

🎤 En una entrevista

"¿Qué es una Server Action?" → "Una Server Function marcada con 'use server' que corre en el servidor y se puede usar como action de un formulario o llamar desde el cliente. El framework crea el endpoint y serializa los datos. Funciona con useActionState, useFormStatus y useOptimistic, y los formularios funcionan aunque no haya cargado el JavaScript. Como es un endpoint público, siempre valido los datos y verifico permisos adentro."

Capítulo 31

Chuleta y preguntas de entrevista

De clases a hooks, qué hook usar para cada caso y respuestas cortas para entrevistas.

Una página para tener a mano mientras trabajás en código legacy o antes de una entrevista.

De clases a hooks

En una clase	Con hooks
`this.state = { … }` y `this.setState`	`const [x, setX] = useState(…)` (uno por dato)
`componentDidMount`	`useEffect(() => { … }, [])`
`componentDidUpdate(prevProps)`	`useEffect(() => { … }, [dep])`

En una clase	Con hooks
`componentWillUnmount`	El `return () => { … }` (cleanup) de un `useEffect`
`shouldComponentUpdate`, `PureComponent`	`memo(Componente)`, o el React Compiler
`getDerivedStateFromProps`	Calcular durante el render, o resetear con `key`
`createRef()`	`useRef(null)`
`static contextType`, `this.context`	`useContext(Ctx)` o `use(Ctx)`
`forceUpdate()`	No existe: si lo necesitás, falta estado
HOCs (`withRouter`, `connect`) y render props	Custom hooks (`useRouter`, `useSelector`)
`componentDidCatch`, `getDerivedStateFromError`	**Sigue siendo una clase** (o `react-error-boundary`)
`getSnapshotBeforeUpdate`	Sin equivalente en hooks (caso muy raro)

¿Qué uso para...?

Necesito...	Usá
Un dato que cambia y se ve en pantalla	`useState`
Muchas formas de cambiar el mismo estado	`useReducer`
Un valor calculado a partir de otros	Una variable en el render (estado derivado)
Sincronizar con algo externo (timer, socket, API del navegador)	`useEffect`
Acceder a un nodo del DOM	`useRef`
Un valor que no se muestra (id de timer, instancia)	`useRef`
Compartir datos con un subárbol profundo	`createContext` + `useContext`
Reutilizar lógica con estado	Un custom hook
Que una actualización lenta no trabe la UI	`useTransition` o `useDeferredValue`
Llamar a una API desde un botón con estado pendiente	Una Action con `useTransition`
Un formulario con validación y mensajes	`useActionState` + `<form action>`

Necesito...	Usá
Un botón que sabe si su form se está enviando	`useFormStatus`
Mostrar el resultado antes de que confirme el servidor	`useOptimistic`
Leer una promesa (con Suspense)	`use(promise)`
Ocultar UI sin perder su estado	`<Activity>`
Leer el último valor en un Effect sin re-ejecutarlo	`useEffectEvent`
Animar cambios	`<ViewTransition>`
Ids para label/input	`useId`
Suscribirse a un store externo	`useSyncExternalStore`
Un modal fuera del contenedor	`createPortal`
Pedir datos en una app real	Server Components, TanStack Query o el loader del router

Las dos reglas de los hooks

1. **Llamalos siempre en el nivel superior** de un componente o de un custom hook: nunca dentro de `if`, loops, callbacks ni después de un

`return` temprano. (La excepción es `use`, que sí puede ir en un `if`).
2. **Llamalos solo desde componentes o custom hooks**, nunca desde funciones comunes.

React identifica cada hook por **el orden** en que se llama. Si un `if` hace que un hook se saltee en un render, todos los siguientes quedan desfasados.

Preguntas de entrevista con respuesta corta

¿Qué es el Virtual DOM y la reconciliación? React mantiene una representación del UI en memoria. Cuando cambia el estado, renderiza de nuevo, compara el árbol nuevo con el anterior (reconciliación) y aplica en el DOM real solo las diferencias.

¿Por qué las keys en las listas? Para que React sepa qué elemento es cuál entre renders y pueda mover, agregar o borrar sin perder estado. El índice como key falla si la lista se reordena.

¿Props vs estado? Las props vienen del padre y son de solo lectura. El estado es la memoria propia del componente y cambiarlo dispara un render.

¿Por qué setState no cambia el valor inmediatamente? El estado es una foto del render

actual. `setState` agenda un nuevo render. Si el valor nuevo depende del anterior, se usa `setX(prev => …)`.

¿Para qué sirve el array de dependencias de useEffect? Le dice a React con qué valores se sincroniza el efecto: si alguno cambia, ejecuta el cleanup y vuelve a correr el setup. Todo valor reactivo que se usa adentro tiene que estar.

¿Qué hace StrictMode? En desarrollo, renderiza dos veces y monta, desmonta y vuelve a montar los efectos, para detectar componentes impuros y efectos sin cleanup. No afecta a producción.

¿useMemo vs useCallback vs memo? `useMemo` memoiza un valor, `useCallback` una función y `memo` un componente entero según sus props. Con el React Compiler esto es automático.

¿Qué es el prop drilling y cómo se evita? Pasar props por componentes que no las usan. Se evita con composición (`children`), Context o un store como Zustand.

¿Controlado vs no controlado? Un input controlado toma su valor del estado de React; uno no controlado guarda el valor en el DOM y se lee con un ref o con `FormData`.

¿Qué son las Actions de React 19? Funciones async dentro de transiciones. React maneja el pending, los

errores, las actualizaciones optimistas y el reset de formularios.

¿Server Components vs Client Components? Los de servidor corren solo en el servidor, pueden ser async y no mandan JavaScript. Los de cliente (`'use client'`) tienen estado, efectos y eventos.

¿Cómo optimizarías una app lenta? Mido primero con el Profiler o las Performance Tracks. Después: bajar el estado, dividir componentes, virtualizar listas largas, code splitting con `lazy`, transiciones para lo no urgente y el React Compiler para la memoización.

¿Qué es la hidratación? Cuando el HTML viene renderizado del servidor, React en el navegador "adopta" ese HTML: conecta los event handlers y el estado sin volver a crear el DOM. Si el HTML no coincide con lo que renderiza el cliente, hay un error de hidratación.

Qué sigue

1. **Misión 1 del Job Tracker**: crear el proyecto en Next.js y listar postulaciones con Server Components y `loading.tsx`.
2. **Misión 2**: el formulario con `useActionState` y una Server Function.
3. **Misión 3**: `useOptimistic` y `useFormStatus`.

4. **Misión 4**: caché y revalidación en Next.js 16.
5. **Misión 5**: deploy en Vercel y un README que venda.

Volvé a esta guía cada vez que un hook no se comporte como esperabas: casi siempre la respuesta está en alguna de las demos.